

Randomized Algorithms for Low-Rank Matrix and Tensor Decompositions

Katherine J. Pearce and Per-Gunnar Martinsson

Abstract This paper surveys randomized algorithms in numerical linear algebra for low-rank decompositions of matrices and tensors. The survey begins with a review of classical matrix algorithms that can be accelerated by randomized dimension reduction, such as the singular value decomposition (SVD) or interpolative (ID) and CUR decompositions. Recent advances in randomized dimensionality reduction are discussed, including new methods of fast matrix sketching and sampling techniques. Randomized dimension reduction maps are incorporated into classical matrix algorithms for fast low-rank matrix approximations. The extension of randomized matrix algorithms to tensors is then explored for several low-rank tensor decompositions in the CP and Tucker formats, including the higher-order SVD, ID, and CUR decomposition.

Katherine J. Pearce
Oden Institute & Dept. of Mathematics, University of Texas at Austin, e-mail: katherine.pearce@austin.utexas.edu

Per-Gunnar Martinsson
Oden Institute & Dept. of Mathematics, University of Texas at Austin, e-mail: pgm@oden.utexas.edu

Contents

Randomized Algorithms for Low-Rank Matrix and Tensor Decompositions .	1
Katherine J. Pearce and Per-Gunnar Martinsson	
1	Introduction 4
2	Matrix Preliminaries 7
2.1	Notation 7
2.2	The Singular Value Decomposition 8
2.3	The QR Decomposition 8
2.4	The Interpolative and CUR Decompositions 9
2.4.1	QR with Column Pivoting (CPQR) 10
2.4.2	LU with Partial Pivoting 10
2.4.3	Truncated SVD for Near-Optimal Skeletons 11
3	Randomized Dimension Reduction: Sketching and Sampling 12
3.1	Randomized Matrix Sketching 13
3.2	Randomized Matrix Sampling 17
3.3	Application: Randomized DRMs for Overdetermined Least Squares 20
4	Randomized Algorithms for Low-Rank Matrix Decompositions 21
4.1	The Randomized Rangefinder 21
4.1.1	Implementation details 22
4.1.2	Error estimation and certificates of accuracy 22
4.1.3	Adaptive rangefinding 24
4.2	Randomized SVD 25
4.3	Randomized ID/CUR 27
4.3.1	With the randomized rangefinder 27
4.3.2	Adaptive algorithms for randomized ID/CUR 28
4.3.3	Algorithms based on coordinate sampling 29
5	Tensor Preliminaries 30
5.1	Notation and Definitions 31
5.2	Matrix and Tensor Products 32
5.3	Tensor Decompositions 34
5.3.1	The Canonical Polyadic (CP) Decomposition 34

	5.3.2	Tucker Decomposition	36
	5.3.3	Rank, Uniqueness, and the Choice of Tensor Representation	38
	5.3.4	Low-Rank Tensor Factorizations	39
6		Randomized Low-Rank Tensor Decompositions	41
	6.1	Randomized Algorithms for CP Decomposition	42
	6.1.1	Randomized CP-ALS with Fast Leverage Score Sampling	42
	6.2	Randomized Algorithms for Tucker Decompositions with Orthogonal Factors	44
	6.2.1	Randomized (ST-)HOSVD	46
	6.2.2	Randomized HOOI	46
	6.2.3	Randomized TSMS	49
	6.3	Randomized Algorithms for Tensor ID/CUR Decompositions	50
	6.3.1	Randomized Algorithms for Tucker Core ID/CUR	50
	6.3.2	Randomized Algorithms for Tucker Factor ID/CUR	52
	6.3.3	Randomized Algorithms for CP Factor ID/CUR	54
7		Concluding Remarks	55
		References	57

1 Introduction

Numerical linear algebra (NLA) is a cornerstone of applied mathematics, scientific computing, and data science, with nascent algorithmic implementations dating back to the beginning of recorded history. Algorithms to accomplish NLA tasks, such as solving linear systems, computing eigenvalues and singular values, orthogonalizing and factorizing matrices, among many others, have been used in applications ranging from ancient Babylonian land surveying [34, Chapter 2] to Nobel Prize-winning work on computer vision [162]. Throughout its long history, NLA algorithmic development has been driven by two competing imperatives, accuracy and efficiency, with the extant challenge to strike the right balance between them.

Origins of RNLA The relatively recent emergence of randomized algorithms marks a major paradigm shift in NLA. Though Monte Carlo methods developed by Ulam and von Neumann as part of the Manhattan Project were made public in the late 1940s and 1950s [193, 194, 272], for decades, the NLA community was largely skeptical of randomization. Numerical analysts questioned the lack of theoretical guarantees for the accuracy of outputs of randomized algorithms, as well as their reproducibility. The general consensus was that the increase in efficiency afforded by randomization meant a sacrifice in accuracy that made the algorithms inherently unreliable, particularly because Monte Carlo methods were often inaccurate as a byproduct of the central limit theorem, cf. [189, Section 1.2].

However, since the 1980s, that skepticism has been replaced by an abundance of work to develop randomized algorithms with provable accuracy and practical efficiency, a subfield of NLA that has grown into a research area in its own right, known as randomized numerical linear algebra (RNLA). More details on historical milestones in the RNLA methods of the 20th century can be found in [120, 148, 179, 189, 255, 278].

RNLA in the 21st century By the early 2000s, randomization was being utilized in ground-breaking applications, such as randomized sampling for web-searching [1, 48], including the notable Google PageRank algorithm [35, 129, 210]. Randomized sampling was also used to approximate matrix operations and compute low-rank matrix decompositions [2, 106, 214]. By the mid-2000s, a unifying framework for RNLA had begun to emerge.

Over the past two decades, RNLA has developed into a firmly established research area, garnering widespread attention from both practitioners and theorists. In 2006, a three-part series laid the theoretical groundwork for randomized sampling in approximate matrix multiplication [88], low-rank matrix decomposition (QR and SVD) [89], and CUR decomposition [90]. The same year, the separate work of [81, 82] investigated adaptive sampling for low-rank approximation, and in [236], a relative error SVD algorithm was proposed that utilized randomized subspace embeddings.

Numerical analysts and theoretical computer scientists also provided compelling empirical evidence of the improved performance of RNLA algorithms for practical applications [82, 104, 106, 175, 187, 231]. These results inspired subsequent investigations of randomized algorithms expressly for linear algebra tasks, such as preconditioning or range-finding, paired with strong probabilistic guarantees [13, 119, 120, 188, 230].

Twenty years later, the scope of research in RNLA now encompasses nearly every aspect of linear algebra, including eigenvalue problems [28, 33, 161, 176, 257, 256, 284]; general linear solvers [12, 13, 16, 19, 62, 203]; orthogonalization [18, 94, 107, 114, 141]; matrix function approximation [9, 38, 60, 70, 117, 220, 266]; trace estimation [15, 113, 121, 219, 232, 267]; and many, many more.

Why the *R* in RNLA? The rise in popularity of randomized algorithms is due in large part to the commensurate progress made in computing technology. In June 2006, the performance of the world’s top supercomputer BlueGene/L was measured at 2.806 Peta-flop/s (floating point operations per second) on the High Performance Linpack (HPL) benchmark, a program that solves a large dense system of linear equations by LU with partial pivoting. As of November 2025, that rate has grown exponentially to 1.742 Exa-flop/s, achieved by today’s top supercomputer El Capitan [250].

Modern computing architectures have made it possible to solve increasingly large-scale problems; however, as evidenced by both theory and practice, the performance of classical algorithms is limited by asymptotic complexity and the “curse of dimensionality” [24]. As a point of reference, the amount of data collected by the Event Horizon Telescope Collaboration to generate the first image of a black hole in 2019 was over 5 Petabytes [277], which is almost the full storage capacity of El Capitan. In other words, classical matrix algorithms with cubic cost in the input size, such as orthogonalization, may be untenable given the large-scale problems that are now considered in scientific investigations. For the best practical performance, it is necessary to develop algorithms and implementations that can fully leverage computing resources.

Perhaps the greatest benefit of randomization is that it gives us the ability to restructure algorithms to attenuate computational demands on whichever computing resources most hinder performance, such as flops, matrix access, or data movement. For example, randomization admits algorithms that require just a single pass over the input data [120, 172, 179, 199, 259, 260, 261, 279], which is especially advantageous for large-scale problems or streaming data. Randomization can also significantly reduce the number of arithmetic operations required by classical algorithms; e.g., for dense $m \times n$ overdetermined least squares problems with $n \ll m$, randomization yields solutions in roughly $O(mn + n^3)$ operations vs. $O(mn^2)$ with classical approaches [13, 97, 167, 209, 224, 231]. In short, randomized algorithms produce accurate solutions with high probability while reducing working storage and communication costs, offering improved asymptotic complexity, admitting straightforward parallelization and GPU acceleration, and enabling practical investigations that would be otherwise impossible. For an expanded discussion, see [189, Sections 1.3-1.4].

RNLA for tensors The successful incorporation of randomization into classical matrix algorithms inspired similar work to exploit randomization for fast tensor computations. Originally proposed in 1927 as a means of working with multi-way physical and chemical measurements [132, 133], tensors are now ubiquitous in scientific computing and data science. Applications that involve tensorial data include 3D image reconstruction [25, 109, 146, 170, 285]; signal processing [59, 67, 69, 185, 197, 239]; quantum chemistry and physics [30, 150, 217, 223, 245]; machine learning [6, 123, 142, 213, 246, 291]; high-dimensional partial differential equations (PDEs) [17, 32, 74, 151, 177]; among

many others. For detailed surveys of tensors and their applications, we refer the reader to [21, 105, 112, 157, 215].

Unfortunately, tensor practitioners suffer harsher consequences of the curse of dimensionality. For a d -way array of uniform size N , the storage and computational costs scale as N^d , quickly becoming unmanageable. As such, randomization has been essential to the development of efficient algorithms for tensor computations, popularized by several foundational works in the mid-2000s [41, 71, 92, 180]. Two decades later, tensor practitioners increasingly rely on randomization for efficient tensor computations that are backed by the strong accuracy guarantees of well-established randomized matrix algorithms. Randomized algorithms have been developed for nearly every type of low-rank tensor decomposition in the literature, including the CP format [23, 29, 100, 178, 227, 270, 274], the Tucker format [5, 58, 160, 195, 196, 262, 290], the Tensor Train format [53, 56, 101, 137, 173], as well as other specialized representations [20, 22, 47, 57, 228, 247, 273, 283, 286, 292].

Outline In the following sections, we highlight recent computational advances in randomized algorithms for low-rank matrix and tensor decompositions.

Sections 1-4 focus on the matrix setting. We begin with a review of requisite linear algebra material and classical matrix decomposition algorithms. We next survey existing methods of randomized dimension reduction in addition to promising new developments in randomized sketching and sampling. We end the first part of our manuscript by discussing recent work on randomized algorithms for low-rank matrix decompositions, which are the foundation of our randomized algorithms for tensors in part two.

Sections 5-6 contain the necessary tensor prerequisites, as well as overviews of the two major tensor formats that we consider in our work: CP and Tucker. We then summarize key developments in randomized algorithms to compute low-rank tensor decompositions in these formats.

We close with some final thoughts and an outlook on the future of RNLA.

Scope Our intent with this survey is to bring attention to important recent developments in the fast-growing body of work on RNLA. This rapid progress means that regular appraisal of the current research landscape is important for guiding new work, drawing connections to prior work, and keeping RNLA practitioners abreast of major achievements and open problems; however, it also means that important subject areas and contributions will necessarily be omitted.

We quickly attempt to rectify this by pointing the interested reader to a few references on topics beyond the scope of our work. For matrices, we do not consider the vast body of work on randomized algorithms for eigenvalues and eigenvalue problems (see, e.g., [127, 203, 237]) or optimization (see, e.g., [75, 202]). We also suppress the details of most theoretical results in favor of more background material to make our presentation self-contained; see, e.g., [44, 148, 189]. We refer the reader to [199] for a survey of RNLA that centers on software implementations, and to [78] for a survey of RNLA from the perspective of machine learning. For tensors, we do not consider the tensor train format or other more specialized tensor decompositions and refer the reader to [21, 112].

2 Matrix Preliminaries

In this section, we summarize notation and key concepts in linear algebra that are frequently referenced in our work. We introduce our notational conventions in Sect. 2.1 and summarize several fundamental matrix decompositions and algorithms to compute them in Sect. 2.2-2.4.

2.1 Notation

Throughout this manuscript, we work over the real numbers $\mathbb{R}$ or complex numbers $\mathbb{C}$ (using $\mathbb{F}$ to signify either), and we let $[m]$ denote the integers $1, 2, \dots, m$.

Vectors are denoted by bold lowercase Roman or Greek letters (e.g., $\mathbf{x}, \omega$), whereas matrices are denoted by bold uppercase letters (e.g., $\mathbf{X}, \mathbf{\Omega}$). We use $\mathbf{0}$ to denote the zero matrix and $\mathbf{I}$ for the identity matrix, with their dimensions made explicit by subscripts when needed. We generally reserve Greek letters for random vectors and matrices.

To refer to coordinates of vectors, we use parentheses with subscripts, i.e. $(\mathbf{x})_i$ refers to the i th coordinate of $\mathbf{x} \in \mathbb{F}^n$. We also adopt the notation of Golub and Van Loan [111] to reference submatrices. Namely, if $\mathbf{A} \in \mathbb{F}^{m \times n}$, and $I = [i_1, i_2, \dots, i_k] \subset [m]$ and $J = [j_1, j_2, \dots, j_l] \subset [n]$ are (row and column, resp.) index sets, then $\mathbf{A}(I, J)$ denotes the $k \times l$ matrix

$$\mathbf{A}(I, J) = \begin{bmatrix} \mathbf{A}(i_1, j_1) & \mathbf{A}(i_1, j_2) & \dots & \mathbf{A}(i_1, j_l) \\ \mathbf{A}(i_2, j_1) & \mathbf{A}(i_2, j_2) & \dots & \mathbf{A}(i_2, j_l) \\ \vdots & \vdots & & \vdots \\ \mathbf{A}(i_k, j_1) & \mathbf{A}(i_k, j_2) & \dots & \mathbf{A}(i_k, j_l) \end{bmatrix}.$$

The abbreviation $\mathbf{A}(I, :)$ refers to the submatrix $\mathbf{A}(I, [n])$, analogously for $\mathbf{A}(:, J)$.

A vector $\mathbf{x} \in \mathbb{F}^n$ is measured in the Euclidean or ℓ_2 -norm $\|\mathbf{x}\|_2 = (\sum_i |(\mathbf{x})_i|^2)^{1/2}$. A matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ may be equipped with the corresponding operator norm, $\|\mathbf{A}\|_2 = \sup_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2$, or with the Frobenius norm $\|\mathbf{A}\|_F = (\sum_{i,j} |\mathbf{A}(i, j)|^2)^{1/2}$.

The (Hermitian) transpose of $\mathbf{A} \in \mathbb{F}^{m \times n}$ is denoted by $\mathbf{A}^*$, and the Moore-Penrose pseudoinverse of $\mathbf{A}$ is denoted by $\mathbf{A}^\dagger$. A matrix $\mathbf{U}$ is said to be *orthonormal* if its columns are orthonormal, i.e. $\mathbf{U}^* \mathbf{U} = \mathbf{I}$.

The trace of a square matrix $\mathbf{A} \in \mathbb{F}^{n \times n}$ is the sum of its diagonal elements $\sum_{i=1}^n \mathbf{A}(i, i)$. For matrices $\mathbf{A}, \mathbf{B} \in \mathbb{F}^{m \times n}$, we can define the inner product $\langle \mathbf{A}, \mathbf{B} \rangle = \text{trace}(\mathbf{A}^* \mathbf{B})$, so $\|\mathbf{A}\|_F^2 = \langle \mathbf{A}, \mathbf{A} \rangle = \text{trace}(\mathbf{A}^* \mathbf{A})$.

The abbreviation *i.i.d.* stands for “independent and identically distributed” to describe random variables. The probability of a random event is denoted by $\mathbb{P}[\cdot]$, and the expectation of a random variable is given by $\mathbb{E}[\cdot]$. A random vector $\mathbf{x}$ is isotropic if $\mathbb{E}[\mathbf{x}\mathbf{x}^*] = \mathbf{I}$. If $\mathbb{F} = \mathbb{R}$, the Rademacher distribution is the uniform distribution on the set $\{\pm 1\}$, or $\text{Uniform}\{\pm 1\}$, and if $\mathbb{F} = \mathbb{C}$, a complex Rademacher random variable has the form $(\rho_1 + i\rho_2)/\sqrt{2}$, where ρ_1 and ρ_2 are *i.i.d.* $\text{Uniform}\{\pm 1\}$. The standard normal or Gaussian distribution with mean 0 and standard deviation 1 is denoted by $\mathcal{N}(0, 1)$.

2.2 The Singular Value Decomposition

Every matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ admits a *singular value decomposition (SVD)*, given by

$$\mathbf{A} = \underset{m \times n}{\mathbf{U}} \underset{m \times r}{\mathbf{\Sigma}} \underset{r \times r}{\mathbf{V}^*}, \quad (1)$$

where $r = \min(m, n)$, $\mathbf{U}$ and $\mathbf{V}$ are orthonormal, and $\mathbf{\Sigma}$ is diagonal. The columns $\{\mathbf{u}_i\}_{i=1}^r$ of $\mathbf{U}$ and $\{\mathbf{v}_i\}_{i=1}^r$ of $\mathbf{V}$, respectively, are called the left and right singular vectors of $\mathbf{A}$. The diagonal elements $\{\sigma_i\}_{i=1}^r$ of $\mathbf{\Sigma}$ are the singular values of $\mathbf{A}$, ordered so that $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$. The rank of $\mathbf{A}$ is equal to the number of nonzero singular values.

Suppose that the rank of $\mathbf{A}$ is at least k . To obtain a rank- k approximation of $\mathbf{A}$, we can truncate its SVD after the first k terms, defining $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^*$. By the Eckart-Young theorem [95], $\mathbf{A}_k$ is the best possible rank- k approximation of $\mathbf{A}$, with approximation error given by the singular values:

$$\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1} \quad \text{and} \quad \|\mathbf{A} - \mathbf{A}_k\|_F = \left(\sum_{j=k+1}^r \sigma_j^2 \right)^{1/2}. \quad (2)$$

Frequently, the Golub-Reinsch algorithm [110] is used to compute an SVD, as in the built-in MATLAB function

$$[\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}] = \text{svd}(\mathbf{A}).$$

We adopt this notation when referring to the computation of the SVD, as well as $\text{svd}(\mathbf{A}, k)$ to represent the function outputting the truncation $\mathbf{A}_k$. Golub-Reinsch and related algorithms rely on QR decompositions, covered next.

2.3 The QR Decomposition

Every matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ admits a *QR decomposition* given by

$$\mathbf{A} = \underset{m \times n}{\mathbf{Q}} \underset{m \times r}{\mathbf{R}}, \quad (3)$$

where $r = \min(m, n)$, $\mathbf{Q}$ is orthonormal and $\mathbf{R}$ is upper triangular.

Using a greedy algorithm that builds the QR factorization incrementally (e.g., with Householder reflections [135]; cf. Sect 2.4.1), we can halt the factorization after computation of the first k terms to obtain a “partial” QR factorization:

$$\mathbf{A} \approx \underset{m \times n}{\mathbf{Q}_k} \underset{m \times k}{\mathbf{R}_k}. \quad (4)$$

We let

$$[\mathbf{Q}, \mathbf{R}] = \text{qr}(\mathbf{A}) \quad \text{and} \quad [\mathbf{Q}_k, \mathbf{R}_k] = \text{qr}(\mathbf{A}, k), \quad (5)$$

denote functions that yield either the full or partial QR factorization of $\mathbf{A}$, respectively. Occasionally, if we need only the factor $\mathbf{Q}$, whose columns form an orthonormal basis for the range of $\mathbf{A}$, we will use the notation

$$\mathbf{Q} = \text{col}(\mathbf{A}) \text{ and } \mathbf{Q}_k = \text{col}(\mathbf{A}, k). \quad (6)$$

2.4 The Interpolative and CUR Decompositions

Every matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ of rank k admits *interpolative decompositions* (ID) and a *CUR decomposition* of the form

$$\begin{aligned} \mathbf{A} &= \underset{m \times k}{\mathbf{A}\mathbf{R}^\dagger} \underset{k \times n}{\mathbf{R}}, & \text{Row ID} \\ \mathbf{A} &= \underset{m \times k}{\mathbf{C}} \underset{k \times n}{\mathbf{C}^\dagger \mathbf{A}}, & \text{Column ID} \\ \mathbf{A} &= \underset{m \times k}{\mathbf{C}} \underset{k \times k}{\mathbf{U}} \underset{k \times n}{\mathbf{R}}, & \text{CUR} \end{aligned} \quad (7)$$

where

$$\mathbf{R} = \mathbf{A}(I, :)$$

for k linearly independent rows indexed by I ;

$$\mathbf{C} = \mathbf{A}(:, J)$$

for k linearly independent columns indexed by J ; and $\mathbf{U} = \mathbf{C}^\dagger \mathbf{A} \mathbf{R}^\dagger$, though in practice, $\mathbf{U}$ is often approximated by $\mathbf{A}(I, J)^\dagger$. The columns of $\mathbf{C}$ or rows of $\mathbf{R}$ are known as skeletons in an ID or CUR decomposition.

Typically, constructing the CUR decomposition is more ill-conditioned than the IDs, but a stable construction can be attained through QR factorizations of $\mathbf{C}$ and $\mathbf{R}$. Namely, let $\mathbf{Q}_\mathbf{C} = \text{col}(\mathbf{C})$ and $\mathbf{Q}_\mathbf{R} = \text{col}(\mathbf{R}^*)$. The CUR factorization is then given by

$$\mathbf{A} = \mathbf{C}\mathbf{U}\mathbf{R} = \mathbf{Q}_\mathbf{C}\mathbf{Q}_\mathbf{C}^* \mathbf{A} \mathbf{Q}_\mathbf{R} \mathbf{Q}_\mathbf{R}^*. \quad (8)$$

This stabilization tactic also works for the construction of the IDs, if needed.

The accuracy of a rank- k ID or CUR approximation depends on the choice of skeletons. It was shown in [80] that it is possible to construct an ID satisfying

$$\|\mathbf{A} - \mathbf{C}\mathbf{C}^\dagger \mathbf{A}\|_F \leq \sqrt{k+1} \|\mathbf{A} - \mathbf{A}_k\|_F, \quad (9)$$

where the coefficient $\sqrt{k+1}$ is optimal and cannot be improved; here, $\mathbf{A}_k$ is the optimal rank- k approximation given by the truncated SVD.

In practice, greedy pivoting algorithms are frequently used to choose skeletons for ID and CUR decompositions. We discuss several popular choices in the following sections, including a recently proposed algorithm that achieves (9) with the best known complexity.

2.4.1 QR with Column Pivoting (CPQR)

Column-pivoted QR (CPQR) is the algorithm behind the `qr` function from Sect. 2.3. In particular, column pivoting may be required for numerical stability, and the outputs of $\text{qr}(\mathbf{A}) = [\mathbf{Q}, \mathbf{R}, \mathbf{P}]$ represent the factorization $\mathbf{A}\mathbf{P} = \mathbf{Q}\mathbf{R}$ for some permutation matrix $\mathbf{P} \in \mathbb{R}^{n \times n}$ such that $\mathbf{P} = \mathbf{I}(:, \mathbf{P})$. This factorization can be achieved through a sequence of permutations and Householder reflections that rank-one update the “active submatrix” of $\mathbf{A} \in \mathbb{R}^{m \times n}$ by greedy pivoting, based on squared column norms.

Assuming $m \geq n$, let $\mathbf{A}^{(t)}$ represent the resulting matrix after t steps of pivoting and updating for $t = 0, \dots, n-2$ (with $\mathbf{A}^{(0)} = \mathbf{A}$). At the $(t+1)$ -th step, CPQR determines the column pivot j_{t+1} by finding the column of maximal Euclidean norm in the active submatrix,

$$j_{t+1} = \arg \max_{t+1 \leq j \leq n} \|\mathbf{A}^{(t)}([t+1 : m], j)\|_2.$$

The selected k (column) skeletons are then $[j_1, \dots, j_k]$. (Row skeletons can be computed via CPQR on $\mathbf{A}^*$.) Once a pivot entry is selected, column j_{t+1} is swapped with column $t+1$, the normalized reflection vector $\mathbf{v}$ is formed, and the lower right block of the active submatrix, starting from the $(t+1)$ -st main diagonal entry, is updated as $\mathbf{A}^{(t+1)} = \mathbf{A}^{(t)} - 2\mathbf{v}\mathbf{v}^* \mathbf{A}^{(t)}$.

In practice, CPQR chooses pivots in an order that reveals the rank of $\mathbf{A}$, so that the chosen skeletons form a good basis for its row or column space. While there are adversarial cases for which CPQR can fail to be rank-revealing (e.g., Kahan-type matrices [145, 221]), these are rarely encountered, cf. [252]. There are variations of CPQR with better theoretical guarantees, such as rank-revealing QR [50, 51] or strong rank-revealing QR [115], but they are more costly to implement.

2.4.2 LU with Partial Pivoting

The LU algorithm with row-wise partial pivoting (LUPP) yields a factorization

$$\begin{matrix} \mathbf{P} & \mathbf{A} & = & \mathbf{L} & \mathbf{U} \\ m \times m & m \times n & & m \times n & n \times n \end{matrix}, \quad (10)$$

where $\mathbf{L}$ is lower triangular with main diagonal entries equal to 1 and off-diagonal entries bounded in magnitude by 1, and $\mathbf{U}$ is upper triangular. The function $[\mathbf{L}, \mathbf{U}, \mathbf{P}] = \text{lu}(\mathbf{A})$ represents the factorization (10) obtained from row-wise partial pivoting.

More specifically, to compute an LU factorization, row pivots are selected from the active submatrix of $\mathbf{A}$ that is updated via Schur complements (e.g., [111, Algorithm 3.2.1] [251, Algorithm 21.1]). Let $\mathbf{A}^{(t)}$ be the resulting matrix after t steps of pivoting and updating for $t = 0, 1, \dots, n-2$. At the $(t+1)$ -st step of LUPP, the largest-magnitude column entry of the active submatrix is selected as the next pivot entry:

$$i_{t+1} = \arg \max_{t+1 \leq i \leq m} |\mathbf{A}^{(t)}(i, t+1)|.$$

The active submatrix is then updated: defining vectors $\mathbf{r} = [t+2 : m]$ and $\mathbf{c} = [t+2 : n]$,

$$\mathbf{A}^{(t+1)}(\mathbf{r}, \mathbf{c}) = \mathbf{A}^{(t)}(\mathbf{r}, \mathbf{c}) - \frac{\mathbf{A}^{(t)}(\mathbf{r}, t+1)\mathbf{A}^{(t)}(t+1, \mathbf{c})}{\mathbf{A}^{(t)}(t+1, t+1)}, \quad (11)$$

where $\mathbf{A}^{(t)}(t+1, t+1)$ has been updated with $\mathbf{A}^{(t)}(i_{t+1}, t+1)$ following row permutation.

This LUPP algorithm leads to an exponential upper bound on the entries of $\mathbf{U}$, which is tight for certain matrices (e.g., Kahan-type matrices [145, 221, 243]). Thus, LUPP can fail to be rank-revealing, but such adversarial inputs are rarely encountered in practice [252]. There exist more sophisticated pivoting strategies to lend rank-revealing properties to the LU factorization [49, 212, 288] with better error bounds, but at significantly greater computational cost. Fortunately, we will see in Sect. 4.3 that randomization can equip LUPP with rank-revealing properties comparable to CPQR even in adversarial cases.

2.4.3 Truncated SVD for Near-Optimal Skeletons

Previously, the tight error bound (9) had been achieved by expensive random volume sampling [80], or computing at least k SVDs to choose each pivot from an updated residual subspace. Recently, in [208], the optimal error in (9) is achieved through a single rank- k truncated SVD, with total complexity $O(mnk)$.

Let $\mathbf{Z} \in \mathbb{R}^{m \times n}$ be an arbitrary rank- k approximation of $\mathbf{A} \in \mathbb{R}^{m \times n}$. We want to find columns $\mathbf{C} \in \mathbb{R}^{m \times k}$ of $\mathbf{A}$ and an interpolation matrix $\mathbf{W} \in \mathbb{R}^{k \times n}$ such that

$$\|\mathbf{A} - \mathbf{C}\mathbf{C}^\dagger \mathbf{A}\|_F \leq \|\mathbf{A} - \mathbf{C}\mathbf{W}\|_F \leq \sqrt{k+1} \|\mathbf{A} - \mathbf{Z}\|_F. \quad (12)$$

For the optimal error, we can take $\mathbf{Z}$ to be the truncated SVD. Otherwise, if $\mathbf{Z}$ is not a rank- k SVD, we compute one singular value decomposition $\mathbf{Z} = \mathbf{U}\Sigma\mathbf{V}^*$. Define

$$\tilde{\mathbf{A}} = \mathbf{A} - \mathbf{A}\mathbf{V}\mathbf{V}^*,$$

noting that $\mathbf{Z} - \mathbf{Z}\mathbf{V}\mathbf{V}^* = \mathbf{0}$, and

$$\|\tilde{\mathbf{A}}\|_F = \|(\mathbf{A} - \mathbf{Z})(\mathbf{I} - \mathbf{V}\mathbf{V}^*)\|_F \leq \|\mathbf{A} - \mathbf{Z}\|_F.$$

Crucially, the construction of $\tilde{\mathbf{A}}$ requires only $O(mnk)$ operations.

Letting $\mathbf{W} = (\hat{\mathbf{V}}^*)^{-1}\mathbf{V}^*$, where $\hat{\mathbf{V}}^* \in \mathbb{R}^{k \times k}$ is the sub-matrix of $\mathbf{V}^* \in \mathbb{R}^{k \times n}$ corresponding to the k column skeletons in $\mathbf{C}$, we have

$$\|\mathbf{A} - \mathbf{C}\mathbf{W}\|_F = \|\tilde{\mathbf{A}} - \tilde{\mathbf{C}}\mathbf{W}\|_F,$$

for $\tilde{\mathbf{C}} = \mathbf{C} - \mathbf{A}\mathbf{V}\hat{\mathbf{V}}^*$. Since $\tilde{\mathbf{A}}$ and $\mathbf{W}$ are not affected by replacing $\mathbf{V}^*$ with any unitary transformation $\mathbf{Q}\mathbf{V}^*$, we can assume unitary transformations are applied to $\mathbf{V}^*$ so that $\hat{\mathbf{V}}^*$ is always upper-triangular. (In [208], this is accomplished via Householder reflections.)

Assuming we have selected the first $i - 1$ columns for $i \geq 1$, the approximation error at the i th step is given by

$$\tilde{\mathbf{A}}^{(i)} = \tilde{\mathbf{A}}^{(i-1)} - \tilde{\mathbf{A}}_{1:m,i}^{(i-1)} (\tilde{\mathbf{V}}^*)_{1:(k-i+1),i}^\dagger \tilde{\mathbf{V}}^*, \quad (13)$$

where $\tilde{\mathbf{V}}^* \in \mathbb{F}^{(k-i+1) \times n}$ are the last $k - i + 1$ rows of $\mathbf{V}^*$, cf. [208, Equation 5]. We choose the next column skeleton as the one that minimizes the error at the i th step, which we can do by selecting the column $j \geq i$ that minimizes the ratio

$$\|\tilde{\mathbf{A}}_{1:m,j}^{(i-1)}\|_2 / \|\tilde{\mathbf{V}}_{1:(k-i+1),j}^*\|_2. \quad (14)$$

By the upper triangularity of $\tilde{\mathbf{V}}^*$ and the relation (13),

$$\sum_{\ell=1}^n \|\tilde{\mathbf{A}}_{1:m,\ell}^{(i-1)}\|_2^2 = \|\tilde{\mathbf{A}}^{(i-1)}\|_{\mathbb{F}}^2, \quad (15)$$

and

$$\sum_{\ell=1}^n \|\tilde{\mathbf{V}}_{1:(k-i+1),\ell}^*\|_2^2 = \|\tilde{\mathbf{V}}^*\|_{\mathbb{F}}^2 = k - i + 1. \quad (16)$$

Then, using the fact that $\sum_{\ell} \alpha_{\ell} / \sum_{\ell} \beta_{\ell} \geq \min_{\ell} \alpha_{\ell} / \beta_{\ell}$ for non-negative α_{ℓ} and positive $\sum_{\ell} \beta_{\ell}$, we can upper bound (14) by the ratio of (15) and (16), to estimate the error in the i th step in terms of the $(i - 1)$ th step:

$$\|\tilde{\mathbf{A}}^{(i)}\|_{\mathbb{F}}^2 = \|\tilde{\mathbf{A}}^{(i-1)} - \tilde{\mathbf{A}}_{1:m,i}^{(i-1)} \tilde{\mathbf{V}}_{1:(k-i+1),i}^* \tilde{\mathbf{V}}^*\|_{\mathbb{F}}^2 \leq \|\tilde{\mathbf{A}}^{(i-1)}\|_{\mathbb{F}}^2 + \frac{1}{k - i + 1} \|\tilde{\mathbf{A}}^{(i-1)}\|_{\mathbb{F}}^2.$$

If we add this up over all previous steps, we have

$$\|\tilde{\mathbf{A}}^{(i)}\|_{\mathbb{F}}^2 \leq \|\tilde{\mathbf{A}}\|_{\mathbb{F}}^2 + \frac{i}{k - i + 1} \|\tilde{\mathbf{A}}\|_{\mathbb{F}}^2. \quad (17)$$

Letting $i = k$ in (17), we achieve the desired bound in (9) in $O(mnk)$ operations. Full details can be found in [208].

3 Randomized Dimension Reduction: Sketching and Sampling

In this section, we discuss ways that randomization can be integrated into classical algorithms in numerical linear algebra to compute low-rank decompositions more efficiently. Namely, we will often rely on special types of linear maps that map the row or column space of an input matrix to a smaller-dimensional space in such a way that the original geometry is roughly preserved, called a *dimension reduction map* (DRM). If the linear

map is drawn from a random matrix distribution that achieves geometry-preserving dimension reduction with high probability, it is known as a randomized DRM.

Randomized dimension reduction is frequently associated with randomized embeddings, linear maps drawn from random matrix distributions that satisfy the following condition with high probability. Let $\mathbf{A} \in \mathbb{F}^{m \times n}$ be a matrix of rank $k \leq \min(m, n)$ and $\varepsilon \in (0, 1)$ a distortion parameter. A linear map $\mathbf{\Gamma} : \mathbb{F}^m \rightarrow \mathbb{F}^d$ is an embedding of $\mathbf{A}$ with distortion ε if

$$(1 - \varepsilon)\|\mathbf{Ax}\| \leq \|\mathbf{\Gamma Ax}\| \leq (1 + \varepsilon)\|\mathbf{Ax}\| \quad \forall \mathbf{x} \in \mathbb{F}^n. \quad (18)$$

Usually, $k \leq d \ll m$, so that the application of $\mathbf{\Gamma} \in \mathbb{F}^{d \times m}$ results in significant dimension reduction. The minimal value of d required for $\mathbf{\Gamma}$ to satisfy (18) depends on the type of embedding $\mathbf{\Gamma}$, the rank k of $\mathbf{A}$, and the distortion parameter ε . As a classical example, the Johnson-Lindenstrauss lemma [144] establishes that $d \geq 8 \log k / \varepsilon^2$ is sufficient for a random matrix $\mathbf{\Gamma}$ with standard Gaussian entries to embed k points in $\mathbb{R}^m$ so that (18) is satisfied.

More generally, if $\mathcal{S}$ is any probability distribution over linear maps $\mathbb{F}^m \rightarrow \mathbb{F}^d$, then $\mathbf{\Gamma} \sim \mathcal{S}$ is a randomized embedding if (18) holds over $\mathcal{S}$ for all $\mathbf{A} \in \mathbb{F}^{m \times n}$ with at least constant probability. If a randomized embedding can be constructed to map an unknown subspace into a lower-dimensional space (i.e. without knowing anything about the geometry of the input space except its dimension), then it is said to be an oblivious subspace embedding [236]. It is often the case in practice that randomized DRMs are assumed to be randomized embeddings, but we will see in Sect. 3.1 that satisfying (18) is sufficient but not necessary to ensure the smaller-dimensional space is a good proxy for the input space.

In Sect. 3.1, we review randomized sketching, a procedure that produces random linear combinations of input matrix coordinates, which we say is “coordinate-mixing.” We then focus on dimension reduction by randomized sampling of matrix coordinates in Sect. 3.2. We end this section with an important application of randomized dimension reduction that will be relevant for our discussion of tensor decompositions: solving overdetermined least squares problems. For detailed treatments of randomized dimension reduction, the reader is referred to [44, 120, 186, 189, 278].

3.1 Randomized Matrix Sketching

The application of a randomized dimension reduction map $\mathbf{\Gamma}$ to $\mathbf{A}$ is known as randomized sketching. We reserve the term randomized sketching for random linear maps that are coordinate-mixing, vs. coordinate-sampling. The matrix product $\mathbf{\Gamma A}$ is said to be a row sketch of $\mathbf{A}$, and a column sketch $\mathbf{A\Omega}$ is defined analogously.

Ideally, the random matrix used for sketching should roughly preserve the geometry of the row or column space of $\mathbf{A}$. Randomized dimension reduction maps $\mathbf{\Gamma}$ that are frequently utilized for sketching include:

- **Gaussian matrices** ([120, Section 4.1], [189, Section 8.3], [278, Theorem 2.3]): $\Gamma \in \mathbb{R}^{d \times m}$ with entries

$$\Gamma(i, j) \sim \mathcal{N}(0, 1/d),$$

i.i.d. Gaussian entries drawn from a normal distribution with mean 0 and standard deviation $1/d$.

- **Subsampled randomized trigonometric transforms (SRTT)** ([120, Section 4.6], [189, Section 9.3], [253], [278, Theorem 2.4]): $\Gamma \in \mathbb{R}^{d \times m}$ defined as

$$\Gamma = \sqrt{\frac{m}{d}} \Pi_m \mathbf{F} \Phi \Pi_{m \rightarrow d}.$$

Here, $\Pi_{m \rightarrow d} \in \mathbb{R}^{m \times d}$ is a uniformly random selection of d out of m rows, whereas $\Pi_m \in \mathbb{R}^{m \times m}$ is a random permutation of the m rows. The matrix $\mathbf{F} \in \mathbb{R}^{m \times m}$ is a discrete Fourier transform, and $\Phi := \text{diag}(\phi_1, \dots, \phi_m)$ has *i.i.d.* Rademacher random variable entries, i.e. when $\mathbb{F} = \mathbb{R}$, entries of ± 1 chosen independently with equal probability.

- **Sparse sign matrices** ([189, Section 9.2], [204], [278, Theorem 2.5]): $\Gamma \in \mathbb{R}^{d \times m}$ defined as

$$\Gamma = \sqrt{\frac{m}{\zeta}} [\mathbf{s}_1, \dots, \mathbf{s}_m]$$

for sparsity parameter $2 \leq \zeta \leq d$, where each column $\mathbf{s}_j \in \mathbb{R}^d$ for $j = 1, \dots, m$ is filled with ζ independent Rademacher random variables at uniformly random coordinates. In other words, each column is an independent random vector having ζ nonzero entries of ± 1 (when $\mathbb{F} = \mathbb{R}$) in uniformly random positions.

- **CountSketch matrices** ([52, 192, 204]): $\Gamma \in \mathbb{R}^{d \times m}$ defined as a sparse sign matrix with sparsity parameter $\zeta = 1$. We caution that CountSketch matrices on their own can perform poorly in practice among randomized DRMs (see the required sketching dimension d in Table 1).
- **SparseStack matrices** ([44, 68, 147]): $\Gamma \in \mathbb{R}^{d \times m}$ defined as

$$\Gamma = \frac{1}{\sqrt{\zeta}} \begin{bmatrix} \rho_{1,1} \mathbf{e}_{s_{1,1}} & \cdots & \rho_{1,m} \mathbf{e}_{s_{1,m}} \\ \rho_{2,1} \mathbf{e}_{s_{2,1}} & \cdots & \rho_{2,m} \mathbf{e}_{s_{2,m}} \\ \vdots & \ddots & \vdots \\ \rho_{\zeta,1} \mathbf{e}_{s_{\zeta,1}} & \cdots & \rho_{\zeta,m} \mathbf{e}_{s_{\zeta,m}} \end{bmatrix},$$

where $d = \zeta b$ for some block size b and sparsity parameter ζ , $\rho_{i,j}$ are *i.i.d.* Rademacher variables, $\mathbf{e}_i \in \mathbb{R}^b$ is the i th standard basis vector, and $s_{i,j} \in [b]$ is chosen uniformly. In other words, the SparseStack test matrix is comprised of ζ independent copies of a (scaled) CountSketch matrix, stacked on top of one another.

- **SparseCol matrices** ([44]): For consistency with the previous definitions of row sketching matrices as subspace embeddings, we give the transpose of the SparseCol matrix in [44, Definition 4.1]. Namely, $\Gamma \in \mathbb{R}^{d \times m}$ is defined by its d rows of sparse

random vectors

$$\omega = \sqrt{\frac{m}{\zeta}} \sum_{i=1}^{\zeta} \rho_i \mathbf{e}_{s_i} \in \mathbb{R}^m, \quad (19)$$

where ζ is the sparsity parameter, $\rho_1, \dots, \rho_{\zeta}$ are *i.i.d.* Rademacher random variables, the indices $s_1, \dots, s_{\zeta}$ are sampled uniformly from $[m]$ without replacement, and $\mathbf{e}_{s_i}$ is the s_i -th standard basis vector. In other words, $\mathbf{\Gamma} \in \mathbb{R}^{d \times m}$ is the transpose of the SparseCol matrix $\mathbf{\Gamma}^* \in \mathbb{R}^{m \times d}$ given by

$$\mathbf{\Gamma}^* := \frac{1}{\sqrt{k}} [\omega_1 \dots \omega_d], \quad (20)$$

where $\omega_j \sim \omega$ are *i.i.d.*

- **SparseRTT matrices** ([44, Section 4.1], [253]): $\mathbf{\Gamma} \in \mathbb{R}^{d \times m}$ defined as

$$\mathbf{\Gamma} = \mathbf{SFD}, \quad (21)$$

where $\mathbf{D} \in \mathbb{R}^{m \times m}$ is a random diagonal matrix populated with *i.i.d.* Rademacher random variables, $\mathbf{F} \in \mathbb{R}^{m \times m}$ is a discrete Fourier or Walsh-Hadamard transform, and $\mathbf{S} \in \mathbb{R}^{d \times m}$ is the transpose of a SparseCol matrix with exactly ζ nonzero entries per column as in the SparseCol matrix above.

These distributions yield randomized DRMs that work well in practice, with theoretical guarantees summarized in Table 1. Gaussian test matrices are oblivious subspace embeddings that enjoy a superior lower bound on the requisite embedding dimension d to guarantee (18), but the costs of storage and matrix-vector products for Gaussians are much more expensive than for “structured” embeddings like SRTT or sparse sign matrices. Put simply, Gaussian sketching makes for straightforward analysis, but randomized sketching with structured random matrices makes for better performance. The reader is referred to [189, Sections 8,9] for detailed discussion.

Randomized DRM $\mathbf{\Gamma} \in \mathbb{R}^{d \times m}$	Dimension d for (18)	Storage cost	Mat-vec cost
Gaussian	$d = \Omega(k/\varepsilon^2)$	$O(md)$	$O(md)$
SRTT	$d = \Omega(k \log k/\varepsilon^2)$	$O(m \log m)$	$O(m \log d)$
Sparse sign	$d = \Omega(k \log k/\varepsilon^2), \zeta = \Omega(\log k/\varepsilon)$	$O(\zeta m \log d)$	$O(\zeta m)$
CountSketch	$d = \Omega(k^2/\varepsilon^2)$	$O(m \log d)$	$O(m)$
SparseStack	$d = \Omega(k \log k/\varepsilon^2), \zeta = \Omega(\log k/\varepsilon)$	$O(d\zeta)$	$O(\zeta m)$

Table 1 Summary of coordinate-mixing randomized DRMs commonly used for (row-)sketching rank- k matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, which are also oblivious subspace embeddings (OSE). Theoretical lower bounds on the dimension d for $\mathbf{\Gamma}$ to satisfy (18) are provided, as well as asymptotic complexities to store $\mathbf{\Gamma}$ and to apply $\mathbf{\Gamma}$ to a vector $\mathbf{x} \in \mathbb{R}^m$, cf. [44, 189, 278].

While the embedding dimensions in Table 1 guarantee the oblivious subspace embedding (OSE) property (18), it has been observed that smaller values of d are often sufficient for many linear algebra tasks in practice when utilizing any of the randomized DRMs of Table 1 (with the exception of CountSketch), e.g., $d = k + 10$, cf. Sect. 4.1. Recently, this observation was thoroughly investigated from the perspective of oblivious subspace *injections*, versus embeddings, which relaxes the OSE property to the following weaker one.

A random matrix $\mathbf{\Gamma} : \mathbb{F}^m \rightarrow \mathbb{F}^d$ is an oblivious subspace injection (OSI) of $\mathbf{A}$ with injectivity α if it satisfies the injectivity property

$$\alpha \|\mathbf{Ax}\| \leq \|\mathbf{\Gamma Ax}\| \quad \forall \mathbf{x} \in \mathbb{F}^n, \quad (22)$$

and the isotropy property that $\mathbb{E}[\|\mathbf{\Gamma Ax}\|^2] = \|\mathbf{Ax}\|^2$ for all $\mathbf{x} \in \mathbb{F}^n$. In other words, an OSI satisfies the lower bound requirement in (18), taking $\alpha = 1 - \varepsilon$, so that it does not annihilate any vector in the column space of $\mathbf{A}$. Arguably, the “injectivity” parameter α is more crucial for the performance of randomized sketching than the “dilation” parameter $(1 + \varepsilon)$ in the OSE definition [44]. All of the random matrices defined above satisfy the isotropy property, and if we focus instead on the weaker injectivity property of OSIs, we can obtain theoretical guarantees on the performance of randomized sketching that more closely align with what is observed in practice. In Table 2, we report the sketching dimension d required to satisfy the OSI property, compared to the OSE property.

With the OSI property guiding our choice of random sketching matrix, we enjoy more favorable sketching dimensions for the random matrices considered above. In the OSI framework, Gaussian random matrices are still the gold standard for approximation accuracy, but suffer the same performance drawbacks in terms of storage and matrix-vector product, or mat-vec, costs. Structured random matrices offer significantly better performance, and specifically, sparse random matrices exhibit the best empirical performance, as demonstrated by many practical investigations [61, 64, 98, 120, 147, 189, 204, 255, 278], and now also supported by the theoretical guarantees of [44, 257] summarized in Table 2.

In general, sparse random matrices are recommended for sketching a dense unstructured input matrix, due to the reduced storage costs and significant acceleration afforded by high-performance sparse arithmetic libraries. To that end, sketching with SparseRTT matrices, or sparsified SRTTs, is more computationally efficient than with standard SRTTs or Gaussians. Compared to the SRTT, the SparseRTT matrix can reduce data movement and better facilitate parallelism at the hardware level. However, the SparseStack matrix is typically faster and thus recommended over SparseRTTs whenever sparse linear algebra packages are available.

As an additional boon, the OSI values of d that are reported in Table 2 are even still too pessimistic for practical implementation purposes. In particular, the results of [44] provide compelling evidence that SparseStack matrices are the superior choice of DRM in practice; it is conjectured that SparseStack matrices can achieve the $\frac{1}{2}$ -OSI property with $d = O(k)$ and constant sparsity $\zeta = O(1)$. This conjecture is supported numerically, with $d = 2k$ and $\zeta = 4$ used in the experiments of [44] to demonstrate that the low-rank approximations obtained from the SparseStack sketches are of comparable accuracy to those obtained from Gaussian sketches.

Finally, we note that for input matrices that arise from tensors (cf. Sect. 5.3.1), we can construct Khatri-Rao sketching matrices that exploit underlying Kronecker product structure [44]. As we show in Sect. 6.1, these sketching matrices are formed as Kronecker products of isotropic random vectors. Often, these random vectors are drawn from (real) Gaussian and Rademacher distributions, but it is advised in [44] that the complex spherical distribution be used to avoid near-zero injectivity α , which can occur in worst-case scenarios. For our sketching matrix constructions in Sect. 6, we do not assume any specific isotropic distribution unless otherwise specified.

DRM $\mathbf{\Gamma} \in \mathbb{F}^{d \times m}$	Dimension d for ε -OSE (18)	Dimension d for 1/2-OSI (22)
Gaussian	$d = \Omega(k/\varepsilon^2)$	$d = O(k)$,
SRTT	$d = \Omega(k \log k/\varepsilon^2)$	$d = O(k \log k)$
Sparse sign	$d = \Omega(k \log k/\varepsilon^2)$, $\zeta = \Omega(\log k/\varepsilon)$	$d = O(k)$, $\zeta = O(\mu(\mathbf{A}) \cdot \log k)$
CountSketch	$d = \Omega(k^2/\varepsilon^2)$	$d = O(k^2)$
SparseStack	$d = \Omega(k \log k/\varepsilon^2)$, $\zeta = \Omega(\log k/\varepsilon)$	$d = O(k)$, $\zeta = O(\log k)$
SparseRTT	$d = O(k)$, $\zeta = O(\log^3 k)^\dagger$	$d = O(k)$, $\zeta = O(\log k)$

Table 2 Summary of coordinate-mixing randomized DRMs commonly used for row-sketching rank- k $\mathbf{A} \in \mathbb{F}^{m \times n}$, which are also oblivious subspace injections (OSI). Theoretical bounds on the dimension d for $\mathbf{\Gamma}$ to satisfy the OSI property (22) vs. the OSE property (18) are provided, cf. [44, 257]. For OSIs, we enforce a constant injectivity parameter, i.e. $\alpha \geq 1/2$, which is generally not attainable for OSEs due to the dilation parameter. For the OSE column of SparseRTTs, the dagger $\dagger$ signifies that a different sparse matrix is used for the OSE property, cf. [44, Section 4.1].

3.2 Randomized Matrix Sampling

In the previous section, we observed that a random sketch $\mathbf{\Gamma A}$ (or $\mathbf{A \Omega}$) is a new matrix whose rows (or columns) are random linear combinations of the rows (or columns) of $\mathbf{A}$. However, we can also construct randomized DRMs that sample coordinates of $\mathbf{A}$ according to a probability distribution. We use the separate term randomized sampling to refer to these types of maps.

To illustrate, suppose that we want to draw d random *i.i.d.* row indices of a matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ according to probabilities $(p_1, \dots, p_m)$. Common sampling methods include:

- **Uniform** ([189, Section 9.6.2], [254, Lemma 3.4], [148]):

$$p_i = 1/m.$$

- **Squared (row) norms** ([106]):

$$p_i = \|\mathbf{a}_i\|_2^2 / \|\mathbf{A}\|_F^2,$$

where $\mathbf{a}_i^*$ is the i th row of $\mathbf{A}$,

- **Row leverage scores** ([88, 93], [189, Section 9.6.3]):

$$p_i = l_i/n \text{ for leverage score } l_i = \|\mathbf{a}_i\|_{(\mathbf{A}^* \mathbf{A})^{-1}}^2, \quad (23)$$

where $\mathbf{a}_i^*$ is the i th row of $\mathbf{A}$ and the norm is defined as $\|\mathbf{v}\|_{\mathbf{B}} = \sqrt{\mathbf{v}^* \mathbf{B} \mathbf{v}}$.

- **Determinantal point processes (DPP)** ([79, Section 5.2], [166]): If $S \subseteq [m]$, $|S| = d$,

$$p_S = \frac{\det((\mathbf{A} \mathbf{A}^*)_{S,S})}{\det(\mathbf{I} + \mathbf{A} \mathbf{A}^*)}, \quad (24)$$

where $(\mathbf{A} \mathbf{A}^*)_{S,S}$ is the sub-matrix of $\mathbf{A} \mathbf{A}^* \in \mathbb{R}^{m \times m}$ indexed by S and all indices S are selected at once. However, in practice, vectors (rows) corresponding to indices S are selected iteratively, as in the algorithm presented in [134]. Let $\mathbf{A} \mathbf{A}^* = \sum_i \lambda_i \mathbf{u}_i \mathbf{u}_i^*$ denote the eigendecomposition with $\lambda_i \in [0, 1]$ for all i . Let $s_i \sim \text{Bernoulli}(\lambda_i)$, and let $\mathbf{U}$ be the matrix with columns $\mathbf{u}_i$. By [134, Theorem 7] and [79, Theorem 12], we can use $\sum_i s_i \mathbf{u}_i \mathbf{u}_i^*$ as a proxy for DPP sampling as follows, letting $\mathbf{v}_i^*$ represent the i th row of $\mathbf{U}$:

1. Sample vector $\mathbf{v}_i$ with $p_i \propto \|\mathbf{v}_i\|_2^2$
 2. Project all vectors $\mathbf{v}_j$ onto the subspace orthogonal to $\mathbf{v}_i$
 3. Update the probabilities and repeat $d - 1$ times.
- **Nuclear norm maximization** ([8, 102, 205, 206]): While neither a traditional sketching or sampling method, we include nuclear norm maximization in this list because of its comparable performance to DPP sampling. Let $\mathbf{K} = \mathbf{A} \mathbf{A}^*$, $S \subseteq [m]$, $|S| = d$, and define

$$\mathcal{L}_{\mathbf{K}}(S) = \text{trace}(\mathbf{K}_{:,S}(\mathbf{K}_{S,S})^{-1} \mathbf{K}_{S,:}) = \text{trace}((\mathbf{K}^2)_{S,S}(\mathbf{K}_{S,S})^{-1}).$$

Maximizing $\mathcal{L}_{\mathbf{K}}(S)$ over all possible subsets of $[m]$ of size d is infeasible as in the DPP setting, so we compute approximate nuclear scores using random sketches of $\mathbf{K}$ and $\mathbf{A}$, and the identities

$$\text{diag}(\mathbf{K}^2) = \mathbb{E}_{\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})}[(\mathbf{K} \mathbf{x})^{\odot 2}], \quad \text{diag}(\mathbf{K}) = \mathbb{E}_{\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})}[(\mathbf{A} \mathbf{x})^{\odot 2}],$$

where $\odot 2$ denotes element-wise squaring. Namely, we compute the averages

$$\text{diag}(\mathbf{K}^2)/\text{diag}(\mathbf{K}) \approx \frac{(\mathbf{K} \mathbf{\Omega})^{\odot 2} \mathbf{1}}{(\mathbf{A} \mathbf{\Phi})^{\odot 2} \mathbf{1}} \Rightarrow \mathbf{K}_{\ell,\ell}^2 / \mathbf{K}_{\ell,\ell} \approx \frac{\|[\mathbf{K} \mathbf{\Omega}]_{\ell,:}\|_2^2}{\|[\mathbf{A} \mathbf{\Phi}]_{\ell,:}\|_2^2},$$

where $\mathbf{\Omega}, \mathbf{\Phi}$ are Gaussian matrices with $O(\log m)$ columns. We then update $\mathbf{K}$ and $\mathbf{A}$ by Schur complements with respect to the selected index, and repeat $d - 1$ times.

We can express the action of randomized sampling as a randomized DRM $\mathbf{\Gamma} \in \mathbb{R}^{d \times m}$. The lower bounds on the minimal dimension required for a random sampling matrix $\mathbf{\Gamma}$ to satisfy (18) are summarized in Table 3, along with the computational complexity of computing the associated probabilities. Unlike the bounds in Table 1, the values of d required for a randomized sampling matrix $\mathbf{\Gamma}$ to satisfy (18) depend not only on the rank k of $\mathbf{A}$ and the distortion parameter ε , but also, in some cases, on quantities associated with the matrix $\mathbf{A}$ that are usually very expensive to compute.

For uniform sampling, the requisite embedding dimension d depends on the coherence μ of $\mathbf{A}$, defined as $\mu(\mathbf{A}) = \max_{1 \leq i \leq m} \frac{n}{m} l_i$, or the maximum leverage score l_i in (23). The coherence is a measure of how evenly information in the matrix is distributed; high coherence indicates that its key information is very localized and that uniform sampling may perform poorly as a result; see, e.g., [139] or [189, Section 9.6.2]. In spite of this drawback, uniform sampling has become a popular choice for large-scale linear algebra computations as a cost-effective oblivious subspace embedding (OSE) method. However, it is unclear how to best choose the embedding dimension d in practice since the coherence is typically unavailable, cf. [189, Section 9.6.4].

Squared norm sampling and leverage score sampling also yield randomized subspace embeddings, but not oblivious ones, though we can still analyze the embedding dimensions required for them to satisfy (18). For squared norm sampling, the value of d depends on the condition number κ of $\mathbf{A}^* \mathbf{A}$, defined as the ratio of its largest to smallest eigenvalue, which is not readily available in practice. The embedding dimension for leverage score sampling only depends on k and ε , but this independence comes at great computational expense to compute the sampling probabilities. In particular, we note that leverage scores are as expensive to directly compute as $\text{svd}(\mathbf{A})$, rendering their direct computations infeasible for most practical purposes. However, there exist fast methods to approximate them, cf. [7, 63, 91, 201].

DPP sampling is known to have nearly optimal theoretical guarantees, cf. [77]. However, naive implementations incur a cost of $O(m^3)$, which makes DPP sampling practically infeasible. In [76], the DPP-VFX algorithm is presented, which, given access to $\mathbf{A}\mathbf{A}^T$, samples exactly from a DPP distribution with a pre-processing cost of $O(mk^{10} + k^{15})$. In particular, the DPP-VFX algorithm was the first to achieve a cost of $O(m \cdot \text{poly}(k))$ for the first sample and $O(\text{poly}(k))$ for subsequent samples; however, there still exists a runtime bottleneck of $\Omega(m)$ due to the computation of m marginals for the associated sampling distribution. The same authors improved on DPP-VFX with the α -DPP algorithm of [43], which avoids the $\Omega(m)$ bottleneck by incorporating uniform sub-sampling before computing marginals.

While nuclear norm maximization is a greedy method of choosing columns based on posterior error minimization, the authors of [102] present a “matrix-free” algorithm for fast implementation with randomization. Its performance is comparable to that of DPP sampling, and nuclear norms could be adapted into a sampling distribution, so we include it in this section for side-by-side comparisons in Table 3. Namely, in [102], randomized sketching is used to compute the approximate scores that dictate column selection. We note that the requisite embedding dimension d for nuclear norm maximization depends on the quantity $\nu = \frac{\text{trace}(\mathbf{K}) - \text{trace}^{(k)}(\mathbf{K})}{\text{trace}^{(k)}(\mathbf{K})}$, where $\mathbf{K} = \mathbf{A}\mathbf{A}^*$ and $\text{trace}^{(k)}(\mathbf{K})$ is the sum of its k largest eigenvalues. For the complexity, we also include

the dependence on the quantity ϕ , which is the maximum cost of a mat-vec with $\mathbf{K}$ or $\mathbf{A}$, cf. [102, Corollary 4.1].

Sampling distribution	Dimension d	Complexity of distribution
Uniform	$d = \Omega(\mu \log k / \varepsilon^2)$	$O(1)$
Squared norms	$d = \Omega(\kappa \log k / \varepsilon^2)$	$O(mn)$
Leverage scores	$d = \Omega(k \log k / \varepsilon^2)$	$O(mn^2)$
DPP [†]	$d = \Omega(k / \varepsilon + k - 1)$	$O(m^3)$
Nuclear norm max	$d = \Omega(k / \varepsilon + k - 1)(\log(v^{-1}) + \log(1/\varepsilon))$	$O(md^2 \log m + d \log m \psi)$

Table 3 Summary of randomized sampling distributions commonly used to sample rows of rank- k matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$. Theoretical lower bounds on the embedding dimension d are given for $\mathbf{\Gamma}$ to satisfy (18), as well as the asymptotic complexity of computing the associated probability distribution, cf. [79, 189]. We note (by [†]) that the cost of DPP sampling can be reduced, cf. [79, Section 5], [76], [43].

3.3 Application: Randomized DRMs for Overdetermined Least Squares

As an illustrative application of randomized DRMs that will also be useful for our tensor algorithms, we consider the overdetermined least squares problem, in which we solve

$$\mathbf{x}_* = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2, \quad (25)$$

for given $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, and $m \gg n$. Using CPQR for (25) requires $O(mn^2)$ operations, which is undesirable for large m .

A natural idea is to apply a randomized DRM to $\mathbf{A}$ and $\mathbf{b}$ in (25), a strategy known as “sketch-and-solve” [236]. Namely, we draw a randomized DRM $\mathbf{\Gamma} \in \mathbb{F}^{d \times m}$ and solve a smaller sketched problem

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{\Gamma}(\mathbf{A}\mathbf{x} - \mathbf{b})\|_2. \quad (26)$$

If $\mathbf{\Gamma}$ is a randomized subspace embedding for the k -dimensional column space of $[\mathbf{A} \ \mathbf{b}]$ with distortion $\varepsilon \in (0, 1)$, then

$$(1 - \varepsilon)\|\mathbf{A}\mathbf{x}_* - \mathbf{b}\|_2 \leq \|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|_2 \leq (1 + \varepsilon)\|\mathbf{A}\mathbf{x}_* - \mathbf{b}\|_2$$

when $d \approx k/\varepsilon^2$. Solving (26) requires just $O(dn^2)$ operations, and often in practice, there are ways to avoid forming $\mathbf{\Gamma}\mathbf{A}$ and $\mathbf{\Gamma}\mathbf{b}$ explicitly.

Randomized DRMs are also utilized in other methods of solving (25). Sketched GMRES [203, Algorithm 1.1] includes a k -truncated Arnoldi process to quickly compute an approximate basis of the Krylov subspace. Sketch-and-precondition methods are also commonly used, but we note that these methods were recently shown to be

numerically unstable when initialized with the zero vector, cf. [191]. With iterative sketching methods, sketched least squares problems are solved until converged, and these methods were recently proven to be forward stable in [97]. We discuss alternative randomized DRMs for overdetermined least squares problems for computing tensor decompositions in Sect. 6, which are based on the randomized algorithms for low-rank matrix decompositions that we discuss next.

4 Randomized Algorithms for Low-Rank Matrix Decompositions

In this section, we illustrate several ways that randomized DRMs are often used to compute low-rank matrix decompositions. We first introduce the rangefinder problem and show how it can be solved with randomized DRMs in Sect. 4.1. We then explain how randomized DRMs can be used to estimate the error in the corresponding low-rank approximation, which can be utilized for adaptive rangefinding. These ideas are foundational for randomized algorithms that have become ubiquitous for fast and reliable low-rank matrix approximation: the randomized SVD (Sect. 4.2) and randomized ID/CUR (Sect. 4.3).

4.1 The Randomized Rangefinder

At the heart of many linear algebraic tasks lies the *rangefinder problem*, the computation of a lower-dimensional space that sufficiently captures the action of a given matrix. More specifically, to solve the rangefinder problem for an input matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and target rank $k \leq \min(m, n)$, we compute an orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{m \times k}$ whose columns approximately span the column space of $\mathbf{A}$. We then obtain a rank- k approximation given by $\mathbf{Q}\mathbf{Q}^*\mathbf{A}$, with approximation error

$$\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^*\mathbf{A}\| = \|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\|, \quad (27)$$

measured in either the Frobenius or spectral norm in our work. The Frobenius norm is often preferable for ease of analysis, but error bounds in the Frobenius norm can be less informative than bounds in the spectral norm, cf. [189, Remark 2.1].

The rangefinder problem can be handled effectively with randomized sketching as shown in Algorithm 1. This algorithm produces an orthonormal matrix $\mathbf{Q}$ via QR factorization of an $m \times \ell$ random sketch of $\mathbf{A}$ for desired embedding dimension ℓ . The total cost of Algorithm 1 includes the cost of simulating the $n \times \ell$ random test matrix, $O(mn\ell)$ operations to form the sketch, and $O(mk^2)$ operations for the QR factorization. Sparsity or availability of fast matrix-vector primitives with $\mathbf{A}$ can lend additional performance improvements.

Algorithm 1 Randomized Rangefinder

Require: $\mathbf{A} \in \mathbb{R}^{m \times n}$, target rank k

Ensure: Orthonormal $\mathbf{Q} \in \mathbb{R}^{m \times k}$ such that $\mathbf{A} \approx \mathbf{Q}\mathbf{Q}^* \mathbf{A}$

- 1: Draw randomized DRM $\mathbf{\Omega}^* \in \mathbb{R}^{\ell \times n}$ for $k < \ell \leq \min(m, n)$
 - 2: Form sketch $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}^*$.
 - 3: Compute $\mathbf{Q} = \text{col}(\mathbf{Y}, k)$.
-

4.1.1 Implementation details

There are many considerations to be made when implementing the randomized rangefinder algorithm in practice. We summarize the important details here and refer to [120, 189] for more thorough treatments.

First, sketching, as opposed to sampling, should be used whenever possible. Gaussian matrices are popular for easy implementation and strong theoretical results, but structured random matrices (e.g., SparseStack) perform almost as well in practice. Second, the numerical accuracy of the rangefinder algorithm can be improved with power iteration [130, 140, 234] or Krylov methods [138, 233, 241], in addition to numerically stable orthogonalization techniques like Householder reflections or rank-revealing QR [50, 51].

One last important detail concerns the subspace dimension. If the target rank k of the low-rank approximation to $\mathbf{A}$ is known *a priori*, then a randomized DRM of size $\ell = k + p$ is usually sufficient with a small amount of oversampling p , such as $p = 5$. (We deal with the situation where k is unknown in Sect. 4.1.3.) For Gaussian test matrices with slight oversampling, [120, Theorem 10.1] guarantees that the expected error of the randomized rangefinder is comparable to the best rank- k approximation error from the truncated SVD of $\mathbf{A}$, so long as its trailing singular values are small. This is generally achievable, for instance, by powering $\mathbf{A}$ in the sketch $\mathbf{Y} = (\mathbf{A}\mathbf{A}^*)^q \mathbf{\Omega}^*$ some small number of iterations q [229].

The theoretical results for Gaussian random matrices often furnish the performance heuristics for structured random sketching matrices, which behave similarly to Gaussians in practice despite the more pessimistic bounds that appear in their analysis [120, Theorem 11.2]. This claim is strengthened by the simple addition of *a posteriori* “certificates of accuracy” upon termination of Algorithm 1, as explained in the next section.

4.1.2 Error estimation and certificates of accuracy

We now demonstrate how the approximation error (27) can be estimated reliably and efficiently with randomized sketching. This method of error estimation also serves as a “certificate of accuracy” for rangefinding, an assurance that the orthonormal basis given by $\mathbf{Q}$ produces a satisfactory low-rank approximation of $\mathbf{A}$.

Suppose that we have an orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{m \times k}$ whose columns potentially form a basis for the range of $\mathbf{A} \in \mathbb{R}^{m \times n}$, e.g., the output of Algorithm 1 for a given k . We estimate the approximation error $\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\|$ using the following method.

Let $\Phi \in \mathbb{R}^{n \times s}$ be a Gaussian matrix that is independent from any random matrix used to compute $\mathbf{Q}$. In practice, s is a small fixed value, such as $s = 10$. We form a new random sketch, known as the auxiliary sample

$$\mathbf{Z} = \mathbf{A}\Phi, \quad (28)$$

incurring only s additional matrix-vector products with $\mathbf{A}$. Using $\mathbf{Z}$, we can obtain an inexpensive Gaussian sketch of the approximation error (27):

$$(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{Z} = (\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\Phi. \quad (29)$$

We then take advantage of randomized methods to estimate the norm of (29), such as randomized trace estimation for the Frobenius norm or randomized Krylov methods for the Schatten $2p$ -norm (a proxy for the spectral norm with sufficiently large p). Recall that the Schatten p -norm of a matrix is the ℓ_p -norm of its singular values, so that the Schatten ∞ -norm coincides with the spectral norm and the Schatten 2-norm with the Frobenius norm.

To estimate the error in the Frobenius norm, we use the approximation

$$\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\|_F^2 \approx \frac{1}{s} \|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{Z}\|_F^2, \quad (30)$$

relying on the theory of randomized trace estimation (recall that $\text{trace}(\mathbf{X}^*\mathbf{X}) = \|\mathbf{X}\|_F^2$). Namely, we exploit the fact that for any matrix $\mathbf{B}$ and isotropic random vector ω , $\text{trace}(\mathbf{B}) = \omega^*\mathbf{B}\omega$ in expectation. To reduce the variance, we take the average of s random variables in (30), using the auxiliary sample $\mathbf{Z}$.

Estimating the error in the spectral norm is not quite as simple. It is possible to construct unbiased estimators for the Schatten $2p$ -norm for any positive integer p , as for the Frobenius norm above, but these estimators are expensive with large variance [159, 174]. Randomized Krylov methods are often used instead to estimate the spectral norm [171, 188, 200, 256, 275]. For example, we can estimate the spectral norm error with a randomized block Krylov method, letting $\mathbf{B} = [(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}] [(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}]^*$,

$$\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\|_2^2 \approx \max_{\mathbf{v} \in \mathcal{K}(\mathbf{B}; \Phi)} \frac{\mathbf{v}^*\mathbf{B}\mathbf{v}}{\mathbf{v}^*\mathbf{v}}, \quad (31)$$

where $\mathcal{K}(\mathbf{B}; \Phi) = \text{span} [\Phi \ \mathbf{B}\Phi \ \mathbf{B}^2\Phi \ \dots \ \mathbf{B}^q\Phi]$ is a block Krylov subspace of depth q . In general, $q \approx \log m$ is necessary to achieve the fast convergence rate of $1/q^2$ for Krylov methods, but smaller values of q can be used in practice for matrices with fast spectral decay. Although Krylov methods enjoy quick convergence, numerical instability is a known shortcoming, so careful implementation is needed, cf. [200, 258, 275].

Regardless of the choice of norm, an auxiliary sketch of the error with a Gaussian random test matrix Φ can provide us with an *a posteriori* “certificate of accuracy” for rangefinding [189, Section 12.2]. For instance, if a structured random matrix (e.g., SRTT, sparse sign) is used in Algorithm 1 instead of a Gaussian matrix, the computations are generally faster but lack formal guarantees on the accuracy of the computed solution. More generally, if we are unsure of the accuracy of a given solution to the rangefinder

problem, we can take a small (independent) Gaussian sketch of the error as in (28). This auxiliary sketch is inexpensive and avails us of the powerful theory of Gaussian matrices, regardless of how the column space approximation was obtained. In the next section, we demonstrate how this method of error estimation can be utilized for adaptive rangefinding when the target rank is unknown *a priori*.

4.1.3 Adaptive rangefinding

Up to this point, we have assumed that the target approximation rank could be passed as an argument to the randomized rangefinder algorithm. Unfortunately, a satisfactory target rank for the approximation is frequently unknown *a priori*. In that case, we can implement the randomized rangefinder algorithm adaptively, using the error estimation techniques of the previous section to approximate the range of $\mathbf{A}$ to a desired precision ε , forming $\mathbf{Q}$ incrementally until that precision is achieved.

For ease of notation, let `norm_est`($\mathbf{Z}$) denote the rangefinder error estimate obtained by one of the techniques in Sect. 4.1.2, e.g., (30). An adaptive version of the randomized rangefinder is given in Algorithm 2, which we quickly summarize.

Algorithm 2 Adaptive Randomized Rangefinder

Require: $\mathbf{A} \in \mathbb{F}^{m \times n}$, error tolerance $\varepsilon > 0$, block size b

Ensure: Orthonormal $\mathbf{Q}$ such that $\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^* \mathbf{A}\| \leq \varepsilon$ with high probability

```

1:  $i = 1$ 
2: Draw Gaussian random matrix  $\mathbf{\Omega} \in \mathbb{F}^{n \times b}$ 
3:  $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ 
4:  $\mathbf{Q}_i = \text{col}(\mathbf{Y})$ 
5: while norm_est( $\mathbf{Y}$ )  $> \varepsilon$  do                                      $\triangleright$  Error estimation of Sect. 4.1.2
6:    $i = i + 1$ 
7:   Draw independent Gaussian  $\mathbf{\Omega} \in \mathbb{F}^{n \times b}$ 
8:    $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ 
9:    $\mathbf{Y} = \mathbf{Y} - \sum_{j=1}^{i-1} \mathbf{Q}_j (\mathbf{Q}_j^* \mathbf{Y})$ 
10:   $\mathbf{Q}_i = \text{col}(\mathbf{Y})$ 
11: end while
12:  $\mathbf{Q} = [\mathbf{Q}_1 \ \mathbf{Q}_2 \ \cdots \ \mathbf{Q}_i]$ 

```

Let $\mathbf{A} \in \mathbb{F}^{m \times n}$ and let b be the number of columns to be processed at a time. The choice of block size b affects the computational efficiency of the algorithm. If b is too small, we are not fully reaping the benefits of blocked matrix operations that help reduce memory traffic and avoid communication bottlenecks [128, 226]. With a larger block size b , more floating-point operations can be cast as matrix-matrix multiplication (BLAS-3) vs. matrix-vector multiplication (BLAS-2) [31, 87, 108, 131]. However, a larger block size runs the risk of overshooting the target rank, meaning we have done more work than necessary for rangefinding. In practice, a block size $10 \leq b \leq 100$ is usually appropriate.

In each iteration of the while-loop in Algorithm 2, the matrix $\mathbf{Y}$ holds a sample of the residual error after the execution of line 9:

$$\mathbf{Y} = (\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}\mathbf{\Omega}, \quad (32)$$

where $\mathbf{Q}$ is the basis that has been built up to the current iteration. Critically, the random test matrix $\mathbf{\Omega}$ used in (32) is independent from the random matrices used to compute the current basis $\mathbf{Q}$. We note that to compute the very first residual error $\text{norm_est}(\mathbf{Y})$ in line 5, we can draw a small auxiliary sample that then can be recycled in the second batch of samples in line 8.

Variations of Algorithm 2 have been designed to optimize performance for specific types of input matrices. If the input matrix $\mathbf{A}$ is small enough to store in RAM, it can be explicitly updated in each iteration via $\mathbf{A} = (\mathbf{I} - \mathbf{Q}\mathbf{Q}^*)\mathbf{A}$ as in [189, Algorithm 14]. Alternatively, the version of [281] effectively handles input matrices stored out-of-core, as well as sparse input matrices. If the spectrum of the input matrix decays slowly, power iteration can also be incorporated into Algorithm 2 to improve performance [190].

4.2 Randomized SVD

Now equipped with the randomized rangefinder, we present a foundational algorithm for low-rank matrix approximation known as the randomized SVD [120]. Pseudocode is provided in Algorithm 3. We quickly go through its steps to show that the error of the resulting approximate SVD is precisely the randomized rangefinder error.

Algorithm 3 Randomized SVD

Require: $\mathbf{A} \in \mathbb{R}^{m \times n}$, target rank k

Ensure: Orthonormal matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$, $\mathbf{V} \in \mathbb{R}^{n \times k}$, and diagonal $\mathbf{\Sigma} \in \mathbb{R}^{k \times k}$ such that $\mathbf{A} \approx \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$

- 1: Compute $\mathbf{Q} = \text{RandomizedRangefinder}(\mathbf{A}, k)$ ▷ Algorithm 1
 - 2: Form $\mathbf{B} = \mathbf{Q}^*\mathbf{A} \in \mathbb{R}^{k \times n}$
 - 3: Compute $[\hat{\mathbf{U}}, \mathbf{\Sigma}, \mathbf{V}] = \text{svd}(\mathbf{B})$
 - 4: Form $\mathbf{U} = \mathbf{Q}\hat{\mathbf{U}}$
-

Given an input matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, our goal is to compute an approximate rank- k truncated singular value decomposition

$$\mathbf{A} \approx \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*, \quad (33)$$

where $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$ are orthonormal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{k \times k}$ is a diagonal matrix of entries approximating the largest singular values of $\mathbf{A}$.

Suppose that $\mathbf{Q} \in \mathbb{R}^{m \times k}$ is a solution to the rangefinder problem (not necessarily from Algorithm 1), which corresponds to a rank- k approximation of $\mathbf{A}$ given by $\mathbf{A} \approx \mathbf{Q}\mathbf{Q}^*\mathbf{A}$. Let $\mathbf{B} = \mathbf{Q}^*\mathbf{A} \in \mathbb{R}^{k \times n}$ and compute singular value decomposition

$$\mathbf{B} = \underset{k \times k}{\hat{\mathbf{U}}} \underset{k \times k}{\mathbf{\Sigma}} \underset{k \times n}{\mathbf{V}^*}. \quad (34)$$

We observe that

$$\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^*\mathbf{A}\| = \|\mathbf{A} - \mathbf{Q}\mathbf{B}\| = \|\mathbf{A} - \mathbf{Q}\hat{\mathbf{U}}\Sigma\mathbf{V}^*\| = \|\mathbf{A} - \mathbf{U}\Sigma\mathbf{V}^*\|, \quad (35)$$

for $\mathbf{U} := \mathbf{Q}\hat{\mathbf{U}}$; i.e. the approximation error of $\mathbf{A} \approx \mathbf{U}\Sigma\mathbf{V}^*$ is exactly the approximation error of $\mathbf{A} \approx \mathbf{Q}\mathbf{Q}^*\mathbf{A}$.

Thus, if $\mathbf{Q} = \text{RandomizedRangefinder}(\mathbf{A}, k)$ is the rangefinder solution computed by Algorithm 1, the accuracy of the randomized SVD is equal to that of the randomized rangefinder. In particular, this means that the error estimation techniques of Sect. 4.1.2 can also be used to estimate the error of the randomized SVD. Then if the target rank k is unknown *a priori*, the adaptive randomized rangefinder (Algorithm 2) can be carried out first, which determines k in the randomized SVD.

The total cost of Algorithm 3 includes the cost of the randomized rangefinder, plus $O(mnk)$ floating-point operations for $\mathbf{B} = \mathbf{Q}^*\mathbf{A}$ in line 2. We note that the cost of the SVD in line 3 is just $O(k^2n)$, so the cost of Algorithm 3 after rangefinding is usually dominated by the matrix-matrix multiplication. Executing Algorithm 3 requires storage for $O((m+n)k)$ numbers.

Recent work has also been done on cost-effective *a posteriori* error estimation methods that use only the information at hand for computing the low-rank decomposition. In [99], an algorithm for *a posteriori* error estimation is presented for the randomized SVD in a matrix-free setting, where the input matrix $\mathbf{A}$ is only accessible via mat-vecs. Given the rank- k randomized SVD $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^* \in \mathbb{F}^{m \times n}$ corresponding to Gaussian random matrix $\mathbf{\Omega} = [\omega_1 \ \omega_2 \ \dots \ \omega_k]$, we want to estimate the mean-square error

$$\text{MSE}(\mathbf{X}_{k-1}, \mathbf{A}) = \mathbb{E}\|\mathbf{A} - \mathbf{X}_{k-1}\|_{\mathbb{F}}^2,$$

where $\mathbf{X}_{k-1} \in \mathbb{F}^{m \times (k-1)}$ is a rank- $(k-1)$ randomized SVD.

By [99, Theorem 2.1], the following is an unbiased estimator for $\text{MSE}(\mathbf{X}_{k-1}, \mathbf{A})$, which we use as a proxy for the mean-square error of the rank- k approximation $\mathbf{X}$, called the leave-one-out error estimator:

$$\text{MSE}(\mathbf{X}_{k-1}, \mathbf{A}) = \mathbb{E} \left[\frac{1}{k} \sum_{j=1}^k \|(\mathbf{A} - \mathbf{X}^{(j)})\omega_j\|^2 \right], \quad (36)$$

where $\mathbf{X}^{(j)} = \mathbf{Q}^{(j)}(\mathbf{Q}^{(j)})^*\mathbf{A}$ is called a replicate, formed from the matrix $\mathbf{Q}^{(j)}$ outputted by the randomized rangefinder using $\mathbf{\Omega}$ without its j -th column ω_j . Implemented naively, the right-hand side of (36) is too expensive for practical use, as it would require k additional randomized rangefinder applications. However, by [99, Equation 4.2],

$$\mathbf{Q}^{(j)}(\mathbf{Q}^{(j)})^* = \mathbf{Q}(\mathbf{I} - \mathbf{t}_j\mathbf{t}_j^*)\mathbf{Q}^*, \quad (37)$$

where $\mathbf{t}_j$, $j = 1, \dots, k$, are the normalized columns of $(\mathbf{R}^*)^{-1}$ in the QR factorization defined by $\mathbf{Q}$ in step 1 of the randomized SVD (now also storing the $\mathbf{R}$ matrix). Then we can obtain each replicate $\mathbf{X}^{(j)}$ inexpensively as

$$\mathbf{X}^{(j)} = \mathbf{U}(\mathbf{I} - \hat{\mathbf{U}}^*\mathbf{t}_j\mathbf{t}_j^*\hat{\mathbf{U}})\Sigma\mathbf{V}^*, \quad (38)$$

which allows us to compute (36) without any additional mat-vecs or applications of Algorithm 3, via

$$\text{MSE}(\mathbf{X}_{k-1}, \mathbf{A}) \approx \frac{1}{k} \sum_{j=1}^k \|[(\mathbf{R}^*)^{-1}](:, j)\|^{-2}.$$

4.3 Randomized ID/CUR

In both the randomized rangefinder and the randomized SVD, orthonormal matrices are computed whose columns form an approximate basis for the row or column space of an input matrix $\mathbf{A}$. The entries of these orthonormal matrices are linear combinations of the entries of $\mathbf{A}$. However, for some problems (e.g., if $\mathbf{A}$ is sparse or non-negative), it may be preferable to have a “natural basis,” consisting of subsets of rows or columns of $\mathbf{A}$, as in the ID/CUR factorizations of Sect. 2.4. In this section, we provide an overview of randomized methods to compute approximate ID/CUR factorizations based on the randomized rangefinder, including recent work to improve skeleton selection and performance [85, 86, 216, 225].

4.3.1 With the randomized rangefinder

Recall from Sect. 2.4 that pivoting algorithms, such as column-pivoted QR (CPQR) or LU with partial pivoting, are frequently used in the deterministic setting to select skeletons for an ID/CUR factorization. CPQR produces skeletons that are close to optimal in practice [66, 187]. In particular, this is true when CPQR is applied to a random sketch of $\mathbf{A}$ with appropriate embedding dimension. It is also conjectured in [252, Remark 4] and exhibited empirically in [86, 216] that randomized sketching equips LUPP with near-optimal skeleton selection capability as well, even in adversarial cases. We illustrate how randomized sketching with CPQR or LUPP can be used to compute a row ID, then discuss alternate skeleton selection strategies that are rank-adaptive or based on sampling instead of sketching.

More specifically, let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be an input matrix of exact rank k for notational convenience. Let $\mathbf{Y} \in \mathbb{R}^{m \times k}$ have columns that span the range of $\mathbf{A}$, e.g., the output of the rangefinder algorithm. By definition,

$$\mathbf{A} = \underset{m \times k}{\mathbf{Y}} \underset{k \times n}{\mathbf{F}}$$

for some matrix $\mathbf{F}$. Compute a row ID of $\mathbf{Y}$, either using CPQR via $[\sim, \sim, I] = \text{qr}(\mathbf{Y}^*, k)$ as in Sect. 2.4.1 or LUPP via $[\sim, \sim, I] = \text{lu}(\mathbf{Y}, k)$ as in Sect. 2.4.2, to obtain

$$\mathbf{Y} = \underset{m \times k}{\mathbf{X}} \underset{k \times k}{\mathbf{Y}(I, :)}. \quad (39)$$

We now observe that index set I and row interpolation matrix $\mathbf{X}$ form a row ID of $\mathbf{A}$:

$$\mathbf{A} = \mathbf{Y}\mathbf{F} = [\mathbf{X}\mathbf{Y}(I, :)] \mathbf{F} = \mathbf{X} [\mathbf{Y}(I, :)\mathbf{F}] = \mathbf{X}\mathbf{A}(I, :).$$

Thus, to compute a row ID of $\mathbf{A}$, it suffices to compute a row ID of a matrix $\mathbf{Y}$ whose columns span the range of $\mathbf{A}$.

We can construct such a matrix $\mathbf{Y} \in \mathbb{R}^{m \times \ell}$ for a rank- k approximation of $\mathbf{A}$ using the randomized rangefinder with high probability with, e.g., $\ell = k + 10$. Additionally, since we do not need $\mathbf{Y}$ to be an orthonormal matrix here, we can save on computational cost by observing that the range of $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ is itself an accurate approximation to the range of $\mathbf{A}$ with high probability [120, Theorems 10.5 and 10.6]. The randomized rangefinder error $\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^*\mathbf{A}\|$ using $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ is equal to $\|\mathbf{A} - \mathbf{Y}\mathbf{Y}^\dagger\mathbf{A}\|$, where $\mathbf{Y}\mathbf{Y}^\dagger$ is the orthogonal projector onto the column space of $\mathbf{Y}$ or, equivalently, the column space of $\mathbf{Q}$. Pseudocode for this randomized row ID algorithm (with sketched CPQR) is provided in Algorithm 4.

Algorithm 4 Randomized Row ID with Sketched CPQR

Require: $\mathbf{A} \in \mathbb{R}^{m \times n}$, target rank k , oversampling p

Ensure: Row indices I and row interpolation matrix $\mathbf{X} \in \mathbb{R}^{m \times k}$ such that $\mathbf{A} \approx \mathbf{X}\mathbf{A}(I, :)$

1: Draw randomized embedding $\mathbf{\Omega}^* \in \mathbb{R}^{(k+p) \times n}$

2: Form $\mathbf{Y} = \mathbf{A}\mathbf{\Omega} \in \mathbb{R}^{m \times (k+p)}$

3: Compute $[\sim, \mathbf{R}, \mathbf{P}] = \text{qr}(\mathbf{Y}^*)$

4: Set $I = P(1 : k)$

5: Set $\mathbf{X}(P, :) = [\mathbf{I} \quad \mathbf{R}_{1:k, 1:k}^{-1} \mathbf{R}_{1:k, k+1:m}]^*$

4.3.2 Adaptive algorithms for randomized ID/CUR

Frequently in applications, the target rank for a low-rank approximation is unknown *a priori*, in which case we turn to rank-adaptive methods. If using CPQR for skeleton selection, the adaptive version of the randomized rangefinder may be used to build an orthonormal basis incrementally from independent random sketches or samples.

However, if we are using LUPP as the skeleton selection method for improved efficiency, we want to estimate the approximation error without computing an orthonormal basis. We briefly illustrate how a row ID can be constructed with the rank-adaptive, LUPP-based method of [216]. Given an error tolerance τ and input matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, after t iterations of this algorithm, we obtain target rank k_t , row skeletons $I_s^{(t)}$ with $|I_s^{(t)}| = k_t$, and interpolation matrix $\mathbf{W}^{(t)}$ such that

$$\|\mathbf{A} - \mathbf{W}^{(t)}\mathbf{R}^{(t)}\|_F < \tau,$$

where $\mathbf{R}^{(t)} = \mathbf{A}(I_s^{(t)}, :)$.

Fix a block size b (e.g., $b = 10$), draw a random matrix $\mathbf{\Omega}^{(0)} \in \mathbb{R}^{n \times b}$, and form $\mathbf{Y}^{(0)} = \mathbf{A}\mathbf{\Omega}^{(0)}$. We use LUPP to compute

$$\mathbf{P}^{(0)}\mathbf{Y}^{(0)} = \mathbf{L}^{(0)}\mathbf{U}^{(0)},$$

so that the first b skeletons $I_s^{(0)}$ are the first b indices of $\mathbf{P}^{(0)}[1 \dots m]^*$. To estimate $\|\mathbf{A} - \mathbf{W}^{(0)}\mathbf{R}^{(0)}\|_F$, we draw an independent random matrix $\mathbf{\Omega}^{(1)} \in \mathbb{F}^{n \times b}$ from the same distribution as $\mathbf{\Omega}^{(0)}$ and form $\mathbf{Y}^{(1)} = \mathbf{A}\mathbf{\Omega}^{(1)}$. By [216, Equation 20],

$$\begin{aligned} \|\mathbf{A} - \mathbf{W}^{(0)}\mathbf{R}^{(0)}\|_F &= \|(\mathbf{P}^{(0)}\mathbf{A})(b+1:m, :) - \mathbf{L}_2^{(0)}(\mathbf{L}_1^{(0)})^{-1}(\mathbf{P}^{(0)}\mathbf{A})(1:b, :)\|_F \\ &\approx \|\mathbf{A}\mathbf{\Omega}^{(1)} - \mathbf{W}^{(0)}\mathbf{R}^{(0)}\mathbf{\Omega}^{(1)}\|_F \\ &= \|\mathbf{P}^{(0)}\mathbf{Y}^{(1)}(b+1:m, :) - \mathbf{L}_2^{(0)}(\mathbf{L}_1^{(0)})^{-1}(\mathbf{P}^{(0)}\mathbf{Y}^{(1)})(1:b, :)\|_F, \end{aligned}$$

where $\mathbf{L}_1^{(0)} = \mathbf{L}^{(0)}(1:b, :)$ and $\mathbf{L}_2^{(0)} = \mathbf{L}^{(0)}(b+1:m, :)$. If the given tolerance is not met, we LU-factorize $[\mathbf{Y}^{(0)} \ \mathbf{Y}^{(1)}]$ re-using the previous factorization, and repeat. In general, the error after $t-1$ iterations is given by the Schur complement

$$\|\mathbf{S}^{(t)}\|_F = \|(\mathbf{P}^{(t-1)}\mathbf{Y}^{(t)})(tb+1:m, :) - \mathbf{L}_2^{(t-1)}(\mathbf{L}_1^{(t-1)})^{-1}(\mathbf{P}^{(t-1)}\mathbf{Y}^{(t)})(1:tb, :)\|_F.$$

A similar idea is presented in [225] for a rank-adaptive CUR factorization, using a single small sketch of the residual matrix $\mathbf{S} = \mathbf{A} - \mathbf{CUR}$. (Here, $\mathbf{U}$ is computed as $\mathbf{A}(I, J)^\dagger$ vs. $\mathbf{C}^\dagger \mathbf{A} \mathbf{R}^\dagger$ as in Sect. 2.4.) The column residual is initialized as a row sketch, $\mathbf{S}_0^{(C)} = \mathbf{GA} \in \mathbb{F}^{1.1b \times n}$, where typically $5 \leq b \leq 250$, and the row and column skeletons are empty sets. While the error criterion is unmet, column skeletons J are first computed from $\mathbf{S}_0^{(C)}$, and the row residual is formed as $\mathbf{S}_1^{(R)} = \mathbf{A}(:, J) - \mathbf{C}_1 \mathbf{U}_1 \mathbf{R}_1(:, J) = \mathbf{A}(:, J)$. Row skeletons I are then computed from $\mathbf{S}_1^{(R)}$, and the column residual is formed as $\mathbf{S}_1^{(C)} = \mathbf{GA} - \mathbf{GC}_1 \mathbf{U}_1 \mathbf{R}_1$, where now $\mathbf{C}_1 = \mathbf{A}(:, J)$, $\mathbf{R}_1 = \mathbf{A}(I, :)$, and $\mathbf{U}_1 = \mathbf{A}(I, J)^\dagger$ are no longer empty. This procedure is repeated, with new skeletons appended to previously selected ones, until the desired approximation accuracy is achieved.

4.3.3 Algorithms based on coordinate sampling

There have been several recent advances in coordinate sampling-based methods for ID/CUR decompositions.

In [62], the randomly pivoted (RP) Cholesky algorithm is introduced for positive semi-definite matrices $\mathbf{A} \in \mathbb{F}^{m \times m}$. Given a target rank k , the algorithm outputs a set of pivot indices $S = \{s_1, \dots, s_k\}$ akin to skeletons, and a matrix $\mathbf{F} \in \mathbb{F}^{m \times k}$ that defines the Nystrom approximation $\hat{\mathbf{A}} = \mathbf{FF}^*$. We begin by defining $\mathbf{d} = \text{diag}(\mathbf{A})$ and initializing the matrix $\mathbf{F}$ as $\mathbf{0}$.

After sampling each pivot $s_i \sim \mathbf{d} / \sum_{j=1}^m \mathbf{d}(j)$, we form $\mathbf{g} = \mathbf{A}(:, s_i)$ and remove any overlap with previously selected columns via

$$\mathbf{g} = \mathbf{g} - \mathbf{F}(:, 1:(i-1))\mathbf{F}(s_i, 1:(i-1))^*.$$

The matrix $\mathbf{F}$ is then updated via $\mathbf{F}(:, i) = \mathbf{g} / \sqrt{\mathbf{g}(s_i)}$, and the vector $\mathbf{d}$ of diagonal entries of the residual matrix is updated via $\mathbf{d} = \mathbf{d} - |\mathbf{F}(:, i)|^2$. To achieve an ε -approximation, $d \geq k/\varepsilon + k \log(\frac{1}{\varepsilon\eta})$ columns are required, where η is the relative trace-norm error, cf. [62, Theorem 2.3]. A blocked variant of RP Cholesky is also provided in [62,

Algorithm 3]. Although RP Cholesky is efficient, simple to implement, and comes with strong theoretical guarantees, it can only be applied to positive semi-definite matrices. If $\mathbf{A}$ is not positive semi-definite, then we would need to form the Gram matrix $\mathbf{A}\mathbf{A}^*$ before applying RP Cholesky, which is computationally undesirable.

The recent work of [85], inspired by RP Cholesky, introduces robust blockwise random pivoting (RBRP) for matrix IDs. Like RP Cholesky, the RBRP algorithm incorporates both randomness and adaptiveness, in that each iteration of skeleton selection is informed by previous ones. Blockwise random pivoting replaces greedy pivoting based on squared column norms in CPQR with squared-norm sampling, and the “robustness” of RBRP refers to a filtering step that eliminates redundant points within each block of selected skeleton candidates. Namely, suppose that we sample b row skeleton candidates S_t for some fixed block size b . We then form

$$\mathbf{V} = \mathbf{A}(S_t, :)^* - \mathbf{Q}^{(t-1)}((\mathbf{Q}^{(t-1)})^* \mathbf{A}(S_t, :)^*) \in \mathbb{R}^{n \times b},$$

where $\mathbf{Q}^{(t-1)}$ is an orthonormal basis for the previously selected skeletons. Given some filtering tolerance τ , we use CPQR to compute the factorization

$$\mathbf{V}(:, P) = \mathbf{Q}\mathbf{R} \approx \mathbf{Q}(:, 1 : b')\mathbf{R}(1 : b', :),$$

so that

$$\|\mathbf{V}(:, P) - \mathbf{Q}(:, 1 : b')\mathbf{R}(1 : b', :)\|_F^2 = \|\mathbf{R}(b' + 1 : b, b' + 1 : b)\|_F^2 < \tau \|\mathbf{V}\|_F^2.$$

This leads to a time complexity of roughly $O(mnk + nk^2)$ for resulting skeleton cardinality k , as RBRP is also rank-adaptive, and the optimal associated interpolation matrix $\mathbf{W} = \mathbf{A}\mathbf{A}(S, :)^{\dagger}$ can be computed efficiently in $O(mk^2)$ time.

5 Tensor Preliminaries

This section marks the beginning of our exploration of tensors. We begin with formal notation and definitions in Sect. 5.1, presenting tensors as natural higher-order generalizations of matrices, from 2-way to d -way arrays. As a result, many of the randomized methods in the previous sections developed for matrices can be extended to tensors, though there are subtleties with higher-order inputs, and many concepts can be obscured by the unwieldy notation that frequently accompanies them. We try to mitigate this by providing visualizations, including a table of notation (Table 4) and illustrations of the matrix and tensor products in Sect. 5.2. In Sect. 5.3, we point out some peculiarities specific to higher-order tensors, such as tensor rank and uniqueness of decompositions.

While there are many different ways to represent tensors, we focus on two fundamental tensor decomposition formats: the canonical polyadic (CP) and Tucker. In Sect. 5.3, we lay out key concepts and practicalities for each of them, including algorithms commonly used to compute them. We then discuss particular low-rank tensor decompositions that can be represented in each format, such as higher-order extensions

of the matrix SVD and ID/CUR. Throughout this section, we try to emphasize that the classical matrix methods covered in previous sections underlie those for tensors.

Symbol	Definition
$\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$	Tensor with d modes
$\ \mathcal{X}\ $	Frobenius norm $\left(\sum_{i_1=1}^{N_1} \dots \sum_{i_d=1}^{N_d} \mathcal{X}(i_1, \dots, i_d)^2\right)^{1/2}$
$N = \prod_{j=1}^d N_j = \text{size}(\mathcal{X})$	Number of entries of $\mathcal{X}$
$N^{(-j)} = \prod_{k \neq j} N_k$	Number of entries of $\mathcal{X}$ excluding mode j
$\mathbf{X}_{(j)} \in \mathbb{R}^{N_j \times N^{(-j)}}$	Mode- j unfolding of $\mathcal{X}$
$\mathbf{A} \otimes \mathbf{B}$	Kronecker product of $\mathbf{A} \in \mathbb{R}^{I \times J}$, $\mathbf{B} \in \mathbb{R}^{K \times L}$
$\mathbf{A} \odot \mathbf{B}$	Khatri-Rao product of $\mathbf{A} \in \mathbb{R}^{I \times K}$, $\mathbf{B} \in \mathbb{R}^{J \times K}$
$\mathbf{A} * \mathbf{B}$	Hadamard product of $\mathbf{A} \in \mathbb{R}^{I \times J}$, $\mathbf{B} \in \mathbb{R}^{I \times J}$

Table 4 Summary of tensor notation.

5.1 Notation and Definitions

A *tensor* $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times \dots \times N_d}$ is a d -way array with entries $\mathcal{X}_{i_1, i_2, \dots, i_d}$, for all $i_1 \in [N_1]$, $i_2 \in [N_2]$, $\dots$, $i_d \in [N_d]$. A d -way tensor is said to be of *order* d , and each of its d ways is referred to as a *mode*; a matrix, for instance, is a tensor of order 2, with mode 1 comprised of its rows and mode 2 comprised of its columns. A *mode- j fiber* of a tensor is a vector obtained by fixing all indices except for the j th, e.g., a column of a matrix is a mode-1 fiber, and a row is a mode-2 fiber. The *mode- j slices* of a tensor are the sub-tensors $\mathcal{X}(:, \dots, :, i_j, :, \dots, :)$ obtained by fixing a single index $i_j \in [N_j]$, e.g., slices of 3-mode tensors are matrices.

We define $N^{(-j)} = \prod_{k=1, k \neq j}^d N_k$ as the product of all dimensions except mode j , and $N = \prod_{j=1}^d N_j = \text{size}(\mathcal{X})$. Through a rearrangement of entries known as *matricization*, a d -mode tensor can be “unfolded” into a matrix. The *mode- j unfolding* of a d -mode tensor $\mathcal{X}$, denoted $\mathbf{X}_{(j)} \in \mathbb{R}^{N_j \times N^{(-j)}}$, $j = 1, \dots, d$, is a matrix whose columns are the mode- j fibers of $\mathcal{X}$. Throughout this section, we assume that the d modes are ordered $1, \dots, d$ for notational simplicity, but for computational reasons, it may be beneficial to re-order the modes for processing.

In our work, the (Frobenius) *norm* of a tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times \dots \times N_d}$, denoted by $\|\mathcal{X}\|$, is the square root of the sum of squares of all elements, analogous to the Frobenius norm for matrices. A tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times \dots \times N_d}$ is said to be *rank-one* if it can be expressed as a vector outer product (denoted by $\circ$) of d vectors, i.e. $\mathcal{X} = \mathbf{v}^{(1)} \circ \mathbf{v}^{(2)} \circ \dots \circ \mathbf{v}^{(d)}$.

5.2 Matrix and Tensor Products

We now define operations involving matrices and tensors used throughout our work.

The *Kronecker product* of matrices $\mathbf{A} \in \mathbb{R}^{I \times J}$ and $\mathbf{B} \in \mathbb{R}^{K \times L}$, denoted $\mathbf{A} \otimes \mathbf{B}$, is the $(IK) \times (JL)$ matrix with entries

$$(\mathbf{A} \otimes \mathbf{B})(K(i-1) + k, L(j-1) + \ell) = \mathbf{A}(i, j) \mathbf{B}(k, \ell).$$

The Kronecker product then takes the form

$$\begin{aligned} \begin{bmatrix} a_{1,1} & \cdots & a_{1,J} \\ \vdots & \ddots & \vdots \\ a_{I,1} & \cdots & a_{I,J} \end{bmatrix} \otimes \begin{bmatrix} b_{1,1} & \cdots & b_{1,L} \\ \vdots & \ddots & \vdots \\ b_{K,1} & \cdots & b_{K,L} \end{bmatrix} &= \begin{bmatrix} a_{1,1}b_{1,1} & \cdots & a_{1,1}b_{1,L} & \cdots & a_{1,J}b_{1,1} & \cdots & a_{1,J}b_{1,L} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{1,1}b_{K,1} & \cdots & a_{1,1}b_{K,L} & \cdots & a_{1,J}b_{K,1} & \cdots & a_{1,J}b_{K,L} \\ \vdots & & & & & & \vdots \\ \vdots & & & \ddots & & & \vdots \\ \vdots & & & & & & \vdots \\ a_{I,1}b_{1,1} & \cdots & a_{I,1}b_{1,L} & \cdots & a_{I,J}b_{1,1} & \cdots & a_{I,J}b_{1,L} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{I,1}b_{K,1} & \cdots & a_{I,1}b_{K,L} & \cdots & a_{I,J}b_{K,1} & \cdots & a_{I,J}b_{K,L} \end{bmatrix} \\ &\quad \mathbf{A} \quad \mathbf{B} \quad \mathbf{A} \otimes \mathbf{B} \\ &= \begin{bmatrix} a_{1,1} \mathbf{B} & \cdots & a_{1,J} \mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{I,1} \mathbf{B} & \cdots & a_{I,J} \mathbf{B} \end{bmatrix}. \end{aligned}$$

The *Khatri-Rao product* of matrices $\mathbf{A} \in \mathbb{R}^{I \times K}$ and $\mathbf{B} \in \mathbb{R}^{J \times K}$, denoted $\mathbf{A} \odot \mathbf{B}$, is the $(IJ) \times K$ matrix with entries

$$(\mathbf{A} \odot \mathbf{B})(:, k) = \mathbf{A}(:, k) \otimes \mathbf{B}(:, k)$$

for $k = 1, \dots, K$. The Khatri-Rao product takes the form

$$\begin{aligned}
\begin{bmatrix} a_{1,1} & \cdots & a_{1,K} \\ \vdots & \ddots & \vdots \\ a_{I,1} & \cdots & a_{I,K} \end{bmatrix} \odot \begin{bmatrix} b_{1,1} & \cdots & b_{1,K} \\ \vdots & \ddots & \vdots \\ b_{J,1} & \cdots & b_{J,K} \end{bmatrix} &= \begin{bmatrix} a_{1,1}b_{1,1} & a_{1,2}b_{1,2} & \cdots & a_{1,K}b_{1,K} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,1}b_{J,1} & a_{1,2}b_{J,2} & \cdots & a_{1,K}b_{J,K} \\ \vdots & \vdots & & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & & \vdots \\ a_{I,1}b_{1,1} & a_{I,2}b_{1,2} & \cdots & a_{I,K}b_{1,K} \\ \vdots & \vdots & \ddots & \vdots \\ a_{I,1}b_{J,1} & a_{I,2}b_{J,2} & \cdots & a_{I,K}b_{J,K} \end{bmatrix} \\
&\quad \mathbf{A} \quad \mathbf{B} \quad \mathbf{A \odot B} \\
&= [\mathbf{a}_1 \otimes \mathbf{b}_1 \quad \mathbf{a}_2 \otimes \mathbf{b}_2 \quad \cdots \quad \mathbf{a}_K \otimes \mathbf{b}_K],
\end{aligned}$$

where $\mathbf{a}_i$ and $\mathbf{b}_i$ are the columns of $\mathbf{A}$ and $\mathbf{B}$. There is a useful property relating the Kronecker and Khatri-Rao products: if $\mathbf{A} \in \mathbb{R}^{K \times J}$, $\mathbf{B} \in \mathbb{R}^{I \times J}$, $\mathbf{C} \in \mathbb{R}^{I \times K}$, and $\mathbf{D} \in \mathbb{R}^{J \times I}$,

$$(\mathbf{C} \otimes \mathbf{D})(\mathbf{A} \odot \mathbf{B}) = (\mathbf{CA}) \odot (\mathbf{DB}).$$

Additionally we have the following inequality of matrix norms:

$$\|\mathbf{A} \odot \mathbf{B}\|_F^2 \leq \|\mathbf{A} \otimes \mathbf{B}\|^2 = \|\mathbf{A}\|^2 \|\mathbf{B}\|^2.$$

The *Hadamard product* of matrices $\mathbf{A} \in \mathbb{R}^{I \times J}$ and $\mathbf{B} \in \mathbb{R}^{I \times J}$, denoted $\mathbf{A} * \mathbf{B}$, is the $I \times J$ matrix of entries

$$(\mathbf{A} * \mathbf{B})(i, j) = \mathbf{A}(i, j)\mathbf{B}(i, j).$$

The Hadamard product can be visualized as element-wise multiplication,

$$\begin{aligned}
\begin{bmatrix} a_{1,1} & \cdots & a_{1,J} \\ \vdots & \ddots & \vdots \\ a_{I,1} & \cdots & a_{I,J} \end{bmatrix} * \begin{bmatrix} b_{1,1} & \cdots & b_{1,J} \\ \vdots & \ddots & \vdots \\ b_{I,1} & \cdots & b_{I,J} \end{bmatrix} &= \begin{bmatrix} a_{1,1}b_{1,1} & \cdots & a_{1,J}b_{1,J} \\ \vdots & \ddots & \vdots \\ a_{I,1}b_{I,1} & \cdots & a_{I,J}b_{I,J} \end{bmatrix} \\
&\quad \mathbf{A} \quad \mathbf{B} \quad \mathbf{A * B}
\end{aligned}$$

The *tensor-times-matrix (TTM) product* is the tensor resulting from the mode- j product of a tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \cdots \times N_j \times \cdots \times N_d}$ with a matrix $\mathbf{U} \in \mathbb{R}^{K \times N_j}$, denoted by $\mathcal{Y} = \mathcal{X} \times_j \mathbf{U} \in \mathbb{R}^{N_1 \times \cdots \times N_{j-1} \times K \times N_{j+1} \times \cdots \times N_d}$, with entries

$$\mathcal{Y}(i_1, \dots, i_{j-1}, k, i_{j+1}, \dots, i_d) = \sum_{i_j=1}^{N_j} \mathcal{X}(i_1, \dots, i_d) \mathbf{U}(k, i_j).$$

We can express this more concisely with mode- j matricization: $\mathbf{Y}_{(j)} = \mathbf{U} \mathbf{X}_{(j)}$.

The *Multi-TTM product* extends the TTM product to multiple modes. For $\mathcal{X} \in \mathbb{R}^{N_1 \times \cdots \times N_j \times \cdots \times N_d}$ and matrices $\mathbf{U}_j \in \mathbb{R}^{K_j \times N_j}$ for $j \in [d]$, a Multi-TTM is given by

$$\mathcal{Y} = \mathcal{X} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \cdots \times_d \mathbf{U}_d.$$

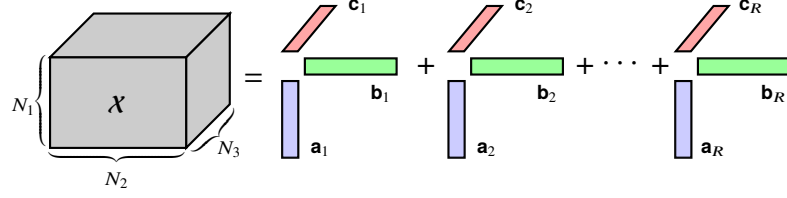


Fig. 1 Rank- R CP decomposition $\mathcal{X} = \sum_{r=1}^R \mathbf{a}_r \circ \mathbf{b}_r \circ \mathbf{c}_r$ for 3-mode tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times N_3}$.

The order of multiplication does not matter, cf. [21, Proposition 3.19]. Equivalently, after mode- j matricization,

$$\mathbf{Y}_{(j)} = \mathbf{U}_j \mathbf{X}_{(j)} (\mathbf{U}_d \otimes \cdots \otimes \mathbf{U}_{j+1} \otimes \mathbf{U}_{j-1} \otimes \cdots \otimes \mathbf{U}_1)^\top.$$

5.3 Tensor Decompositions

This work considers two fundamental formats of tensor decompositions: the canonical polyadic (CP) decomposition and the Tucker decomposition. We introduce them in Sect. 5.3.1 and 5.3.2, along with prototypical algorithms to compute them.

The CP decomposition has been known by many different names since its introduction in 1927: the polyadic form [132], PARAFAC (parallel factors) [124], CANDECOMP (canonical decomposition) [45]; see also [157, Table 3.1]. It is often viewed as a higher-order extension of the matrix SVD [157].

The Tucker decomposition [263, 264, 265] is a representation of a tensor as a Multi-TTM product of a core tensor with a factor matrix for each mode. Like the CP decomposition, the Tucker decomposition has been known by several different names in the literature [157, Table 4.1], and the Tucker decomposition can also be viewed as an extension of the matrix SVD, in the form of higher-order principal component analysis (PCA) [149, 163, 169, 269].

We briefly summarize a few key properties of these decompositions in Sect. 5.3.3 before discussing low-rank tensor approximations that can be expressed in each format in Sect. 5.3.4.

5.3.1 The Canonical Polyadic (CP) Decomposition

The CP decomposition of a tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \cdots \times N_d}$ expresses it as a sum of R rank-one tensors:

$$\mathcal{X} = \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \cdots \circ \mathbf{a}_r^{(d)}, \quad \text{where } \mathbf{a}_r^{(j)} \in \mathbb{R}^{N_j} \text{ for } j = 1, \dots, d. \quad (40)$$

For example, the CP decomposition of a 3-mode tensor is visualized in Fig. 1.

The *factor matrices* of the CP decomposition are formed from the vectors corresponding to each of the d modes. Namely, the mode- j factor matrix is given by

$$\mathbf{A}^{(j)} = \begin{bmatrix} \mathbf{a}_1^{(j)} & \mathbf{a}_2^{(j)} & \dots & \mathbf{a}_R^{(j)} \end{bmatrix} \in \mathbb{F}^{N_j \times R}.$$

We can express (40) in terms of mode- j unfoldings using the Khatri-Rao product:

$$\mathbf{X}_{(j)} = \mathbf{A}^{(j)} \left(\mathbf{A}^{(d)} \odot \dots \odot \mathbf{A}^{(j+1)} \odot \mathbf{A}^{(j-1)} \odot \dots \odot \mathbf{A}^{(1)} \right)^\top.$$

The notation of [155] is frequently used as shorthand for the CP decomposition:

$$\mathcal{X} = [\![\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(d)}]\!] := \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(d)}. \quad (41)$$

If we enforce unit length, a weight vector $\lambda \in \mathbb{F}^R$ may be incorporated into (41), so that

$$\mathcal{X} = [\![\lambda; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(d)}]\!] := \sum_{r=1}^R \lambda_r \mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(d)}. \quad (42)$$

In practice, the CP decomposition is often computed by the *alternating least squares* (ALS) method [45, 124]. Pseudocode for the general ALS procedure is given in Algorithm 5, which we quickly summarize.

Given a tensor $\mathcal{X} \in \mathbb{F}^{N_1 \times \dots \times N_d}$, the ALS procedure outputs a CP decomposition $\widehat{\mathcal{X}} = [\![\lambda; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)}]\!]$ with R rank-one components, once some convergence criteria is met. This is accomplished through a sequence of linear least squares problems, where all but one factor matrix is fixed:

$$\min_{\widehat{\mathbf{A}} \in \mathbb{F}^{N_j \times R}} \left\| \mathbf{X}_{(j)} - \widehat{\mathbf{A}} \left(\mathbf{A}^{(d)} \odot \dots \odot \mathbf{A}^{(j+1)} \odot \mathbf{A}^{(j-1)} \odot \dots \odot \mathbf{A}^{(1)} \right)^\top \right\|_F^2. \quad (43)$$

The solution of (43) is given by

$$\mathbf{A}^{(j)} := \mathbf{X}_{(j)} \left[\left(\mathbf{A}^{(d)} \odot \dots \odot \mathbf{A}^{(j+1)} \odot \mathbf{A}^{(j-1)} \odot \dots \odot \mathbf{A}^{(1)} \right)^\top \right]^\dagger, \quad (44)$$

where the columns of $\mathbf{A}^{(j)}$ are normalized to determine λ_r for $r = 1, \dots, R$. This process is repeated for each mode until some convergence criterion is met; see Sect. 5.3.3.

If Algorithm 5 is naively implemented, the per-iteration cost to compute a CP decomposition is dominated by $N = \text{size}(\mathcal{X})$, which is computationally undesirable. Fortunately, since its introduction in 1970, CP-ALS has benefited from many modifications for improved efficiency, cf. Sect. 6.1 for recent algorithmic developments.

One strategy that has garnered attention for efficient CP-ALS incorporates a preprocessing step to compress the input tensor before executing Algorithm 5; this method is known as CANDELINC [36, 46]. The compression is often achieved through a Tucker decomposition, which we discuss next.

Algorithm 5 Alternating Least Squares for CP Decomposition (CP-ALS)

Require: Input tensor $\mathcal{X} \in \mathbb{F}^{N_1 \times \dots \times N_d}$, number of rank-one components R

Ensure: CP decomposition $\hat{\mathcal{X}} = \llbracket \lambda; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

- 1: Initialize $\mathbf{A}^{(j)} \in \mathbb{F}^{N_j \times R}$, $j = 1, \dots, d$ ▷ e.g., $\mathbf{A}^{(j)} = \text{randn}(N_j, R)$
 - 2: **while** not converged **do**
 - 3: **for** $j = 1, \dots, d$ **do**
 - 4: $\mathbf{A}^{(j)} = \mathbf{X}_{(j)} \left[\left(\mathbf{A}^{(d)} \odot \dots \odot \mathbf{A}^{(j+1)} \odot \mathbf{A}^{(j-1)} \odot \dots \odot \mathbf{A}^{(1)} \right)^\top \right]^\dagger$
 - 5: $\lambda_r = \left\| \mathbf{A}^{(j)}(:, r) \right\|_2$, $r = 1, \dots, R$
 - 6: $\mathbf{A}^{(j)}(:, r) = \mathbf{A}^{(j)}(:, r) / \lambda_r$, $r = 1, \dots, R$
 - 7: Update $\hat{\mathcal{X}} = \llbracket \lambda; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$
 - 8: **end for**
 - 9: **end while**
-

5.3.2 Tucker Decomposition

The Tucker decomposition expresses a tensor $\mathcal{X} \in \mathbb{F}^{N_1 \times \dots \times N_d}$ as a d -way Multi-TTM product:

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \dots \times_d \mathbf{A}^{(d)} =: \llbracket \mathcal{G}; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(d)} \rrbracket, \quad (45)$$

where $\mathcal{G} \in \mathbb{F}^{R_1 \times \dots \times R_d}$ is called the core tensor, and $\mathbf{A}^{(j)} \in \mathbb{F}^{N_j \times R_j}$ is the factor matrix for mode $j = 1, \dots, d$. We can express (45) in terms of mode- j unfoldings by

$$\mathbf{X}_{(j)} = \mathbf{A}^{(j)} \mathbf{G}_{(j)} \left(\mathbf{A}^{(d)} \otimes \dots \otimes \mathbf{A}^{(j+1)} \otimes \mathbf{A}^{(j-1)} \otimes \dots \otimes \mathbf{A}^{(1)} \right)^\top. \quad (46)$$

A Tucker decomposition of a 3-mode tensor is visualized in Fig. 2.

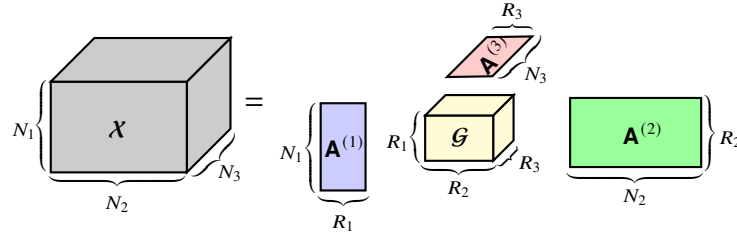


Fig. 2 Rank- (R_1, R_2, R_3) Tucker decomposition $\mathcal{X} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \mathbf{A}^{(3)}$ of $\mathcal{X} \in \mathbb{F}^{N_1 \times N_2 \times N_3}$.

A prototypical algorithm to compute the Tucker decomposition is Tucker’s “Method 1” [265], which has come to be known as the *higher-order SVD* (HOSVD) [72], summarized in Algorithm 6. For an input tensor $\mathcal{X} \in \mathbb{F}^{N_1 \times \dots \times N_d}$, each factor matrix $\mathbf{A}^{(j)}$ is computed as the leading- R_j left singular vectors of the mode- j unfolding $\mathbf{X}_{(j)}$. If $R_j < \text{rank}(\mathbf{X}_{(j)})$, the decomposition is said to be “truncated.” When $R_j \ll N_j$, the Tucker decomposition is said to be a compressed representation of the input tensor.

Once the factor matrices are computed, the core tensor is obtained as a Multi-TTM product of the original tensor with the factor matrix transposes. The computational cost of the HOSVD is $O(dn^{d+1} + \sum_{j=1}^d k^j n^{d-j+1})$, letting $n = N_1 = \dots = N_d$ and $k = R_1 = \dots = R_d$ for notational simplicity.

For improved efficiency, the *sequentially truncated HOSVD* (ST-HOSVD) is often used [118, 268]. In the ST-HOSVD, once a factor matrix is computed for mode j , the input tensor is compressed via mode- j TTM before the next factor matrix is computed; i.e., for input tensor $\mathcal{X}$, we first compute $\mathbf{A}^{(1)}$ and form $\mathcal{Y} = \mathcal{X} \times_1 \mathbf{A}^{(1)}$, then compute $\mathbf{A}^{(2)}$ from $\mathcal{Y}$, then form $\mathcal{Z} = \mathcal{Y} \times_2 \mathbf{A}^{(2)}$, and so on. ST-HOSVD successively reduces the size of the tensor used in subsequent computations, and the core tensor is available immediately after computing the factor matrix for the last mode. The computational cost of ST-HOSVD is $O(\sum_{j=1}^d k^{j-1} n^{d-j+2} + k^J n^{d-J})$.

Unlike the matrix SVD, the (ST-)HOSVD does not yield optimal approximation error. It has been shown that the relative error with (ST-)HOSVD is within $\sqrt{d}$ of an optimal Tucker decomposition of a given rank for a d -mode tensor, cf. [21, Chapter 7]. However, we can use the HOSVD output as a starting point for an ALS-style algorithm to compute Tucker decompositions, known as *higher-order orthogonal iteration* (HOOI) [163, 72]. Pseudocode for HOOI is given in Algorithm 7.

As in CP-ALS, in HOOI, we seek to minimize the approximation error $\|\mathcal{X} - \hat{\mathcal{X}}\|$, now over $\mathcal{G}$ and $\mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)}$, subject to the constraints that $\mathcal{G} \in \mathbb{R}^{R_1 \times \dots \times R_d}$ and that $\mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R_j}$ is orthogonal for $j = 1, \dots, d$. This tensorial minimization problem can be transformed into matrix least squares problems in each mode j , where we solve for each factor matrix via

$$\mathbf{A}^{(j)} = \arg \min_{\hat{\mathbf{A}} \in \mathbb{R}^{N_j \times R_j}} \left\| \left(\mathbf{A}^{(d)} \otimes \dots \otimes \mathbf{A}^{(j+1)} \otimes \mathbf{A}^{(j-1)} \otimes \dots \otimes \mathbf{A}^{(1)} \right) \mathbf{G}_{(j)}^{\top} \hat{\mathbf{A}}^{\top} - \mathbf{X}_{(j)}^{\top} \right\|_F^2.$$

Note the optimal solution to this least squares problem is given by the leading left singular vectors of each unfolding $\mathbf{X}_{(j)}$ [11, 73, 155]. The core tensor can be computed as a column vector $\mathbf{g} = \arg \min_{\mathbf{z} \in \mathbb{R}^{\prod_j R_j \times 1}} \left\| \left(\bigotimes_{j=d}^1 \mathbf{A}^{(j)} \right) \mathbf{z} - \mathbf{x} \right\|_2^2$, where $\mathbf{x}$ is the vectorization or unfolding of tensor $\mathcal{X}$ into a large column vector of size $N \times 1$. To get $\mathcal{G}$, we “re-fold” $\mathbf{g} \in \mathbb{R}^{\prod_j R_j}$ back into a tensor of dimension $\mathbb{R}^{R_1 \times \dots \times R_d}$. If the factor matrices are computed as the leading R_j left singular vectors of unfoldings, then the optimal core tensor is given by $\mathcal{G} = \mathcal{X} \times_1 \left(\mathbf{A}^{(1)} \right)^{\top} \times \dots \times_d \left(\mathbf{A}^{(d)} \right)^{\top}$. The per-iteration

Algorithm 6 Higher-order SVD for Tucker Decomposition (HOSVD)

Require: Input tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, factor matrix target ranks $R_1, \dots, R_d$

Ensure: Tucker decomposition $\hat{\mathcal{X}} = \llbracket \mathcal{G}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

- 1: **for** $j = 1, \dots, d$ **do**
 - 2: $[\mathbf{A}^{(j)}, \sim, \sim] = \text{svd}(\mathbf{X}_{(j)}, R_j)$
 - 3: **end for**
 - 4: $\mathcal{G} = \mathcal{X} \times_1 \left(\mathbf{A}^{(1)} \right)^{\top} \times \dots \times_d \left(\mathbf{A}^{(d)} \right)^{\top}$
-

Algorithm 7 Higher-order Orthogonal Iteration for Tucker Decomposition (HOOI)

Require: Input tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, factor matrix ranks $R_1, \dots, R_d$

Ensure: Tucker decomposition $\hat{\mathcal{X}} = \llbracket \mathcal{G}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

- 1: Initialize $\mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R_j}$, $j = 2, \dots, d$
 - 2: **while** not converged **do**
 - 3: **for** $j = 1, \dots, d$ **do**
 - 4: $\mathbf{A}^{(j)} = \arg \min_{\mathbf{A} \in \mathbb{R}^{N_j \times R_j}} \left\| \left(\bigotimes_{i=d, i \neq j}^1 \mathbf{A}^{(i)} \right) \mathbf{G}_{(j)}^\top \mathbf{A}^\top - \mathbf{X}_{(j)}^\top \right\|_F^2$
 - 5: **end for**
 - 6: $\mathbf{g} = \arg \min_{\mathbf{z} \in \mathbb{R}^{\prod_{j=1}^d R_j \times 1}} \left\| \left(\bigotimes_{j=d}^1 \mathbf{A}^{(j)} \right) \mathbf{z} - \mathbf{x} \right\|_2^2$ ▷ Column vectorization $\mathbf{x}$ of $\mathcal{X}$
 - 7: Re-fold vectorization $\mathbf{g}$ into $\mathcal{G} \in \mathbb{R}^{R_1 \times \dots \times R_d}$
 - 8: **end while**
-

cost of Algorithm 7 is $O(dkn^d + \sum_{j=1}^d k^j n^{d-j+1})$, letting $k = R_1 = \dots = R_d$ and $n = N_1 = \dots = N_d$.

While Algorithm 7 will converge to a local solution where the approximation error no longer decreases, it may not converge to the global optimum [73, 163]. Modified versions with better guarantees have been proposed, such as HOOI with Newton-Grassmann optimization [96], but these come at greater computational cost. The Tucker decomposition may also be used as a preprocessing compression step to achieve better performance in CP-ALS, known as CANDELINC or a Tucker+CP approach [36].

In the next section, we offer some practical guidelines for choosing a tensor format and target rank for a given problem, beginning with a discussion of tensor rank and ending with a comparison of the presented tensor formats.

5.3.3 Rank, Uniqueness, and the Choice of Tensor Representation

The *rank* of a tensor $\mathcal{X}$ is the minimal number of rank-one tensors that are required for equality in (41):

$$\text{rank}(\mathcal{X}) = \min\{R \in \mathbb{N} \mid \mathcal{X} = \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket, \mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R} \forall j \in [d]\}. \quad (47)$$

Mode ordering and TTM products with nonsingular matrices do not affect tensor rank. However, it is NP-hard to determine [136].

In practice, the target ranks for low-rank tensor decompositions are chosen heuristically. For example, we might choose the smallest rank R for a CP decomposition that significantly reduces the relative error, in comparison to rank $R - 1$, cf. [21, Section 9.4.1]. Generally, the optimization procedure in CP-ALS is repeated until the error no longer decreases or a maximum number of iterations is reached.

For Tucker decompositions, it is useful to consider the j -rank of a tensor, denoted by $\text{rank}_j(\mathcal{X})$ and defined as the column rank of its mode- j unfolding $\mathbf{X}_{(j)}$ [165, 72]. If $R_j = \text{rank}_j(\mathcal{X})$ for $j = 1, \dots, d$, then we say that $\mathcal{X}$ is a rank- $(R_1, \dots, R_d)$ tensor (distinct from its rank, defined above). We can compute an exact rank- $(R_1, \dots, R_d)$ Tucker decomposition for any tensor $\mathcal{X}$ with $R_j = \text{rank}_j(\mathcal{X})$ for $j = 1, \dots, d$. However, if $R_j < \text{rank}_j(\mathcal{X})$ for any j , the decomposition is inexact and more difficult to compute.

In practice, we often prescribe a core j -rank for $j = 1, \dots, d$ that meets some error tolerance or compression ratio [21, Section 4.2]. The (ST-)HOSVD algorithm has the nice property that it computes a rank- $(R_1, \dots, R_d)$ decomposition with an approximation error that is within $\sqrt{d}$ of the optimal rank- $(R_1, \dots, R_d)$ approximation error.

Whether it is better to represent an input tensor in the CP or Tucker format is generally problem-dependent. Under mild assumptions, the CP decomposition of a tensor is unique, up to scaling and permutation of its rank-one components [26, 125, 124, 164, 165, 238]. The same cannot be said for Tucker decompositions; the mode- j factor matrix can be multiplied by a nonsingular matrix and the core tensor updated via mode- j product of the original core tensor with the inverse of the nonsingular matrix. As a result, the factors of a Tucker decomposition are less interpretable in the context of the original data than those of a CP decomposition, cf. [21, Section 9.6–9.8].

However, as previously mentioned, the Tucker decomposition is highly advantageous for data compression, even as a preprocessing step for a CP decomposition, e.g., CANDELINC [36, 46]. Despite its utility in data compression, the Tucker decomposition still suffers from the curse of dimensionality: letting $k = R_1 = \dots = R_d$ and $n = N_1 = \dots = N_d$, the cost of storing a Tucker decomposition is $O(k^d + dkn)$. By contrast, the storage cost of a CP decomposition is linear in its rank, but its rank may be very large to achieve the desired approximation accuracy.

In short, the notion of rank for tensor decompositions is more nuanced than for matrices, and the choice of representation and target rank (or j -rank) should be informed by the desired trade-off between approximation accuracy and computational performance. For more detailed treatments, we refer the reader to [21, 72, 112, 157].

5.3.4 Low-Rank Tensor Factorizations

In the previous sections, we introduced two fundamental tensor representations, the CP and the Tucker decomposition, and we summarized their key properties. In this section, we survey low-rank tensor factorizations that can be expressed in each format, which we characterize primarily by whether or not the factor matrices are orthogonal. Namely, we assume that the standard Tucker decomposition has orthonormal factor matrices, whereas CP decompositions generally do not. Higher-order versions of ID/CUR may be expressed using orthogonal factor matrices or factor matrices that contain entries of the input tensor, so we further characterize them by whether the core subtensor or the factor matrices contain entries of the original tensor.

Orthogonal factor matrices

Recall from Sect. 5.3.2 that the (ST-)HOSVD and HOOI methods used to compute Tucker decompositions return orthonormal factor matrices $\mathbf{A}^{(j)}$ for $j = 1, \dots, d$. Alternatively, there are algorithms for Tucker decompositions where orthogonality of factor matrices is not enforced, such as algorithms for the tensor ID/CUR factorizations in which factor matrices are computed as sub-matrices of unfoldings. Other examples beyond the scope of the current work include diagonalization algorithms for

Tucker decompositions [248, 249] or algorithms that compute non-negative or sparse Tucker factorizations [152, 198]. These algorithms may be preferable in specific application domains, such as image processing [240, 293], but in general, many tensor computations are greatly simplified when the Tucker factor matrices are orthogonal. Moreover, the (ST-)HOSVD with enforced orthogonality comes with quasi-optimality guarantees. As such, with the exception of the tensor ID/CUR, we focus on algorithms for Tucker decompositions that compute orthogonal (in fact, orthonormal) factor matrices. There is also historical precedence for this decision, as the seminal algorithms to compute Tucker decompositions enforced the orthogonality of factor matrices [72, 73, 92, 153, 264, 265, 268].

Finally, we note that the factor matrices in a CP decomposition are not orthogonal in general. Some tensors may admit CP decompositions with orthogonal factors [10, 228], called ODECO tensors; however, ODECO tensors occur with probability 0 [153, 154, 156]. This is in contrast with Tucker factor matrices, which can be transformed straightforwardly into orthogonal matrices, cf. [21, Sections 5.4, 17.1.4]. As such, in this work, we focus solely on Tucker decompositions in our discussion of randomized algorithms to compute orthogonal factor matrices in Sect. 6.

Tensor ID and CUR factorizations

As in the matrix setting (Sect. 2.4), a tensor ID or CUR factorization comprises actual entries of the input tensor. The advantages of ID/CUR factorizations for tensors are similar to those for matrices. The tensor ID/CUR factorization can be interpreted more easily in the context of the original tensorial data than factorizations involving multilinear transformations of the data. Moreover, the ID/CUR preserves the structure of the input tensor (e.g., sparsity or non-negativity), and it can be computed efficiently with minimal storage requirements.

Analogous to the different versions of matrix ID/CUR (e.g., row, column, double-sided, CUR), there are several variants in the tensor setting that have been investigated since the ID/CUR was first extended to 3-mode tensors in [92]. In general, these variants fall into one of two categories:

1. The core tensor (in the Tucker format) is a sub-tensor of the input tensor, or
2. The factor matrices (in the CP or Tucker format) are sub-matrices of unfoldings.

The work of [180] that introduced “tensor CUR factorizations” falls into the first category. In their algorithm, a rank- (N_1, N_2, R) core tensor is constructed by selecting R indices from one mode and keeping all indices from the other two. Core tensor skeleton selection methods for 3-mode tensor CUR were generalized to d -mode tensors in [42], for Tucker decompositions with the same j -rank in each mode. The “structure-preserving ST-HOSVD” of [196], initially developed for sparse rank- $(R_1, \dots, R_d)$ tensors, may also be interpreted as a tensor CUR factorization of the first category. The “Chidori” and “Fiber” tensor CUR factorizations with rank- $(R_1, \dots, R_d)$ core tensors were characterized in [40], inspired by those in [42]; these factorizations were utilized for robust tensorial principal component analysis in [39]. The recently proposed “CoreID” in [287] is similar to the CUR factorizations of [40]. In all of these methods, the

entries of the associated factor matrices are computed based on the specific form of the factorization and on the skeletons selected for the core tensor, but the factor matrices are not themselves sub-matrices of unfoldings.

The second category above comprises those tensor ID/CUR factorizations in which the entries of the factor matrices are entries of the input tensor. The early algorithm of [92] to compute a rank- $(R_1, \dots, R_d)$ “approximate tensor SVD” forms the factor matrices in a Tucker decomposition as randomly sampled columns of unfoldings; this factorization would later be termed a “higher-order ID” in [235], where the mode- j factor matrix in a Tucker decomposition is computed as column skeletons of the mode- j unfolding selected by greedy pivoting. In [103], an HOOI procedure is developed for 3-mode Tucker tensors that yields factor matrices of original tensor entries, and the “CURT” algorithm of [289, 242] computes tensor CUR decompositions in the Tucker format with strong accuracy guarantees. The CURT algorithm was recently generalized in [287] to compute rank- $(R_1, \dots, R_d)$ Tucker decompositions where the core tensor is unconstrained and the factor matrices are column skeletons of mode unfoldings; this algorithm is also suitable for input tensors expressed in the CP format. The tensor ID algorithm of [29] computes a CP decomposition of a d -mode tensor, in which factor matrices are comprised of entries of the input tensor, as does the more recent work of [183]. We detail several algorithms that leverage randomization to compute tensor ID/CUR factorizations efficiently in Sect. 6.

6 Randomized Low-Rank Tensor Decompositions

As in the matrix setting, we are interested in algorithms that compute low-rank tensor decompositions accurately and efficiently, with provable performance guarantees that are substantiated by empirical evidence. Because many tensor algorithms fundamentally rely on classical matrix algorithms, we can frequently fall back on the ideas introduced in Sect. 4 to leverage randomization for efficient tensor decompositions. However, given the rapidly increasing number of applications that involve large-scale tensorial data, it is often impractical or infeasible to rely on basic implementations of the randomized rangefinder, e.g., with Gaussian sketching. The curse of dimensionality for d -way arrays is even more pernicious than for matrices, so great care must be taken to develop randomized algorithms that are not only accurate, but also scalable.

In this section, we survey new developments in fast randomized algorithms for low-rank tensor decompositions in the CP and Tucker formats. We draw connections and refer back to our presented matrix algorithms when relevant, augmenting them with emerging ideas from the recent proliferation of work on randomized algorithms for tensors. Sect. 6.1 explores how CP-ALS can be accelerated with randomization, specifically through fast sampling methods that exploit the structure of the overdetermined least squares problems solved in each iteration. Sect. 6.2 summarizes randomized methods to compute Tucker decompositions having orthogonal factor matrices, which naturally extend the randomized rangefinder to the tensor setting via the standard randomized (ST-)HOSVD and randomized HOOI, as well as a newly proposed algorithm for more efficient sketching. These randomized algorithms are contrasted with those for tensor

ID/CUR decompositions in Sect. 6.3, which includes both the CP and Tucker formats. Our intent throughout this section is not to provide an exhaustive summary of all recent developments, but rather to broadly survey them to direct the interested reader to the relevant literature, and then hone in on key ideas that may stimulate future work in these emergent areas.

6.1 Randomized Algorithms for CP Decomposition

Over the last decade, many randomized algorithms have been proposed to compute CP decompositions more efficiently. We survey several notable algorithms, before summarizing the recent randomized CP-ALS procedure of [168], which underlies the high-performance algorithm of [27], as an illustration of how randomization can be leveraged for efficient CP decompositions.

Many randomized algorithms for CP decompositions seek to exploit the structure of the Khatri-Rao product (KRP) in CP-ALS, cf. (44), for fast sketching and sampling. In [276], the TensorSketch algorithm of [222] is used to sketch the input tensor without explicitly forming it, then the KRP is efficiently computed. The “randomized ALS” algorithm of [227] incorporates projections onto random tensors to improve the conditioning of the least squares problems in CP-ALS. A procedure to sample rows of the KRP near-optimally via fast leverage score approximations is presented in [65]; however, the least squares problem is not sketched in this work (only the KRP). In [23], a Kronecker fast Johnson-Lindenstrauss transform (KFJLT) is applied to the KRP to reduce the coherence for better sampling performance, cf. [143, 184], which is then utilized in a sketched ALS problem without explicitly forming the KRP. The algorithm of [3] builds on [23] with options for adaptive sketch sizes and regularization terms. We now expand on the recent work of [168] on approximate leverage score sampling that exploits the structure of the KRP.

6.1.1 Randomized CP-ALS with Fast Leverage Score Sampling

Recall from Sect. 5.3.1 that within each iteration of CP-ALS (Algorithm 5), we compute a rank- R CP decomposition that approximates our input tensor by solving d least squares problems in succession. More precisely, given input tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, in each iteration of CP-ALS, we solve a least squares problem in each mode $j = 1, \dots, d$, in which every factor matrix except $\mathbf{A}^{(j)}$ is fixed, and $\mathbf{A}^{(j)}$ is computed as the solution of

$$\min_{\hat{\mathbf{A}} \in \mathbb{R}^{N_j \times R}} \left\| \hat{\mathbf{A}} \mathbf{Z}^\top - \mathbf{X}_{(j)} \right\|_F^2 = \min_{\hat{\mathbf{A}} \in \mathbb{R}^{N_j \times R}} \left\| \mathbf{Z} \hat{\mathbf{A}}^\top - \mathbf{X}_{(j)}^\top \right\|_F^2, \quad (48)$$

where $\mathbf{Z} = \mathbf{A}^{(d)} \odot \dots \odot \mathbf{A}^{(j+1)} \odot \mathbf{A}^{(j-1)} \odot \dots \odot \mathbf{A}^{(1)} \in \mathbb{R}^{N^{(-j)} \times R}$.

If we solve (48) without exploiting any inherent problem structure, the cost of computing the QR decomposition of $\mathbf{Z}$ is $O(R^2 N^{(-j)})$, and the cost of applying it N_j times brings the total cost to $O(R^2 N^{(-j)} + RN)$.

In [168], a randomized sampling strategy based on fast leverage score approximation, CP-ARLS-LEV, is used to accelerate each least squares sub-problem (48). Namely, $S \ll N^{(-j)}$ rows of $\mathbf{Z}$ are sampled according to an approximate leverage score distribution. The corresponding S rows of $\mathbf{X}_{(j)}^\top$ are then selected for a smaller “sketched” problem whose solution is a good approximation of the solution of (48) with high probability. Sketching reduces the cost of solving the j th least squares problem to $O(R^2S + RSN_j)$. However, forming $\mathbf{Z}$ or $\mathbf{X}$ and computing leverage scores would incur a cost of $O(R^2N^{(-j)})$. To avoid this, in CP-ARLS-LEV, the leverage scores are estimated as the product of leverage scores of the respective factor matrices in the KRP, and the KRP structure is exploited so that only the S selected rows of $\mathbf{Z}$ are formed explicitly.

To this end, let $j \in [d]$ be fixed. We first observe that there is a bijection mapping each row $i \in [N]$ of $\mathbf{Z}$ to a $(d-1)$ -tuple $(i_d, \dots, i_{j+1}, i_{j-1}, \dots, i_1)$, with $i_k \in [N_k]$, $k \neq j$, which indexes rows of factor matrices in the KRP:

$$\mathbf{Z}(i, :) = \mathbf{A}^{(d)}(i_d, :) * \dots * \mathbf{A}^{(j+1)}(i_{j+1}, :) * \mathbf{A}^{(j-1)}(i_{j-1}, :) * \dots * \mathbf{A}^{(1)}(i_1, :), \quad (49)$$

where $*$ again denotes the Hadamard product. Using [65, Theorem 3.3], the leverage scores of the rows of $\mathbf{Z}$ are bounded above by the product of the leverage scores of the rows of its constituent factor matrices:

$$\ell_i(\mathbf{Z}) \leq \hat{\ell}_i(\mathbf{Z}) := \prod_{k \neq j} \ell_{i_k}(\mathbf{A}^{(k)}). \quad (50)$$

From (50), we define a probability distribution on the rows of $\mathbf{Z}$; namely, row i is selected with probability

$$p_i = \hat{\ell}_i(\mathbf{Z}) / R^{d-1}. \quad (51)$$

Computing the probabilities in (51) explicitly would require the Kronecker product of leverage scores, costing $O(N^{(-j)})$ operations. However, [168, Lemma 9] establishes that each i_k in the $(d-1)$ -tuple corresponding to row i of $\mathbf{Z}$ can be sampled independently, using the leverage scores of the rows of $\mathbf{A}^{(k)}$, $k \neq j$, which cost $O(N_k R^2)$ to compute. This gives a multi-index corresponding to the i -th row of $\mathbf{Z}$ without computing the Kronecker product of leverage scores, and $\mathbf{Z}(i, :)$ can be formed in $O(R(d-1))$ operations using (49). There are additional modifications that can be made to the CP-ARLS-LEV algorithm to further improve performance; we refer the reader to [168] for details. The basic pseudocode for this procedure is given in Algorithm 8.

The CP-ARLS-LEV algorithm enjoys strong theoretical guarantees. Namely, the resulting rank- R CP decomposition is an ε -accurate approximation to the input tensor with high probability when the number of samples $S = O(R^{d-1}/\varepsilon)$. In particular, CP-ARLS-LEV improves on the sampling complexity required by the KFJLT-based method of [143]. A distributed-memory implementation of CP-ARLS-LEV is also developed in [27], along with a distributed-memory version of an algorithm that performs random walks on a binary tree for fast leverage score estimation instead.

The recent work of [181] improves on the sampling complexity of CP-ARLS-LEV by avoiding exponential dependence on d , using the fast leverage score estimation tech-

Algorithm 8 Alternating Randomized Least Squares for CP with Leverage Scores (CP-ARLS-LEV)

Require: Input tensor $\mathbf{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, number of rank-one components R , number of samples S

Ensure: CP decomposition $\widehat{\mathbf{X}} = \llbracket \lambda; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

```

1: Initialize  $\mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R}$ ,  $j = 1, \dots, d$  ▷ e.g.,  $\mathbf{A}^{(j)} = \text{randn}(N_j, R)$ 
2: for  $j = 1, \dots, d$  do
3:   Initialize  $\mathbf{p}_j = \ell(\mathbf{A}^{(j)})/R$ , the vector of row leverage scores for  $\mathbf{A}^{(j)}$ 
4: end for
5: while not converged do
6:   for  $j = 1, \dots, d$  do
7:      $I = \emptyset$ 
8:     while  $|I| < S$  do
9:       for  $k = 1, \dots, j-1, j+1, \dots, d$  do
10:        Sample  $i_k \sim \mathbf{p}_k$  from  $[N_k]$ 
11:       end for
12:       Add  $i = (i_1, \dots, i_{j-1}, i_{j+1}, \dots, i_d)$  to  $I$ 
13:     end while
14:     Form submatrix  $\widehat{\mathbf{Z}} = \mathbf{Z}(I, :)$ 
15:     Form submatrix  $\widehat{\mathbf{X}}_{(j)}^* = \mathbf{X}_{(j)}^*(I, :)$ 
16:     Compute  $\mathbf{A}^{(j)} = \arg \min_{\mathbf{A} \in \mathbb{R}^{N_j \times R}} \|\widehat{\mathbf{Z}}\mathbf{A}^* - \widehat{\mathbf{X}}_{(j)}^*\|_F^2$ 
17:     Update  $\mathbf{p}_j = \ell(\mathbf{A}^{(j)})/R$ 
18:   end for
19: end while
```

nique of [91], but replacing the FJLT with recursive sketching [4]. This strategy yields a sampling distribution closer to the exact leverage score distribution, but performs similarly to CP-ARLS-LEV in practice, and its implementation details are beyond the scope of the current manuscript; we refer the reader to [4, 181]. There has also been success in using stochastic gradient descent on the least squares objective function in (48), cf. [158, 274, 282]. Randomized algorithms to compute low-rank Tucker decompositions as a pre-processing step for CP-ALS have also been investigated [100, 280, 290]. We discuss several of these algorithms in the next section.

6.2 Randomized Algorithms for Tucker Decompositions with Orthogonal Factors

Because the Tucker decomposition provides a natural framework for data compression, methods based on the HOSVD (Algorithm 6) and HOOI (Algorithm 7) have been utilized in many data science and scientific computing applications. However, the Tucker decomposition suffers from the curse of dimensionality, thus classical algorithms can be prohibitively expensive for large problems. As randomized algorithms, particularly the randomized SVD (Algorithm 3), have become mainstays for fast and reliable matrix compression, it is natural to consider how randomization can be leveraged for fast and reliable tensor compression in the Tucker format. In this section, we survey recent work

on randomized algorithms for Tucker decompositions with orthogonal factor matrices (as opposed to interpolating, cf. Sect. 5.3.4). We begin with an overview of major developments in randomized Tucker decompositions, before detailing two algorithms based on the randomized SVD: the randomized HOSVD and the randomized HOOI.

Randomized algorithms for Tucker decompositions have been investigated in many problem settings, generally involving either randomized sampling or sketching. One of the earliest works on randomized HOSVD and HOOI algorithms is [262], called the MACH-HOSVD and MACH-HOOI, so named for the work of Achlioptas-McSherry [2] that inspired these randomized sampling algorithms. In MACH-HOSVD and MACH-HOOI, the input tensor is sparsified according to a coin flip for each nonzero entry; with probability p , a nonzero entry is kept and reweighted by $1/p$, and with probability $1 - p$, the entry is set to 0. The HOSVD and HOOI algorithms are then performed on the sparsified tensor, outputting a Tucker decomposition that approximates the original tensor well in practice, even for small values of p , e.g., $p = 0.1$. Leverage score sampling of matrix unfoldings to sparsify the tensor is explored in [218], and Tucker decompositions of sub-sampled tensors are also investigated in [207, 271]. A detailed survey of these methods can be found in [5].

As the randomized rangefinder gained traction in the numerical linear algebra community, many investigations began to explore randomized sketching for fast Tucker decompositions. In [290], the RandTucker algorithm is presented, which relies on the randomized rangefinder to compute an HOSVD from Gaussian sketches of mode unfoldings. Formal analysis of RandTucker is performed in [100], which incorporates power iteration into RandTucker for faster spectral decay; it is noted, though, that uniform sampling instead of Gaussian sketching performs similarly and may be preferable for large problems. Power iteration and randomized sketching are utilized in the related work of [54, 55] for randomized HOSVD and HOOI, which build on the rank-adaptive algorithm of [53] that iteratively computes an orthonormal basis for the range of each mode unfolding with ε -accuracy.

Rank-adaptive versions of the randomized (ST-)HOSVD are also presented in [196], which apply the adaptive randomized rangefinder to each mode unfolding. In [244], a single-pass algorithm based on the HOSVD is developed that significantly improves on the cost of storing the random sketching matrices, via Khatri-Rao products of random dimension reduction maps (called tensor random projections); details can also be found in [261]. The Sketch-STHOSVD algorithm of [84] is inspired by [244] but incorporates power iteration, whereas the Tucker-TS and Tucker-TTMTS algorithms of [182] are single-pass variants of HOOI using TensorSketch [211]. Cost-effective sketching for Tucker decompositions is also investigated in [37], which replaces the randomized SVD step in computing approximate bases with generalized Nyström. In the recent work of [126], the Randomized Tucker with Single-Mode Sketching (RTSMS) algorithm is introduced as a rank-adaptive method that sequentially builds approximate bases of unfoldings by applying small random sketches one mode at a time, similar to the randomized ST-HOSVD of [196], which we describe next.

6.2.1 Randomized (ST-)HOSVD

In the HOSVD (Algorithm 6), we compute factor matrices for a Tucker decomposition as the left singular vectors of each mode unfolding. The randomized HOSVD is a natural extension, where we substitute the SVD of each unfolding with the randomized SVD (Algorithm 3). Pseudocode is given in Algorithm 9, this time for the ST-HOSVD to highlight the slight difference in its computation vs. the HOSVD; the R-HOSVD algorithm of [196] is precisely the HOSVD algorithm, but using the randomized SVD. Note that this algorithm can be made adaptive by using the adaptive randomized rangefinder with a prescribed error tolerance for the low-rank decomposition of each unfolding.

If $\hat{\mathcal{X}}$ is the output of Algorithm 9 (or the randomized HOSVD, with Gaussian sketching), it is proven in [196, Theorem 3.1, 3.2] that the expected error is

$$\mathbb{E}_{\{\Omega^{(j)}\}_{j=1}^d} \|\mathcal{X} - \hat{\mathcal{X}}\| \leq \left(d + \frac{\sum_{j=1}^d R_j}{p-1} \right)^{1/2} \|\mathcal{X} - \hat{\mathcal{X}}_{opt}\|. \quad (52)$$

Letting $n = N_1 = \dots = N_d$ and $k = R_1 = \dots = R_d$ for notational simplicity, the computational cost of R-STHOSVD as in Algorithm 9 is $O(\sum_{j=1}^d k^j n^{d-j+1} + k^j n^{d-j})$. By contrast, R-HOSVD (substituting the SVD with the randomized SVD in Algorithm 6) has computational cost $O(dkn^d + \sum_{j=1}^d k^j n^{d-j+1})$. In other words, the cost is reduced roughly by a factor of n when using randomized (ST-)HOSVD vs. deterministic. Algorithmic performance can also be improved in the R-STHOSVD by processing the largest modes first, and while the error of a given output does depend on processing order, the worst-case expected error does not, cf. [196, Sect. 3.3].

6.2.2 Randomized HOOI

Recall from Section 5.3.2 that higher-order orthogonal iteration (HOOI) is a method of computing Tucker decompositions by iteratively solving least-squares problems for each mode's factor matrix, which is typically more accurate than the HOSVD but more computationally expensive. Similar to our goal in CP-ALS, for a given input

Algorithm 9 Randomized Sequentially Truncated Higher-Order SVD (R-STHOSVD)

Require: Input tensor $\mathcal{X} \in \mathbb{F}^{N_1 \times \dots \times N_d}$, target ranks $R_1, \dots, R_d$, oversampling parameter p

Ensure: Tucker decomposition $\hat{\mathcal{X}} = \llbracket \mathcal{G}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

- 1: Initialize $\mathcal{G} = \mathcal{X}$
 - 2: **for** $j = 1, \dots, d$ **do**
 - 3: Draw Gaussian matrix $\Omega^{(j)} \in \mathbb{R}^{N^{(-j)} \times (R_j + p)}$
 - 4: Form sketch $\mathbf{Y}^{(j)} = \mathcal{X}_{(j)} \Omega^{(j)}$
 - 5: $[\hat{\mathbf{U}}, \hat{\Sigma}, \hat{\mathbf{V}}] = \text{svd}(\mathbf{Y}^{(j)}, R_j)$
 - 6: Set $\mathbf{A}^{(j)} = \hat{\mathbf{U}}$
 - 7: Update $\mathcal{G}_{(j)} = \hat{\Sigma} \hat{\mathbf{V}}^\top$
 - 8: **end for**
-

$\mathbf{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, we are seeking a rank- $(R_1, \dots, R_d)$ Tucker decomposition $\widehat{\mathbf{X}} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \dots \times_d \mathbf{A}^{(d)}$ that minimizes

$$\min_{\mathcal{G}, \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)}} \|\mathbf{X} - \mathcal{G} \times_1 \mathbf{A}^{(1)} \dots \times_d \mathbf{A}^{(d)}\|, \quad (53)$$

where $\mathcal{G} \in \mathbb{R}^{R_1 \times \dots \times R_d}$ and $\mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R_j}$. Through an appropriate transformation of the objective function, we can re-write the HOOI procedure in Algorithm 7 as two main steps that are repeated until convergence; cf. [182]:

1. For $j = 1, \dots, d$, solve $\mathbf{A}^{(j)} = \arg \min_{\mathbf{A} \in \mathbb{R}^{N_j \times R_j}} \|\mathbf{Z}^{(j)} \mathbf{G}_{(j)}^\top \mathbf{A}^\top - \mathbf{X}_{(j)}^\top\|^2$, where

$$\mathbf{Z}^{(j)} = \mathbf{A}^{(d)} \otimes \dots \otimes \mathbf{A}^{(j+1)} \otimes \mathbf{A}^{(j-1)} \otimes \dots \otimes \mathbf{A}^{(1)},$$

2. Update $\mathcal{G} = \arg \min_{\mathcal{G} \in \mathbb{R}^{R_1 \times \dots \times R_d}} \|\mathbf{Z} \mathbf{y} - \mathbf{x}\|^2$, where $\mathbf{Z} = \mathbf{A}^{(d)} \otimes \dots \otimes \mathbf{A}^{(1)}$, and $\mathbf{y}, \mathbf{x}$ are (column) vectorizations of $\mathcal{G}, \mathbf{X}$, respectively.

We observe that each of these two steps involve large overdetermined least squares problems, and similar to the strategy in CP-ARLS-LEV, we will exploit the matrix product structures to solve smaller sketched least squares problems efficiently, now with TensorSketch, cf. [182, Algorithm 2 (TUCKER-TS)].

TensorSketch We briefly summarize TensorSketch [211, 222, 14, 83], a specialized version of the CountSketch operator of [52]. A CountSketch operator can be defined as a linear map $\mathbf{S} : \mathbb{R}^I \rightarrow \mathbb{R}^J$ given by $\mathbf{S} = \mathbf{P}\mathbf{D}$, where $\mathbf{P} \in \mathbb{R}^{J \times I}$ is a matrix with $\mathbf{P}(h(i), i) = 1$ and all other entries equal to 0, for a random map $h : [I] \rightarrow [J]$ such that $\mathbb{P}[h(i) = j] = 1/J$ for all $i \in [I], j \in [J]$. The matrix $\mathbf{D} \in \mathbb{R}^{I \times I}$ is a diagonal matrix with each entry equal to ± 1 with equal probability.

In particular, a CountSketch operator $\mathbf{S}$ can be rapidly applied to a matrix $\mathbf{A}$ in $O(\text{nnz}(\mathbf{A}))$ operations, without explicitly forming $\mathbf{S}$ or $\mathbf{A}$. More specifically, if $\mathbf{S} \in \mathbb{R}^{J \times I}$ and $\mathbf{A} \in \mathbb{R}^{I \times L}$, typically with $J \ll I$, then we can form $\mathbf{Y} = \mathbf{S}\mathbf{A}$ by hashing each row i of $\mathbf{A}$ with an integer h sampled uniformly from $[J]$, assigning it a value s of ± 1 with equal probability, then summing the rows with the same hash value (i.e. adding $s\mathbf{A}_{i,:}$ to the h th row of $\mathbf{Y}$ if $\mathbf{A}_{i,:}$ has hash value h).

A TensorSketch operator is a specific type of CountSketch operator, which can be rapidly applied to Kronecker products, e.g., $\mathbf{Z} = \mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(d)} \in \mathbb{R}^{n^d \times k^d}$. (We again assume here that $n = N_1 = \dots = N_d$ and $k = R_1 = \dots = R_d$.) If $\mathbf{T} : \mathbb{R}^{n^d} \rightarrow \mathbb{R}^\ell$ is a TensorSketch operator with embedding dimension $\ell \ll n^d$, then the cost of forming $\mathbf{T}\mathbf{Z}$ is only $O(\ell d n k + \ell k^d)$, suppressing log factors, instead of $O(\ell n^d k^d)$ for naive multiplication. This is accomplished by sketching each factor matrix with independent CountSketch operators, then convolving with Fast Fourier Transforms (FFTs).

To that end, let $h_j : [n] \rightarrow [\ell]$ for $j = 1, \dots, d$ be 3-wise independent hash functions, i.e. for any distinct $i_1, i_2, i_3 \in [n]$, the hash codes $h_j(i_1), h_j(i_2), h_j(i_3)$ are independent random variables, and $h_j(i)$ is uniformly distributed in $[\ell]$ for any fixed $i \in [n]$. Let $s_j : [n] \rightarrow \{+1, -1\}$ be 4-wise independent sign functions (so that analogously, $s_j(i_1), s_j(i_2), s_j(i_3), s_j(i_4)$ are independent random variables with $s_j(i) \sim \text{Uniform}\{\pm 1\}$). We

form independent CountSketch matrices $\mathbf{S}^{(j)} = \mathbf{P}^{(j)} \mathbf{D}^{(j)}$ for $j = 1, \dots, d$, using the hash functions h_j to form $\mathbf{P}^{(j)}$ and $s_j(i) = \mathbf{D}_{i,i}^{(j)}$ for $i \in [n]$. The application of $\mathbf{S}^{(j)}$ to each column $\mathbf{A}_{:,r_j}^{(j)}$, for $r_j = 1, \dots, k$, can be represented by an $(\ell - 1)$ -degree polynomial

$$\mathcal{P}_{r_j}^{(j)}(\omega) = \sum_{i=1}^n s_j(i) \mathbf{A}_{i,r_j}^{(j)} \omega^{h_j(i)-1} =: \sum_{l=1}^{\ell} c_{l,r_j}^{(j)} \omega^{j-1}, \quad (54)$$

and we define $\mathbf{c}_{r_j}^{(j)}$ to be the vector of coefficients $(c_{1,r_j}^{(j)}, \dots, c_{\ell,r_j}^{(j)})$, which are grouped in (54) by hash value.

Let $\mathbf{T}$ be the TensorSketch operator, which is defined as the CountSketch operator formed from hash function H and sign function S given by

$$H : [n]^d \rightarrow \ell, \quad (i_1, \dots, i_d) \mapsto \left(\sum_{j=1}^d (h_j(i_j) - 1) \mod \ell \right) + 1, \quad (55)$$

$$S : [n]^d \rightarrow \{+1, -1\}, \quad (i_1, \dots, i_d) \mapsto \prod_{j=1}^d s_j(i_j). \quad (56)$$

Then the r th column of $\mathbf{T}(\mathbf{A}^{(1)} \otimes \dots \otimes \mathbf{A}^{(d)}) = \mathbf{TZ}$ for $r = 1, \dots, k^d$ can also be represented as a polynomial

$$\mathcal{P}_r(\omega) = \sum_{i=1}^{n^d} S(i_1, \dots, i_d) \mathbf{Z}_{i,r} \omega^{H(i_1, \dots, i_d)}, \quad (57)$$

$$= \sum_{i=1}^{n^d} s_1(i_1) \dots s_d(i_d) \mathbf{A}_{i_1, r_1}^{(1)} \dots \mathbf{A}_{i_d, r_d}^{(d)} \omega^{(h_1(i_1) + \dots + h_d(i_d) - d) \mod \ell} \quad (58)$$

$$= \text{FFT}^{-1} \left(\text{FFT}(\mathbf{c}_{r_1}^{(1)}) * \dots * \text{FFT}(\mathbf{c}_{r_d}^{(d)}) \right), \quad (59)$$

where $*$ denotes the Hadamard product, and i in (57) corresponds to the tuple $(i_1, \dots, i_d) \in [n]^d$, and similarly r corresponds to the tuple $(r_1, \dots, r_d) \in [k]^d$. In other words,

$$\mathbf{TZ} = \text{FFT}^{-1} \left(\left(\bigodot_{j=1}^d \left(\text{FFT}(\mathbf{S}^{(j)} \mathbf{A}^{(j)}) \right)^\top \right)^\top \right), \quad (60)$$

where $\odot$ is the KRP, and the FFT is applied column-wise. Instead of the usual $O(\ell n^d k^d)$ cost of matrix multiplication, the cost of $\mathbf{TZ}$ is $O(\ell d n k + \ell k^d \log(\ell))$ using (60).

Using TensorSketch in HOOI We now turn our attention back to the randomized HOOI algorithm of [182], which uses TensorSketch to reduce the size of each least squares subproblem in the two primary steps of HOOI listed above. The least squares problem to find the factor matrix for each mode $j = 1, \dots, d$ is highly overdetermined, involving a Kronecker product $\mathbf{Z}^{(j)}$ of size $N^{(-j)} \times (\prod_{k \neq j} R_k)$. The least squares problem to find

Algorithm 10 Randomized Higher-order Orthogonal Iteration with TensorSketch (R-HOOI)

Require: Input tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, factor matrix target ranks $R_1, \dots, R_d$, sketching dimensions $\ell_1, \dots, \ell_d, \ell_c$

Ensure: Tucker decomposition $\widehat{\mathcal{X}} = \llbracket \mathcal{G}; \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(d)} \rrbracket$

```

1: Initialize  $\mathbf{A}^{(j)} \in \mathbb{R}^{N_j \times R_j}$ ,  $j = 2, \dots, d$ 
2: Draw TensorSketch matrices  $\mathbf{T}^{(j)} \in \mathbb{R}^{\ell_j \times (\prod_{i \neq j} N_i)}$  for  $i, j \in [d]$  and  $\mathbf{T}^{(c)} \in \mathbb{R}^{\ell_c \times \prod_j N_j}$ 
3: while not converged do
4:   for  $j = 1, \dots, d$  do
5:      $\mathbf{A}^{(j)} = \arg \min_{\mathbf{A} \in \mathbb{R}^{N_j \times R_j}} \left\| \left( \mathbf{T}^{(j)} \otimes_{i=d, i \neq j}^1 \mathbf{A}^{(i)} \right) \mathbf{G}_{(j)}^\top \mathbf{A}^\top - \mathbf{T}^{(j)} \mathbf{X}_{(j)}^\top \right\|_F^2$ 
6:   end for
7:    $\mathbf{g} = \arg \min_{\mathbf{z} \in \mathbb{R}^{\prod_j R_j \times 1}} \left\| \left( \mathbf{T}^{(c)} \otimes_{j=d}^1 \mathbf{A}^{(j)} \right) \mathbf{z} - \mathbf{T}^{(c)} \mathbf{x} \right\|_2^2$  ▷ Column vectorization  $\mathbf{x}$  of  $\mathcal{X}$ 
8:   Re-fold vectorization  $\mathbf{g}$  into  $\mathcal{G} \in \mathbb{R}^{R_1 \times \dots \times R_d}$ 
9: end while

```

the core tensor is also highly overdetermined, involving a Kronecker product of size $\prod_{j=1}^d N_j \times \prod_{j=1}^d R_j$, or $n^d \times k^d$ for convenience. To compute the factor matrices and core tensor, we form independent TensorSketches to solve smaller sketched least squares problems corresponding to both of these steps. Pseudocode for R-HOOI, equivalently [182, TUCKER-TS], is given in Algorithm 10. This algorithm can be implemented as a single-pass algorithm through the input tensor, cf. [182, Algorithm S3].

The dominant cost of Algorithm 10 (single-pass) is $O(dnk^2 + \ell dk^d + \ell_c k^d + k^{2d})$, or roughly $k^{O(d)}$, versus the dominant cost of $O(n^d)$ for deterministic HOOI (Algorithm 7), if we suppress log factors and assume sketching dimension $\ell = \ell_1 = \dots = \ell_d$. We refer the reader to [182, S3.2.6] for details on the complexity analysis. However, we observe that the single-pass R-HOOI algorithm as presented in [182] uses the same TensorSketches for sketching the updated Kronecker products until some convergence criteria is met. In practice, the performance of R-HOOI as written in Algorithm 10 is curiously better in terms of error reduction than if new independent TensorSketch operators were drawn in every iteration, though it is unclear why this is the case. However, for the theoretical guarantees of [83] to hold for TensorSketched least squares, it is technically necessary to form new independent TensorSketch operators in each iteration of R-HOOI. In [182], the numerical results suggest that sketching dimensions $\ell = Ck^{d-1}$ and $\ell_c = Cr^d$ for constant $C > 4$ (e.g., $C = 10$) work well in practice.

6.2.3 Randomized TSMS

We end this section by briefly highlighting the recent work of [126] on Randomized Tucker decompositions with Single-Mode Sketching (RTSMS). The RTSMS algorithm sketches the input tensor one mode at a time, i.e. sketching on the left of the mode- j unfolding vs. sketching on the right, to avoid the computational bottleneck of multiplying along the larger dimension $N^{(-j)} = \prod_{i \neq j} N_i$. The algorithm is also single-pass, so that the input tensor is not revisited after the first sketch, and rank-adaptive if the target ranks for the factor matrices are unknown *a priori*.

To illustrate, suppose we want to compute a rank- (R_1, R_2, R_3) HOSVD that approximates an input tensor $\mathcal{X} \in \mathbb{R}^{N_1 \times N_2 \times N_3}$. (We omit rank-adaptivity and iterative refinement; the reader is referred to [126] for practical implementation details.) We initialize tensors

$$\mathcal{G}^{\text{old}} = \mathcal{X} \text{ and } \mathcal{G}^{\text{new}} = \mathcal{X} \times_1 \mathbf{\Omega}^{(1)} \iff \mathbf{G}_{(1)}^{\text{new}} = \left(\mathbf{\Omega}^{(1)} \right)^\top \mathbf{X}_{(1)} \in \mathbb{R}^{\tilde{R}_1 \times (N_2 N_3)}$$

for some sketching dimension $\tilde{R}_1 > R_1$. There exists a factor matrix $\mathbf{A}^{(1)} \in \mathbb{R}^{N_1 \times \tilde{R}_1}$ such that $\mathcal{G}^{\text{new}} \approx \mathcal{G}^{\text{old}} \times_1 \mathbf{A}^{(1)}$. In practice, this is computed as an approximate solution of the corresponding least squares problem, which is being sub-sampled from the right, cf. [126, Section 4.2]. We then update $\mathcal{G}^{\text{old}} = \mathcal{G}^{\text{new}}$.

For the next mode, we first update $\mathcal{G}^{\text{new}}$ via mode-2 sketching:

$$\mathbf{G}_{(2)}^{\text{new}} = \left(\mathbf{\Omega}^{(2)} \right)^\top \mathbf{G}_{(2)}^{\text{old}} \in \mathbb{R}^{\tilde{R}_2 \times (\tilde{R}_1 N_3)}$$

for some sketching dimension $\tilde{R}_2 > R_2$. To compute $\mathbf{A}^{(2)}$, we solve a smaller sketched least squares problem of size $\tilde{R}_2 \times (\tilde{R}_1 N_3)$, and repeat this for the final mode.

The computational complexity of RTSMS is dominated by the first TTM $\mathcal{X} \times_1 \mathbf{\Omega}_1$. Letting $k = R_1 = \dots = R_d$ and $n = N_1 = \dots = N_d$, the dominant cost is $O(kn^d)$ to compute the first TTM, but the sketch size is only $k \times n$ vs. $k \times n^{d-1}$ for R-HOSVD. The least squares solves are also significantly reduced by sub-sampling and by computing a generalized Nyström decomposition vs. SVD for the factor matrices. The full tensor does not need to be sketched again in modes $j = 2, \dots, d$, and the core tensor is available immediately upon termination, as in the ST-HOSVD algorithm. Additional details can be found in [126].

6.3 Randomized Algorithms for Tensor ID/CUR Decompositions

As we observed in Sect. 5.3.4, the tensor ID/CUR decomposition has been formulated in many ways in the literature, with no one standard representation. However, there exists a natural categorization of tensor ID/CUR decompositions, based on whether the factor matrices or core tensors in the computed decompositions are comprised of entries of the input tensor. In this section, we explore how randomization can be leveraged in prototypical algorithms to compute a Tucker ID/CUR decomposition from each category. We also discuss recent work on randomized algorithms for tensor ID/CUR in the CP format, which can be viewed as a form of input tensor pruning.

6.3.1 Randomized Algorithms for Tucker Core ID/CUR

We first consider randomized algorithms for tensor ID/CUR factorizations in the Tucker format of the first category, where the core tensor in the compressed representation is a sub-tensor of the input tensor. In particular, we focus on the tensor ID/CUR

decompositions that were recently characterized in [39, 40], termed the “Chidori” and “Fiber” tensor CUR factorizations. This work was inspired by numerous investigations of randomized sampling algorithms for tensor CUR decompositions, including [41, 42, 92, 180].

In [40], given an input tensor $\mathbf{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, the rank- $(R_1, \dots, R_d)$ tensor CUR factorization is defined as

$$\widehat{\mathbf{X}} = \mathcal{G} \times_1 (\mathbf{C}_1 \mathbf{U}_1^\dagger) \times_2 (\mathbf{C}_2 \mathbf{U}_2^\dagger) \times \dots \times_d (\mathbf{C}_d \mathbf{U}_d^\dagger), \quad (61)$$

where the rank- $(R_1, \dots, R_d)$ core tensor is given by

$$\mathcal{G} = \mathbf{X}(I_1, \dots, I_d), \quad (62)$$

for index sets $I_j \subset [N_j]$, $j = 1, \dots, d$. The mode- j factor matrix $\mathbf{C}_j \mathbf{U}_j^\dagger$ is defined in terms of the mode- j unfolding:

$$\begin{aligned} \mathbf{C}_j &= \mathbf{X}_{(j)}(:, J_j), \quad \text{and} \\ \mathbf{U}_j &= \mathbf{C}_j(I_j, :), \end{aligned} \quad (63)$$

where $J_j \subset [N^{(-j)}]$. If indices J_j are computed independently from indices I_j , the decomposition in (61) is called a “Fiber” CUR decomposition in [40]. If $J_j = \otimes_{k \neq j} I_k$, then (61) is called a “Chidori” CUR decomposition. By [40, Theorem 3.3], [122, Corollary 5.2], if the indices I_j and J_j are each sampled uniformly with $|I_j| = O(R_j \log N_j)$ and $|J_j| = O(R_j \log N^{(-j)})$, then the computed Tucker decomposition is a rank- $(R_1, \dots, R_d)$ approximation with high probability. We present pseudocode for either the Chidori or Fiber tensor CUR decomposition in Algorithm 11.

With uniform sampling, the computational complexity of computing either tensor CUR decomposition is dominated by the cost of the pseudoinverse of $\mathbf{U}_j$, which is $O(k^{d+1} \log^d(n))$ for Chidori and $O(k^2 \log^2(n))$ for Fiber, letting $n = N_1 = \dots = N_d$ and $k = R_1 = \dots = R_d$ as before. Recall that the HOSVD has complexity $O(kn^d)$ by comparison.

The recent work of [287] proposes a similar factorization to (61), called the “CoreID.” The CoreID algorithm is reminiscent of the sequentially-truncated HOSVD, specifically the “structure-preserving” ST-HOSVD developed for sparse input tensors in [196]. Namely, in the CoreID, as each mode’s factor matrix is computed, the input tensor is compressed along that mode before proceeding to the next one, so that each subsequent unfolding is reduced in size.

Through matricization, computing the CoreID is reduced to computing a randomized matrix row ID in each mode, cf. Algorithm 4. For $j = 1, \dots, d$, the mode- j unfolding of the tensor is sketched by a random matrix $\mathbf{Y}_j = \mathbf{X}_{(j)} \mathbf{\Omega}_j$. Then a row ID of $\mathbf{Y}_j$ is computed by some skeleton selection method, e.g., greedy pivoting or squared-norm sampling. The selected skeletons I_j form the core sub-tensor, and the mode- j factor matrix for the CoreID is defined as the associated interpolation matrix. The tensor is then compressed along mode j as in ST-HOSVD before proceeding to the next mode. Sketching is the dominating cost in this algorithm, which is why many of the numerical

Algorithm 11 Randomized Tucker ID/CUR with interpolating core

Require: $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, sample sizes $\{S_j\}_{j=1}^d$, probability distributions $\mathbf{p}_j$ on $[N_j]$ for $j = 1, \dots, d$, sample sizes $\{T_j\}_{j=1}^d$, probability distributions $\mathbf{q}_j$ on $[N^{(-j)}]$ for $j = 1, \dots, d$

Ensure: chidori or fiber decomposition $\widehat{\mathcal{X}} = \mathcal{G} \times_{j=1}^d (\mathbf{C}_j \mathbf{U}_j^\dagger)$

```

1: for  $j = 1, \dots, d$  do
2:   if chidori then
3:     Sample indices  $I_j$  from  $[N_j]$ ,  $|I_j| = S_j$  without replacement according to  $\mathbf{p}_j$ 
4:     Set  $J_j = \otimes_{i \neq j} I_i$ 
5:   end if
6:   if fiber then
7:     Sample indices  $I_j$  from  $[N_j]$ ,  $|I_j| = S_j$  without replacement according to  $\mathbf{p}_j$ 
8:     Sample indices  $J_j$  from  $[N^{(-j)}]$ ,  $|J_j| = T_j$  without replacement according to  $\mathbf{q}_j$ 
9:   end if
10:  Form  $\mathbf{C}_j = \mathbf{X}_{(j)}(:, J_j)$ 
11:  Form  $\mathbf{U}_j = \mathbf{C}_j(I_j, :)$ 
12: end for
13:  $\mathcal{G} = \mathcal{X}(I_1, \dots, I_d)$ 

```

investigations in [287] consider structured input tensors for fast sketching, e.g., a sparse input tensor or a CP-factorized input tensor.

6.3.2 Randomized Algorithms for Tucker Factor ID/CUR

We now turn our attention to randomized algorithms for tensor ID/CUR factorizations in the Tucker format in which the factor matrices contain entries of the input tensor. We illustrate such factorizations with the “higher-order ID” of [235], which we contrast with the “SatID” of [287].

For a given $\mathcal{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, the higher-order ID (HOID) of [235] is given by

$$\widehat{\mathcal{X}} = \mathcal{G} \times_1 \mathbf{C}_1 \times_2 \mathbf{C}_2 \times \dots \times_d \mathbf{C}_d, \quad (64)$$

where the rank- $(R_1, \dots, R_d)$ core tensor $\mathcal{G}$ is defined as

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{C}_1^\dagger \times_2 \mathbf{C}_2^\dagger \times \dots \times_d \mathbf{C}_d^\dagger, \quad (65)$$

and each factor matrix $\mathbf{C}_j \in \mathbb{R}^{N_j \times R_j}$ consists of column skeletons of the mode- j unfolding $\mathbf{X}_{(j)}$. The core tensor in (65) is optimal for the factorization in (64), cf. [73]. For the theoretical guarantees of [235], the strong rank-revealing QR algorithm of [116] is used for skeleton selection on a random sketch $\mathbf{Y}_j = \mathbf{\Omega}_j \mathbf{X}_{(j)}$, where $\mathbf{\Omega}_j$ is an $(R_j + p) \times N_j$ random matrix for some small oversampling parameter p . However, in practice, CPQR or LUPP in the randomized matrix ID (e.g., Algorithm 4, with input $\mathbf{X}_{(j)}^*$) performs similarly and is less computationally expensive. Pseudocode is provided in Algorithm 12.

For each mode j , the cost of sketching is $O(R_j N_j N^{(-j)}) = O(R_j N)$. Skeleton selection as in [235] costs $O(R_j^2 N^{(-j)})$. The cost of computing the core tensor is

Algorithm 12 Randomized Tucker ID/CUR with interpolating factors

Require: $\mathbf{X} \in \mathbb{R}^{N_1 \times \dots \times N_d}$, Tucker ranks $(R_1, \dots, R_d)$

Ensure: Tensor ID $\widehat{\mathbf{X}} = \mathcal{G} \times_{j=1}^d \mathbf{C}_j$

- 1: **for** $j = 1, \dots, d$ **do**
 - 2: Compute rank- R_j randomized column ID $\mathbf{X}_{(j)} \approx \mathbf{C}_j \mathbf{W}_j$ ▷ e.g., Algorithm 4 on $\mathbf{X}_{(j)}^*$
 - 3: **end for**
 - 4: Compute core $\mathcal{G} = \mathbf{X} \times_{j=1}^d \mathbf{C}_j^\dagger$
-

$O\left(\sum_{j=1}^d \left[R_j^2 N_j + \prod_{i \leq j} R_i \cdot N^{(-j)}\right]\right)$. Letting $k = R_1 = \dots = R_d$ and $n = N_1 = \dots = N_d$ again, the total cost is roughly $O(dkn^d + dk^{d-1}n^{d-1})$. A sequentially-truncated version of the HOID (ST-HOID), analogous to the ST-HOSVD, is given in [235, Algorithm 4].

A very similar decomposition is developed in [287], termed the ‘‘SatID,’’ which can be more efficiently computed. In particular, for an n^d -sized tensor, a QR-based algorithm to compute skeletons for the mode- j factor matrix as in [235] requires n^{d-1} column norms, which is not optimal for structured tensors and can even have larger storage costs than the input tensor. By contrast, the SatID algorithm is based on a randomized matrix ID computed by marginalized norm sampling, which can be viewed as an extension of the original tensor CUR construction of [92], which uses norm sampling.

We briefly summarize the marginalized norm sampling procedure of [287]. As in Sect. 6.1, for a fixed mode j , we can uniquely identify a column index $i \in [N^{(-j)}]$ of $\mathbf{X}_{(j)}$ with a $(d-1)$ -tuple $(i_1, \dots, i_{j-1}, i_{j+1}, \dots, i_d)$. Similar to CP-ARLS-LEV in [168], the idea is to accelerate sampling of $i \in [N^{(-j)}]$, now based on column norm sampling of $\mathbf{X}_{(j)}$.

Suppose we have already selected some subset of column indices S of $\mathbf{X} = \mathbf{X}_{(j)}$ to determine the mode- j column skeletons comprising $\mathbf{C}_j$, and we want to select the next column. Norm sampling (cf. Sect. 3.2) requires the computation of column scores

$$d_i^{(S)} = \|\mathbf{X}_{:,i} - \mathbf{Q}_S \mathbf{Q}_S^\top \mathbf{X}_{:,i}\|^2 = \min_{\mathbf{v}} \|\mathbf{X}_{:,S} \mathbf{v} - \mathbf{X}_{:,i}\|^2, \quad i \in [N^{(-j)}], \quad (66)$$

where $\mathbf{Q}_S$ is an orthonormal basis for the range of $\mathbf{X}_{:,S}$. If $X = (X_1, \dots, X_{j-1}, X_{j+1}, \dots, X_d)$ is a random variable representing the next column index, then using (66),

$$\mathbb{P}(X = i) \propto d_i^{(S)}.$$

The key observation is that

$$\mathbb{P}(X_1 = i_1) \propto \sum_{i_2, \dots, i_d} d_{i_1, i_2, \dots, i_d}^{(S)},$$

so that we can sample i_1 from its marginal distribution, then sample i_2 given i_1 , and continue for all of $i = (i_1, i_2, \dots, i_{j-1}, i_{j+1}, \dots, i_d)$. This sum can be computed efficiently via the equation derived in [287, Eq. 18]:

$$\mathbb{P}(X_1 = i_1) \propto \left\| \tilde{\mathbf{X}}_{:,i_1} \right\|^2, \quad (67)$$

where $\tilde{\mathbf{X}} = (\mathbf{X} \times_j \mathbf{Q}_{Sc})_{(1)}^\top$ with $\mathbf{Q}_{Sc}$ an orthonormal basis for the range of $\mathbf{X}_{:, [N^{(-j)}] \setminus S}$. In other words, to sample i_1 , we need only compute the N_1 column norms of $\tilde{\mathbf{X}}$, which can be approximated efficiently using randomized sketching, cf. [287, Algorithm 6]. Once i_1 has been sampled, this process is repeated on the sliced tensor $\mathbf{X}(i_1, :, \dots, :)$.

For structured inputs, such as sparse tensors, this procedure can be modified for better performance. The total complexity (including randomized sketching) is $O(\text{nnz}(\mathbf{X}) + \ell n^2 k^2)$ for a rank- $(k, \dots, k)$ SatID of an n^d -sized tensor with sketch size ℓ ; see [287, Algorithm 7] for details.

6.3.3 Randomized Algorithms for CP Factor ID/CUR

We now consider tensor ID/CUR decompositions in the CP format, which necessarily fall into the category of decompositions in which the factor matrices contain entries of the input tensor. These decompositions can be interpreted as the result of pruning the input tensor, leaving only its most important rank-one components.

The seminal work of [29] introduces a method to obtain rank- k CP-tensor IDs from rank- R CP-factorized tensors, given by

$$\mathbf{X} = \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(d)} \in \mathbb{R}^{N_1 \times \dots \times N_d}. \quad (68)$$

To better illustrate the algorithm, we express (68) in a vectorized form, where $\mathbf{X}_r \in \mathbb{R}^{N \times 1}$ is a vectorization of the rank-one component $\mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(d)}$. Then we can consider the matrix

$$\mathbf{X} = [\mathbf{X}_1 \mid \dots \mid \mathbf{X}_R] \in \mathbb{R}^{N \times R}, \quad (69)$$

and within this framework, we are seeking a rank- k matrix column ID of $\mathbf{X}$, corresponding to a rank- k CP decomposition $\hat{\mathbf{X}}$ denoted by

$$\hat{\mathbf{X}} = \sum_{r=1}^k \mathbf{X}_{s_r}, \quad (70)$$

for k column skeletons of (69) indexed by $\{s_1, \dots, s_k\} \subset [R]$. In particular, we want to select these k indices without explicitly forming the matrix $\mathbf{X}$.

To do this, the randomized CP compression algorithm of [29] uses rank-one random tensors $\mathbf{r}_r^{(1)} \circ \dots \circ \mathbf{r}_r^{(d)}$, with vectorized form $\mathbf{R}_r$ for $r = 1, \dots, \ell$ and $\ell = k + p$ for some small oversampling parameter p . An $\ell \times R$ random sample matrix is formed via

$$\mathbf{Y} = \begin{bmatrix} \langle \mathbf{R}_1, \mathbf{X}_1 \rangle & \langle \mathbf{R}_1, \mathbf{X}_2 \rangle & \cdots & \langle \mathbf{R}_1, \mathbf{X}_R \rangle \\ \vdots & \vdots & & \vdots \\ \langle \mathbf{R}_\ell, \mathbf{X}_1 \rangle & \langle \mathbf{R}_\ell, \mathbf{X}_2 \rangle & \cdots & \langle \mathbf{R}_\ell, \mathbf{X}_R \rangle \end{bmatrix} =: \mathbf{R}\mathbf{X}, \quad (71)$$

whose rank- k column ID is then computed to obtain skeletons $\{s_1, \dots, s_k\}$ as in (70). We note that the matrix $\mathbf{Y}$ can be computed efficiently without ever explicitly forming $\mathbf{R}$ or $\mathbf{X}$, since each $\mathbf{Y}(l, r) = \prod_{j=1}^d \langle \mathbf{r}_l^{(j)}, \mathbf{a}_r^{(j)} \rangle$. However, we note that the elements of $\mathbf{R}$ are not independent, and so theoretical guarantees on the randomized matrix decomposition computed with $\mathbf{Y}$ are not given, though it is demonstrated in [29] that it performs well in practice. Pseudocode is provided in Algorithm 13.

Algorithm 13 Randomized CP ID/CUR with interpolating factors

Require: CP tensor $\mathcal{X} = \sum_{r=1}^R \mathbf{a}_r^{(1)} \circ \cdots \circ \mathbf{a}_r^{(d)}$, target rank k , oversampling p

Ensure: Rank- k CP tensor $\hat{\mathcal{X}} = \sum_{i=1}^k \mathbf{a}_{s_i}^{(1)} \circ \cdots \circ \mathbf{a}_{s_i}^{(d)}$

1: Draw $\ell = k + p$ random rank-one tensors $\mathbf{r}_i^{(1)} \circ \cdots \circ \mathbf{r}_i^{(d)}$

2: Form $\mathbf{Y} \in \mathbb{R}^{\ell \times R}$ via $\mathbf{Y}(\ell, r) = \prod_{j=1}^d \langle \mathbf{r}_\ell^{(j)}, \mathbf{a}_r^{(j)} \rangle$

3: Compute rank- k column ID of $\mathbf{Y}$ for skeletons $\{s_1, \dots, s_k\}$ $\triangleright$ e.g., Algorithm 4 applied to $\mathbf{Y}^*$

To analyze the cost of this algorithm, let $n = N_1 = \cdots = N_d$. Forming the ℓ random tensors costs $O(t_R \ell dn)$, where t_R is the cost of generating a single random number. The cost of computing each entry of $\mathbf{Y}$ is $O(dn)$, thus the total cost to compute $\mathbf{Y}$ is $O(\ell R \cdot dn)$. The rank- k matrix ID of $\mathbf{Y}$ comes at a cost of $O(k\ell R)$, and forming the rank- k tensor ID from the rank-one terms of $\mathcal{X}$ requires $O(dkn)$ additional operations. In particular, the method of [29] is faster than a rank- k ALS procedure by a factor of $k \cdot n_{iter}$, where n_{iter} is the number of iterations of ALS.

The recent work of [183] constructs an identical rank- k CP approximation to a rank- R CP-factorized tensor $\mathcal{X}$, but uses the TensorSketch operator instead of rank-one random tensors. The complexity of the TensorSketch-based algorithm of [183] is dominated by the sketching step, which costs $O(d(nR + R\ell \log \ell))$; the ID of the $\ell \times R$ matrix $\mathbf{Y}$ costs the same as above. In addition to its theoretical performance guarantees, the algorithm's practical performance is demonstrated through numerical experiments, in which the TensorSketch-based method exhibits a runtime speed-up of roughly one order of magnitude over other randomized CP-tensor ID algorithms, including the random projection method of [29].

7 Concluding Remarks

We end our survey with an outlook on the future of RNLA, which will continue to be an indispensable tool for scientific computing and data science in the face of increasingly large problems. We have seen throughout this work that randomization is not only useful for dimension reduction, but also admits algorithmic implementations that can be optimized for specific hardware or problem constraints, such as parallelization for

GPU computing or factorizing streaming data. Randomization is especially critical for the development of scalable tensor algorithms.

Future work may touch on several different aspects of computing fast and accurate matrix and tensor decompositions with randomized algorithms. Probabilistic guarantees are still needed for the resulting error of many randomized algorithms that are used in practice, as well as tighter bounds that more closely align with observed behavior. For example, it is generally an open question as to what the weakest assumptions are for a particular type of randomized DRM to achieve a desired approximation accuracy. The question can be further refined by the specific problem setting, e.g., the weakest assumptions to obtain a certain accuracy of rank-structured matrix decomposition or sketched least squares solutions.

As stronger theoretical foundations are developed for new and existing randomized algorithms, high-performance software is also needed to fully leverage available computing resources. For example, there remains a need for standardized RNLA kernels and interfaces that can offer both practical speed and front-end accessibility, e.g., in Python or Julia. RNLA software libraries like RandBLAS or RandLAPACK are continually being updated with new functionality, such as mixed-precision arithmetic or sparse linear algebra packages. Additionally, there is an ongoing effort to develop high-performance implementations of randomized tensor algorithms that take advantage of GPU acceleration. This includes the growing body of work on tensor decompositions other than those presented here, some of which can break the curse of dimensionality.

In summary, RNLA has become an established area of applied mathematics that will prove increasingly important as larger problems are tackled and the size and complexity of scientific data grow. As such, there remain many interesting open questions to investigate, which build on the fundamental concepts surveyed in this work.

References

- [1] D. Achlioptas, A. Fiat, A. R. Karlin, and F. McSherry. Web search via hub synthesis. In Proceedings of the 42nd Annual IEEE Symposium on Foundations of Computer Science (FOCS), pages 500–509, Las Vegas, Nevada, USA, 2001. IEEE Computer Society.
- [2] D. Achlioptas and F. McSherry. Fast computation of low rank matrix approximations. In Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing, STOC '01, page 611–618, New York, NY, USA, 2001. Association for Computing Machinery.
- [3] K. S. Aggour, A. Gittens, and B. Yener. Adaptive sketching for fast and convergent canonical polyadic decomposition. In H. D. III and A. Singh, editors, Proceedings of the 37th International Conference on Machine Learning, volume 119 of Proceedings of Machine Learning Research, pages 82–92. PMLR, 2020.
- [4] T. D. Ahle, M. Kapralov, J. B. Knudsen, R. Pagh, A. Velingker, D. P. Woodruff, and A. Zandieh. Oblivious sketching of high-degree polynomial kernels. In Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 141–160. SIAM, 2020.
- [5] S. Ahmadi-Asl, S. Abukhovich, M. G. Asante-Mensah, A. Cichocki, A. H. Phan, T. Tanaka, and I. V. Oseledets. Randomized Algorithms for Computation of Tucker Decomposition and Higher Order SVD (HOSVD). IEEE Access, 9:28684–28706, 2021.
- [6] A. Aidini, G. Tsagkatakis, N. D. Sidiropoulos, and P. Tsakalides. Few-shot classification using tensor completion. In Proceedings of the 57th Annual Asilomar Conference on Signals, Systems, and Computers, pages 1283–1287. IEEE, 2023.
- [7] A. E. Alaoui and M. W. Mahoney. Fast randomized kernel ridge regression with statistical guarantees. In Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1, NIPS'15, page 775–783, Cambridge, MA, USA, 2015. MIT Press.
- [8] J. Altschuler, A. Bhaskara, G. Fu, V. Mirrokni, A. Rostamizadeh, and M. Zadimoghaddam. Greedy column subset selection: New bounds and distributed algorithms. In Proceedings of the 33rd International Conference on Machine

- Learning, volume 48 of Proceedings of Machine Learning Research, pages 2539–2548. PMLR, 2016.
- [9] N. Amsel, T. Chen, A. Greenbaum, C. Musco, and C. Musco. Nearly optimal approximation of matrix functions by the lanczos method. In Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.
 - [10] A. Anandkumar, R. Ge, D. Hsu, S. M. Kakade, and M. Telgarsky. Tensor decompositions for learning latent variable models. Journal of Machine Learning Research, 15(1):2773–2832, 2014.
 - [11] C. A. Andersson and R. Bro. Improving the speed of multi-way algorithms: Part i. Tucker3. Chemometrics and Intelligent Laboratory Systems, 42(1-2):93–103, 1998.
 - [12] H. Avron, A. M. Druinsky, and A. Gupta. Revisiting asynchronous linear solvers: Provable convergence rate through randomization. Journal of the ACM (JACM), 62(6):51:1–51:28, 2015.
 - [13] H. Avron, P. Maymounkov, and S. Toledo. Blendenpik: Supercharging lapack’s least-squares solver. SIAM Journal on Scientific Computing, 32(3):1217–1236, 2010.
 - [14] H. Avron, H. L. Nguyen, and D. P. Woodruff. Subspace embeddings for the polynomial kernel. In Advances in Neural Information Processing Systems 27, volume 2, pages 2258–2266, Cambridge, MA, USA, 2014. MIT Press.
 - [15] H. Avron and S. Toledo. Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix. Journal of the ACM, 58(2):1–34, 2011.
 - [16] M. Baboulin, X. S. Li, and F. Rouet. Using random butterfly transformations to avoid pivoting in sparse direct methods. In M. J. Dayd’*e*, O. Marques, and K. Nakajima, editors, High Performance Computing for Computational Science – VECPAR 2014, volume 8969 of Lecture Notes in Computer Science, pages 135–144. Springer, Cham, 2015.
 - [17] M. Bachmayr, R. Schneider, and A. Uschmajew. Tensor networks and hierarchical tensors for the solution of high-dimensional partial differential equations. Foundations of Computational Mathematics, 16(6):1423–1472, 2016.
 - [18] O. Balabanov and L. Grigori. Randomized gram–schmidt process with application to gmres. SIAM Journal on Scientific Computing, 44(3):A1450–A1474, 2022.
 - [19] O. Balabanov and L. Grigori. Randomized block gram–schmidt process for the solution of linear systems and eigenvalue problems. SIAM Journal on Scientific Computing, 47(1):A553–A585, 2025.
 - [20] J. Ballani, L. Grasedyck, and M. Kluge. Black box approximation of tensors in hierarchical tucker format. Linear Algebra and its Applications, 438(2):639–657, 2013.
 - [21] G. Ballard and T. G. Kolda. Tensor Decompositions for Data Science. Cambridge University Press, 2025.
 - [22] K. Batselier, W. Yu, L. Daniel, and N. Wong. Computing low-rank approximations of large-scale matrices with the tensor network randomized svd. SIAM Journal on Matrix Analysis and Applications, 39(3):1221–1244, 2018.

- [23] C. Battaglino, G. Ballard, and T. G. Kolda. A Practical Randomized CP Tensor Decomposition. SIAM Journal on Matrix Analysis and Applications, 39(2):876–901, 2018.
- [24] R. E. Bellman. Adaptive Control Processes: A Guided Tour. Princeton University Press, Princeton, NJ, 1961.
- [25] T. Bendory, Y. Khoo, J. Kileel, O. Mickelin, and A. Singer. Autocorrelation analysis for cryo-EM with sparsity constraints: Improved sample complexity and projection-based algorithms. Proceedings of the National Academy of Sciences, 120(18):e2216507120, 2023.
- [26] J. M. F. T. Berge and N. D. Sidiropoulos. On uniqueness in candecomp/parafac. Psychometrika, 67(3):399–409, 2002.
- [27] V. Bharadwaj, O. A. Malik, R. Murray, A. Buluç, and J. Demmel. Distributed-Memory Randomized Algorithms for Sparse Tensor CP Decomposition. In Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA). ACM, 2024.
- [28] R. Bhattacharjee, G. Dexter, P. Drineas, C. Musco, and A. Ray. Sublinear Time Eigenvalue Approximation via Random Sampling. In K. Etessami, U. Feige, and G. Puppis, editors, 50th International Colloquium on Automata, Languages, and Programming (ICALP 2023), volume 261 of Leibniz International Proceedings in Informatics (LIPIcs), pages 21:1–21:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [29] D. J. Biagioni, D. Beylkin, and G. Beylkin. Randomized interpolative decomposition of separated representations. Journal of Computational Physics, 281:116–134, 2015.
- [30] F. A. Bischoff, R. J. Harrison, and E. F. Valeev. Computing many-body wave functions with guaranteed precision: The first-order møller-plesset wave function for the ground state of helium atom. The Journal of Chemical Physics, 137(10):104103, 2012.
- [31] L. S. Blackford, J. Demmel, J. Dongarra, I. Duff, S. Hammarling, G. Henry, M. Heroux, L. Kaufman, A. Lumsdaine, A. Petitet, R. Pozo, K. Remington, and R. C. Whaley. An updated set of basic linear algebra subprograms (BLAS). ACM Transactions on Mathematical Software, 28(2):135–151, June 2002.
- [32] A. M. P. Boelens, D. Venturi, and D. M. Tartakovsky. Parallel tensor methods for high-dimensional linear pdes. Journal of Computational Physics, 375:519–539, 2018.
- [33] V. Braverman, A. Krishnan, and C. Musco. Sublinear time spectral density estimation. In Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2022, page 1144–1157, New York, NY, USA, 2022. Association for Computing Machinery.
- [34] C. Brezinski, G. Meurant, and M. Redivo-Zaglia. A Journey through the History of Numerical Linear Algebra. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2022.
- [35] S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. Computer Networks and ISDN Systems, 30(1–7):107–117, 1998.

- [36] R. Bro and C. A. Andersson. Improving the speed of multiway algorithms: Part ii: Compression. Chemometrics and Intelligent Laboratory Systems, 42(1-2):105–113, 1998.
- [37] A. Bucci and L. Robol. A multilinear nystrom algorithm for low-rank approximation of tensors in tucker format. SIAM Journal on Matrix Analysis and Applications, 45(4):1929–1953, 2024.
- [38] L. Burke and S. Güttel. Krylov subspace recycling with randomized sketching for matrix functions. SIAM Journal on Matrix Analysis and Applications, 45(4):2243–2262, 2024.
- [39] H. Cai, Z. Chao, L. Huang, and D. Needell. Robust Tensor CUR Decompositions: Rapid Low-Tucker-Rank Tensor Recovery with Sparse Corruptions. SIAM Journal on Imaging Sciences, 17(1):225–247, 2024.
- [40] H. Cai, K. Hamm, L. Huang, and D. Needell. Mode-wise Tensor Decompositions: Multi-dimensional Generalizations of CUR Decompositions. Journal of Machine Learning Research, 22(185):1–36, 2021.
- [41] C. F. Caiafa and A. Cichocki. Methods for factorization and approximation of tensors by partial fiber sampling. In 2009 3rd IEEE International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP), pages 73–76, 2009.
- [42] C. F. Caiafa and A. Cichocki. Generalizing the column-row matrix decomposition to multi-way arrays. Linear Algebra and its Applications, 433(3):557–573, 2010.
- [43] D. Calandriello, M. Dereziński, and M. Valko. Sampling from a k -dpp without looking at all items. In Advances in Neural Information Processing Systems, volume 33, pages 18688–18698, 2020.
- [44] C. Camaño, E. Epperly, R. Meyer, and J. Tropp. Faster linear algebra algorithms with structured random matrices. arXiv preprint:arXiv.2508.21189, 08 2025.
- [45] J. D. Carroll and J. J. Chang. Analysis of individual differences in multidimensional scaling via an N-way generalization of “Eckart-Young” decomposition. Psychometrika, 35(3):283–319, 1970.
- [46] J. D. Carroll, S. Pruzansky, and J. B. Kruskal. CANDELINC: A general approach to multidimensional analysis of many-way arrays with linear constraints on parameters. Psychometrika, 45(1):3–24, 1980.
- [47] A. Castillo, J. Haddock, I. Hartsock, P. Hoyos, L. Kassab, A. Kryshchenko, K. Larripa, D. Needell, S. Suryanarayanan, and K. Yacoubou-Djima. Randomized Kaczmarz Methods for t-Product Tensor Linear Systems with Factorized Operators. BIT Numerical Mathematics, 65:38, 2025.
- [48] S. Chakrabarti, B. Dom, D. Gibson, J. M. Kleinberg, S. R. Kumar, P. Raghavan, S. Rajagopalan, and A. Tomkins. Hypersearching the web. Scientific American, 280(6):54–60, June 1999.
- [49] T. F. Chan. On the existence and computation of LU-factorizations with small pivots. Mathematics of Computation, 42:535–547, 1984.
- [50] T. F. Chan. Rank-Revealing QR Factorizations. Linear Algebra and its Applications, 88/89:67–82, 1987.
- [51] S. Chandrasekaran and I. C. F. Ipsen. On Rank-Revealing Factorisations. SIAM Journal on Matrix Analysis and Applications, 15(2):592–622, 1994.

- [52] M. Charikar, K. Chen, and M. Farach-Colton. Finding frequent items in data streams. Theor. Comput. Sci., 312(1):3–15, Jan. 2004.
- [53] M. Che and Y. Wei. Randomized algorithms for the approximations of Tucker and the tensor train decompositions. Advances in Computational Mathematics, 45:395–428, 2019.
- [54] M. Che, Y. Wei, and H. Yan. The computation of low multilinear rank approximations of tensors via power scheme and random projection. SIAM J. Matrix Anal. Appl., 41(2):605–636, Jan. 2020.
- [55] M. Che, Y. Wei, and H. Yan. Randomized algorithms for the low multilinear rank approximations of tensors. Journal of Computational and Applied Mathematics, 390:113380, 2021.
- [56] M. Che, Y. Wei, and H. Yan. Randomized algorithms for computing the tensor train approximation and their applications. arXiv preprint arXiv:2405.07147, 2024.
- [57] M. Che, Y. Wei, and H. Yan. Sketch-based multiplicative updating algorithms for symmetric nonnegative tensor factorizations with applications to face image clustering. Journal of Global Optimization, 489(4):995–1032, 2024.
- [58] M. Che, Y. Wei, and H. Yan. Efficient algorithms for tucker decomposition via approximate matrix multiplication. Advances in Computational Mathematics, 51(3), 2025.
- [59] H. Chen, F. Ahmad, S. Vorobyov, and F. Porikli. Tensor decompositions in wireless communications and mimo radar. IEEE Journal of Selected Topics in Signal Processing, 15(3):438–453, 2021.
- [60] T. Chen, A. Greenbaum, C. Musco, and C. Musco. Low-memory krylov subspace methods for optimal rational matrix function approximation. SIAM Journal on Matrix Analysis and Applications, 44(2):670–692, 2023.
- [61] T. Chen, P. Niroula, A. Ray, P. Subrahmanya, M. Pistoia, and N. Kumar. Gpu-parallelizable randomized sketch-and-precondition for linear regression using sparse sign sketches. arXiv preprint:arXiv 2506.03070, 2025.
- [62] Y. Chen, E. N. Epperly, J. A. Tropp, and R. J. Webber. Randomly pivoted Cholesky: Practical approximation of a kernel matrix with few entry evaluations. Communications on Pure and Applied Mathematics, 78(5):995–1041, 2025.
- [63] Y. Chen and Y. Yang. Fast Statistical Leverage Score Approximation in Kernel Ridge Regression. In Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS) 2021, PMLR: Volume 130, 2021.
- [64] S. Chenakkod, M. Dereziński, X. Dong, and M. Rudelson. Optimal embedding dimension for sparse subspace embeddings. In Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC), pages 1106–1117, New York, NY, USA, 2024. ACM.
- [65] D. Cheng, R. Peng, I. Perros, and Y. Liu. Spals: Fast alternating least squares via implicit leverage scores sampling. In Advances in Neural Information Processing Systems (NeurIPS), 2016.
- [66] H. Cheng, Z. Gimbutas, P. G. Martinsson, and V. Rokhlin. On the compression of low rank matrices. SIAM Journal on Scientific Computing, 26(4):1389–1404, 2005.

- [67] A. Cichocki, D. Mandic, L. De Lathauwer, G. Zhou, Q. Zhao, C. Caiafa, and H. A. Phan. Tensor decompositions for signal processing applications: From two-way to multiway component analysis. IEEE Signal Processing Magazine, 32(2):145–163, 2015.
- [68] M. B. Cohen. Nearly tight oblivious subspace embeddings by trace inequalities. In Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 278–287, Philadelphia, PA, USA, 2016. SIAM.
- [69] P. Comon. Tensor decompositions: State of the art and applications. In G. McWhirter, J. and K. Proudlar, I. editors, Mathematics in Signal Processing V, pages 1–24. Oxford University Press, Oxford, UK, 2001.
- [70] A. Cortinovis, D. Kressner, and Y. Nakatsukasa. Speeding up krylov subspace methods for computing $f(a)b$ via randomization. SIAM Journal on Matrix Analysis and Applications, 45(1):619–633, 2024.
- [71] W. F. de la Vega, M. Karpinski, R. Kannan, and S. Vempala. Tensor decomposition and approximation schemes for constraint satisfaction problems. In Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing, STOC ’05, page 747–754, New York, NY, USA, 2005. Association for Computing Machinery.
- [72] L. De Lathauwer, B. De Moor, and J. Vandewalle. A Multilinear Singular Value Decomposition. SIAM Journal on Matrix Analysis and Applications, 21(4):1253–1278, 2000.
- [73] L. De Lathauwer, B. D. Moor, and J. Vandewalle. On the Best Rank-1 and Rank- $(r_1, r_2, \dots, r_N)$ Approximation of Higher-Order Tensors. SIAM Journal on Matrix Analysis and Applications, 21(4):1324–1342, 2000.
- [74] A. Dektor, A. Rodgers, and D. Venturi. Rank-adaptive tensor methods for high-dimensional nonlinear pdes. Journal of Scientific Computing, 88(36), 2021.
- [75] M. Dereziński, B. Bartan, M. Pilanci, and M. W. Mahoney. Debiasing distributed second order optimization with surrogate sketching and scaled regularization. Advances in Neural Information Processing Systems, 33:6684–6695, 2020.
- [76] M. Dereziński, D. Calandriello, and M. Valko. Exact sampling of determinantal point processes with sublinear time preprocessing. In Advances in Neural Information Processing Systems, volume 32, pages 11546–11558, 2019.
- [77] M. Dereziński, R. Khanna, and M. W. Mahoney. Improved guarantees and a multiple-descent curve for column subset selection and the nystrom method. In Advances in Neural Information Processing Systems, volume 33, pages 4953–4964, 2020.
- [78] M. Dereziński and M. W. Mahoney. Recent and upcoming developments in randomized numerical linear algebra for machine learning. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD ’24, page 6470–6479, New York, NY, USA, 2024. Association for Computing Machinery.
- [79] M. Dereziński and M. W. Mahoney. Determinantal Point Processes in Randomized Numerical Linear Algebra. Notices of the American Mathematical Society, 68(1), Jan 2021.

- [80] A. Deshpande and L. Rademacher. Efficient volume sampling for row/column subset selection. In Proceedings of the 42nd Annual ACM Symposium on Theory of Computing (STOC). ACM, 2010.
- [81] A. Deshpande, L. Rademacher, S. S. Vempala, and G. Wang. Matrix approximation and projective clustering via volume sampling. In Proceedings of the 17th Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), pages 1117–1126, 2006.
- [82] A. Deshpande and S. S. Vempala. Adaptive sampling and fast low-rank matrix approximation. Electron. Colloquium Comput. Complex., TR06, 2006.
- [83] H. Diao, Z. Song, W. Sun, and D. P. Woodruff. Sketching for kronecker product regression and p-splines. In A. Storkey and F. Perez-Cruz, editors, Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics (AISTATS), volume 84 of Proceedings of Machine Learning Research, pages 1299–1308. PMLR, April 2018.
- [84] W. Dong, G. Yu, L. Qi, and J. Cai. Practical sketching algorithms for low-rank tucker approximation of large tensors. Journal of Scientific Computing, 95(52), 2023.
- [85] Y. Dong, C. Chen, P.-G. Martinsson, and K. J. Pearce. Robust blockwise random pivoting: Fast and accurate adaptive interpolative decomposition, 2023. arXiv preprint arXiv:2309.16002.
- [86] Y. Dong and P.-G. Martinsson. Simpler is better: a comparative study of randomized pivoting algorithms for CUR and interpolative decompositions. Advances in Computational Mathematics, 49(66), Aug 2023.
- [87] J. Dongarra, J. Du Croz, S. Hammarling, and I. Duff. A set of level 3 basic linear algebra subprograms. ACM Transactions on Mathematical Software, 16(1):1–28, Mar. 1990.
- [88] P. Drineas, R. Kannan, and M. W. Mahoney. Fast Monte Carlo algorithms for matrices I: Approximating matrix multiplication. SIAM Journal on Computing, 36(1):132–157, 2006.
- [89] P. Drineas, R. Kannan, and M. W. Mahoney. Fast Monte Carlo algorithms for matrices II: Computing a Low-Rank Approximation to a Matrix. SIAM Journal on Computing, 36(1):158–183, 2006.
- [90] P. Drineas, R. Kannan, and M. W. Mahoney. Fast Monte Carlo algorithms for matrices III: Computing a Compressed Approximate Matrix Decomposition. SIAM Journal on Computing, 36(1):184–206, 2006.
- [91] P. Drineas, M. Magdon-Ismail, M. W. Mahoney, and D. P. Woodruff. Fast approximation of matrix coherence and statistical leverage. Journal of Machine Learning Research, 13(1):3475–3506, 2012.
- [92] P. Drineas and M. W. Mahoney. A Randomized Algorithm for a Tensor-Based Generalization of the Singular Value Decomposition. Linear Algebra and its Applications, 420(2-3):553–571, 2007.
- [93] P. Drineas, M. W. Mahoney, and S. Muthukrishnan. Sampling algorithms for ℓ_2 regression and applications. In Proceedings of the seventeenth annual ACM-SIAM Symposium on Discrete Algorithms, pages 1127–1136, 2006.
- [94] J. A. Duersch and M. Gu. Randomized QR with Column Pivoting. SIAM Journal on Scientific Computing, 39(4):C263–C291, 2017.

- [95] C. Eckart and G. Young. The approximation of one matrix by another of lower rank. Psychometrika, 1(3):211–218, Sep 1936.
- [96] L. Eldén and B. Savas. A newton–grassmann method for computing the best multilinear rank- (r_1, r_2, r_3) approximation of a tensor. SIAM Journal on Matrix Analysis and Applications, 31(2):248–271, 2009.
- [97] E. N. Epperly. Fast and forward stable randomized algorithms for linear least-squares problems. SIAM Journal on Matrix Analysis and Applications, 45(4):1782–1804, 2024.
- [98] E. N. Epperly, M. Meier, and Y. Nakatsukasa. Fast randomized least-squares solvers can be just as accurate and stable as classical direct solvers. arXiv preprint:arXiv 2406.03468v2, 2024.
- [99] E. N. Epperly and J. A. Tropp. Efficient error and variance estimation for randomized matrix computations. SIAM Journal on Scientific Computing, 46(1):A508–A528, 2024.
- [100] N. B. Erichson, K. Manohar, S. L. Brunton, and J. N. Kutz. Randomized CP tensor decomposition. Machine Learning: Science and Technology, 1(2):025012, May 2020.
- [101] K. Fonal and R. Zdunek. Distributed and randomized tensor train decomposition for feature extraction. In 2019 International Joint Conference on Neural Networks (IJCNN), pages 1–8, 2019.
- [102] M. Fornace and M. Lindsey. Column and row subset selection using nuclear scores: algorithms and theory for Nyström approximation, CUR decomposition, and graph Laplacian reduction. arXiv preprint arXiv:2407.01698, 2024.
- [103] S. Friedland, V. Mehrmann, A. Miedlar, and M. Nkengla. Fast low rank approximations of matrices and tensors. Electronic Journal of Linear Algebra, 22:1031–1048, 2011.
- [104] S. Friedland, A. Niknejad, M. Kaveh, and H. Zare. Fast monte-carlo low rank approximations for matrices. 2006 IEEE/SMC International Conference on System of Systems Engineering, page 6 pp., 2005.
- [105] S. Friedland and V. Tammali. Low-rank approximation of tensors. In P. Benner, M. Bollhöfer, D. Kressner, C. Mehl, and T. Stykel, editors, Numerical Algebra, Matrix Theory, Differential-Algebraic Equations and Control Theory, pages 377–411. Springer, Cham, 2015.
- [106] A. M. Frieze, R. Kannan, and S. Vempala. Fast Monte-Carlo algorithms for finding low-rank approximations. Journal of the ACM, 51:1025–1041, 2004.
- [107] T. Fukaya, R. Kannan, Y. Nakatsukasa, Y. Yamamoto, and Y. Yanagisawa. Shifted cholesky qr for computing the qr factorization of ill-conditioned matrices. SIAM Journal on Scientific Computing, 42(1):A477–A503, 2020.
- [108] K. Gallivan, W. Jalby, and U. Meier. The Use of BLAS3 in Linear Algebra on a Parallel Processor with a Hierarchical Memory. SIAM Journal on Scientific and Statistical Computing, 8(6):1079–1084, 1987.
- [109] S. Ghosh and P. Rigollet. Sparse multi-reference alignment: Phase retrieval, uniform uncertainty principles and the beltway problem. Found. Comput. Math., 23(5):1851–1898, Aug. 2022.
- [110] G. H. Golub and C. Reinsch. Singular Value Decomposition and Least Squares Solutions. Numerische Mathematik, 14:403–420, 1970.

- [111] G. H. Golub and C. F. Van Loan. Matrix Computations. JHU Press, 2013.
- [112] L. Grasedyck, D. Kressner, and C. Tobler. A literature survey of low-rank tensor approximation techniques. GAMM-Mitteilungen, 36(1):53–78, 2013.
- [113] S. Gratton and D. Tittley-Peloquin. Improved bounds for small-sample estimation. SIAM Journal on Matrix Analysis and Applications, 39(2):922–931, 2018.
- [114] L. Grigori and E. Timsit. Randomized Householder QR. arXiv Preprint, arXiv:2405.10923, 2024.
- [115] M. Gu and S. C. Eisenstat. Efficient algorithms for computing a strong rank-revealing QR factorization. SIAM Journal on Scientific Computing, 17(4):848–869, 1996.
- [116] M. Gu and S. C. Eisenstat. Efficient Algorithms for Computing a Strong Rank-Revealing QR Factorization. SIAM J. Sci. Comput., 17(4):848–869, 1996.
- [117] S. Güttel and M. Schweitzer. Randomized Sketching for Krylov Approximations of Large-Scale Matrix Functions. SIAM Journal on Matrix Analysis and Applications, 44(3):1073–1095, 2023.
- [118] W. Hackbusch. Tensor Spaces and Numerical Tensor Calculus. Springer, Cham, 2nd edition, 2019.
- [119] N. Halko, P.-G. Martinsson, Y. Shkolnisky, and M. Tygert. An Algorithm for the Principal Component Analysis of Large Data Sets. SIAM Journal on Scientific Computing, 33(5):2580–2594, 2011.
- [120] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. SIAM Review, 53(2):217–288, 2011.
- [121] E. Hallman, I. C. F. Ipsen, and A. K. Saibaba. Monte carlo methods for estimating the diagonal of a real symmetric matrix. SIAM Journal on Matrix Analysis and Applications, 44(1):240–269, 2023.
- [122] K. Hamm and L. Huang. Stability of sampling for CUR decompositions. Foundations of Data Science, 2:83, 2020.
- [123] Z. Hao, L. He, B. Chen, and X. Yang. A linear support higher-order tensor machine for classification. IEEE Transactions on Image Processing, 22(7):2911–2920, 2013.
- [124] R. A. Harshman. Foundations of the PARAFAC Procedure: Models and Conditions for an “Explanatory” Multimodal Factor Analysis. UCLA Working Papers in Phonetics, 16:1–84, 1970.
- [125] R. A. Harshman and M. E. Lundy. Data preprocessing and the extended parafac model. In H. G. Law, C. W. Snyder, J. Hattie, and R. K. McDonald, editors, Research Methods for Multi-mode Data Analysis, pages 216–284. Praeger Publishers, New York, 1984.
- [126] B. Hashemi and Y. Nakatsukasa. RTSMS: Randomized Tucker with Single-Mode Sketching. Electronic Transactions on Numerical Analysis, 63:247–280, 2025. Accepted manuscript; arXiv preprint arXiv:2311.14873.
- [127] H. He, D. Kressner, and B. Plestenjak. Randomized methods for computing joint eigenvalues, with applications to multiparameter eigenvalue problems and root finding. Numerical Algorithms, 100(3):861–892, 2025.
- [128] N. Heavner, F. D. Igual, G. Quintana-Ortí, and P.-G. Martinsson. Algorithm 1022: Efficient Algorithms for Computing a Rank-Revealing UTV Factoriza-

- tion on Parallel Computing Architectures. ACM Transactions on Mathematical Software, 48(2):1–42, May 2022.
- [129] M. R. Henzinger, A. Heydon, M. Mitzenmacher, and M. Najork. Measuring index quality using random walks on the web. Computer Networks, 31(11–16):1291–1303, 1999.
- [130] M. R. Hestenes and E. L. Stiefel. Methods of Conjugate Gradients for Solving Linear Systems. Journal of Research of the National Bureau of Standards, 49(6):409–436, 1952.
- [131] N. J. Higham. Exploiting fast matrix multiplication within the level 3 blas. ACM Transactions on Mathematical Software, 16(4):352–368, Dec. 1990.
- [132] F. L. Hitchcock. The expression of a tensor or a polyadic as a sum of products. Journal of Mathematics and Physics, 6:164–189, 1927.
- [133] F. L. Hitchcock. Multiple invariants and generalized rank of a p-way matrix or tensor. Journal of Mathematics and Physics, 7:39–79, 1927.
- [134] J. Hough, M. Krishnapur, Y. Peres, and B. Virág. Determinantal processes and independence. Probability Surveys, 3:206–229, 04 2006.
- [135] A. S. Householder. Unitary Triangularization of a Nonsymmetric Matrix. Journal of the ACM, 5:339–242, 1958.
- [136] J. Håstad. Tensor rank is NP-complete. Journal of Algorithms, 11(4):644–654, Dec. 1990.
- [137] B. Huber, R. Schneider, and S. Wolf. A randomized tensor train singular value decomposition. In Compressed Sensing and Its Applications, pages 261–290. Springer, Cham, 2017.
- [138] I. C. F. Ipsen and C. D. Meyer. The Idea Behind Krylov Methods. The American Mathematical Monthly, 105(10):889–899, 1998.
- [139] I. C. F. Ipsen and T. Wentworth. The effect of coherence on sampling from matrices with orthonormal columns, and preconditioned least squares problems. SIAM Journal on Matrix Analysis and Applications, 35(4):1490–1520, 2014.
- [140] I. C. F. Ipsen and R. Wills. Mathematical Properties and Analysis of Google’s PageRank. Boletín de la Sociedad Española de Matemática Aplicada, 34:191–196, 01 2006.
- [141] Y. Jang and L. Grigori. Randomized orthogonalization process with reorthogonalization. Numerical Linear Algebra with Applications, 32(4):e70029, 2025.
- [142] Y. Ji, Q. Wang, X. Li, and J. Liu. A survey on tensor techniques and applications in machine learning. IEEE Access, 7:162950–162990, 2019.
- [143] R. Jin, T. G. Kolda, and R. Ward. Faster Johnson-Lindenstrauss transforms via Kronecker products. Information and Inference: A Journal of the IMA, 10(4):1533–1562, Dec. 2021.
- [144] W. Johnson and J. Lindenstrauss. Extensions of Lipschitz maps into a Hilbert space. Contemporary Mathematics, 26:189–206, 01 1984.
- [145] W. Kahan. Numerical linear algebra. Canadian Mathematical Bulletin, 9(5):757–801, 1966.
- [146] Z. Kam. The reconstruction of structure from electron micrographs of randomly oriented particles. Journal of Theoretical Biology, 82(1):15–39, 1980.
- [147] D. M. Kane and J. Nelson. Sparser johnson-lindenstrauss transforms. J. ACM, 61(1), Jan. 2014.

- [148] R. Kannan and S. Vempala. Randomized algorithms in numerical linear algebra. Acta Numerica, 26:95–135, 2017.
- [149] A. Kapteyn, H. Neudecker, and T. Wansbeek. An approach to n-mode components analysis. Psychometrika, 51(2):269–275, 1986.
- [150] V. Khoromskaia and B. N. Khoromskij. Tensor numerical methods in quantum chemistry: from hartree-fock to excitation energies. Physical Chemistry, Chemical Physics: PCCP 17, 47:31491–509, 2015.
- [151] B. N. Khoromskij. Tensor numerical methods for multidimensional PDEs: Theoretical analysis and initial applications. In CEMRACS 2013 – Modelling and Simulation of Complex Systems: Stochastic and Deterministic Approaches, volume 48 of ESAIM: Proceedings and Surveys, pages 1–28. EDP Sciences, Les Ulis, France, 2015.
- [152] Y.-D. Kim and S. Choi. Nonnegative tucker decomposition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 1–8. IEEE, 2007.
- [153] T. G. Kolda. Orthogonal tensor decompositions. SIAM Journal on Matrix Analysis and Applications, 23(1):243–255, 2001.
- [154] T. G. Kolda. A counterexample to the possibility of an extension of the eckart-young low-rank approximation theorem for the orthogonal rank tensor decomposition. SIAM Journal on Matrix Analysis and Applications, 24(3):762–767, 2003.
- [155] T. G. Kolda. Multilinear operators for higher-order decompositions. Technical Report SAND2006-2081, Sandia National Laboratories, Albuquerque, NM, Livermore, CA, 2006.
- [156] T. G. Kolda. Symmetric orthogonal tensor decomposition is trivial. arXiv preprint, arXiv:1503.01375, 2015.
- [157] T. G. Kolda and B. W. Bader. Tensor Decompositions and Applications. SIAM Review, 51(3):455–500, 2009.
- [158] T. G. Kolda and D. Hong. Stochastic gradients for large-scale tensor decomposition. SIAM Journal on Mathematics of Data Science, 2(4):1066–1095, October 2020.
- [159] W. Kong and G. Valiant. Spectrum estimation from samples. Annals of Statistics, 45(5):2218–2247, 2017.
- [160] D. Kressner and L. Periša. Recompression of hadamard products of tensors in tucker format. SIAM Journal on Scientific Computing, 39(5):A1879–A1902, 2017.
- [161] D. Kressner and B. Plestenjak. Analysis of eigenvalue condition numbers for a class of randomized numerical methods for singular matrix pencils. BIT Numerical Mathematics, 64(32), 2024.
- [162] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1, NIPS’12, page 1097–1105, Red Hook, NY, USA, 2012. Curran Associates Inc.
- [163] P. M. Kroonenberg and J. De Leeuw. Principal component analysis of three-mode data by means of alternating least squares algorithms. Psychometrika, 45(1):69–97, 1980.

- [164] J. B. Kruskal. Three-way arrays: Rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. Linear Algebra and its Applications, 18:95–138, 1977.
- [165] J. B. Kruskal. Rank, decomposition, and uniqueness for 3-way and n-way arrays. In R. Coppi and S. Bolasco, editors, Multiway Data Analysis, pages 7–18. North-Holland, Amsterdam, 1989.
- [166] A. Kulesza and B. Taskar. Determinantal point processes for machine learning. Foundations and Trends in Machine Learning, 5(2–3):123–286, 2012.
- [167] J. Lacotte and M. Pilanci. Faster least squares optimization. arXiv preprint arXiv:1911.02675, 2021.
- [168] B. W. Larsen and T. G. Kolda. Practical Leverage-Based Sampling for Low-Rank Tensor Decomposition. SIAM Journal on Matrix Analysis and Applications, 43(3):1488–1517, 2022.
- [169] D. Leibovici and R. Sabatier. A singular value decomposition of a k-way array for a principal component analysis of multiway data, PTA-k. Linear Algebra and its Applications, 269:307–329, 1998.
- [170] E. Levin, T. Bendory, N. Boumal, J. Kileel, and A. Singer. 3D ab initio modeling in cryo-EM by autocorrelation analysis. In 2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018), pages 1569–1573. IEEE, 2018.
- [171] Z. Leyk and H. Woźniakowski. Estimating a largest eigenvector by Lanczos and polynomial algorithms with a random start. Numerical Linear Algebra with Applications, 5(3):147–164, 1998.
- [172] H. Li and S. Yin. Single-pass randomized algorithms for lu decomposition. Linear Algebra and Its Applications, 595:101–122, 2020.
- [173] L. Li, W. Yu, and K. Batselier. Faster tensor train decomposition for sparse data. Journal of Computational and Applied Mathematics, 405:113972, 2022.
- [174] Y. Li, H. L. Nguyen, and D. P. Woodruff. On Sketching Matrix Norms and the Top Singular Vector. In Proceedings of the 2014 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 1562–1581, 2014.
- [175] E. Liberty, F. Woolfe, P.-G. Martinsson, V. Rokhlin, and M. Tygert. Randomized algorithms for the low-rank approximation of matrices. Proceedings of the National Academy of Sciences, 104(51):20167–20172, 2007.
- [176] L. Lin, Y. Saad, and C. Yang. Approximating spectral densities of large matrices. SIAM Review, 58(1):34–65, 2016.
- [177] C. Lubich, I. V. Oseledets, and B. Vandereycken. Time integration of tensor trains. SIAM Journal on Numerical Analysis, 53(2):917–941, 2015.
- [178] L. Ma and E. Solomonik. Fast and accurate randomized algorithms for low-rank tensor decompositions. In Advances in Neural Information Processing Systems 34 (NeurIPS 2021), pages 24299–24312, 2021.
- [179] M. W. Mahoney. Randomized algorithms for matrices and data. Foundations and Trends in Machine Learning, 3(2):123–224, 2011.
- [180] M. W. Mahoney, M. Maggioni, and P. Drineas. Tensor-CUR decompositions for tensor-based data. SIAM Journal on Matrix Analysis and Applications, 30(3):957–987, 2008.

- [181] O. A. Malik. More efficient sampling for tensor decomposition with worst-case guarantees. In Proceedings of the 39 th International Conference on Machine Learning, PMLR 162, Baltimore, MD, 2022.
- [182] O. A. Malik and S. Becker. Low-rank tucker decomposition of large tensors using tensorsketch. In Advances in Neural Information Processing Systems, volume 31. Curran Associates, Inc., 2018.
- [183] O. A. Malik and S. Becker. Fast randomized matrix and tensor interpolative decomposition using CountSketch. Advances in Computational Mathematics, 46(76):1–28, 2020.
- [184] O. A. Malik and S. Becker. Guarantees for the kronecker fast johnson–lindenstrauss transform using a coherence and sampling argument. Linear Algebra and its Applications, 602:120–137, 2020.
- [185] J. Marot, C. Fossati, and S. Bourennane. About advances in tensor data denoising methods. EURASIP Journal on Advances in Signal Processing, page 235357, 2008.
- [186] P.-G. Martinsson. Randomized methods for matrix computations. The Mathematics of Data, 25(4):187–231, 2019.
- [187] P.-G. Martinsson, V. Rokhlin, and M. Tygert. On interpolation and integration in finite-dimensional spaces of bounded functions. Communications in Applied Mathematics and Computational Science, pages 133–142, 2006.
- [188] P.-G. Martinsson, V. Rokhlin, and M. Tygert. A randomized algorithm for the decomposition of matrices. Applied and Computational Harmonic Analysis, 30(1):47–68, 2011.
- [189] P.-G. Martinsson and J. A. Tropp. Randomized numerical linear algebra: Foundations and algorithms. Acta Numerica, 29:403–572, 2020.
- [190] P.-G. Martinsson and S. Voronin. A randomized blocked algorithm for efficiently computing rank-revealing factorizations of matrices. SIAM Journal on Scientific Computing, 38(5):S485–S507, 2016.
- [191] M. Meier, Y. Nakatsukasa, A. Townsend, and M. Webb. Are sketch-and-precondition least squares solvers numerically stable? SIAM Journal on Matrix Analysis and Applications, 45(2):905–929, 2024.
- [192] X. Meng and M. W. Mahoney. Low-distortion subspace embeddings in input-sparsity time and applications to robust linear regression. In Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC ’13), pages 91–100. ACM, 2013.
- [193] N. Metropolis. The beginning of the Monte Carlo method. Los Alamos Science, 1987 Special Issue Dedicated to Stanislaw Ulam:125–130, 1987.
- [194] N. Metropolis and S. Ulam. The Monte Carlo Method. J. Amer. Statist. Assoc., 44:335–341, 1949.
- [195] R. Minster, Z. Li, and G. Ballard. Parallel randomized tucker decomposition algorithms. SIAM Journal on Scientific Computing, 46(2):A1186–A1213, 2024.
- [196] R. Minster, A. K. Saibaba, and M. E. Kilmer. Randomized Algorithms for Low-Rank Tensor Decompositions in the Tucker Format. SIAM Journal on Mathematics of Data Science, 2(1):189–215, 2020.

- [197] S. Miron, Y. Zniyed, R. Boyer, A. Lima Ferrer de Almeida, G. Favier, D. Brie, and P. Comon. Tensor methods for multisensor signal processing. IET Signal Processing, 14(10):693–709, 2020.
- [198] M. Mørup, L. K. Hansen, and S. M. Arnfred. Algorithms for sparse nonnegative tucker decompositions. Neural Computation, 20(8):2112–2131, Aug. 2008.
- [199] R. Murray, J. Demmel, M. W. Mahoney, N. B. Erichson, M. Melnichenko, O. A. Malik, L. Grigori, P. Luszczek, M. Derezinski, M. E. Lopes, T. Liang, H. Luo, and J. Dongarra. Randomized numerical linear algebra: A perspective on the field with an eye to software. Technical Report UCB/EECS-2023-19, Electrical Engineering & Computer Sciences Department, University of California, Berkeley, February 2023.
- [200] C. Musco and C. Musco. Randomized block Krylov methods for stronger and faster approximate singular value decomposition. In Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1, NIPS’15, page 1396–1404, Cambridge, MA, USA, 2015. MIT Press.
- [201] C. Musco and C. Musco. Recursive sampling for the Nyström method. In Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17, page 3836–3848, Red Hook, NY, USA, 2017. Curran Associates Inc.
- [202] S. Na, M. Dereziński, and M. W. Mahoney. Hessian averaging in stochastic newton methods achieves superlinear convergence. Mathematical Programming, 201(1):473–520, 2023.
- [203] Y. Nakatsukasa and J. A. Tropp. Fast and accurate randomized algorithms for linear systems and eigenvalue problems. SIAM Journal on Matrix Analysis and Applications, 45(2):1183–1214, 2024.
- [204] J. Nelson and H. L. Nguyễn. OSNAP: Faster Numerical Linear Algebra Algorithms via Sparser Subspace Embeddings. In 2013 IEEE 54th Annual Symposium on Foundations of Computer Science, pages 117–126, 2013.
- [205] B. Ordozgoiti, S. G. Canaval, and A. Mozo. Iterative column subset selection. Knowledge and Information Systems, 54(1):65–94, Jan. 2018.
- [206] B. Ordozgoiti, A. Mozo, and J. G. L. de Lacalle. Regularized greedy column subset selection. Information Sciences, 486:393–418, June 2019.
- [207] I. V. Oseledets, D. V. Savostianov, and E. E. Tyrtyshnikov. Tucker dimensionality reduction of three-dimensional arrays in linear time. SIAM Journal on Matrix Analysis and Applications, 30(3):939–956, 2008.
- [208] A. I. Osinsky. Close to optimal column approximation using a single svd. Linear Algebra and its Applications, 725:359–377, 2025.
- [209] I. K. Ozaslan, M. Pilanci, and O. Arikan. Iterative hessian sketch with momentum. In Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 7470–7474, 2019.
- [210] L. Page, S. Brin, R. Motwani, and T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report 1999-66, Stanford University, Computer Science Department, 1998.
- [211] R. Pagh. Compressed matrix multiplication. ACM Transactions on Computation Theory, 5(3):9:1–9:17, Aug 2013.

- [212] C. Pan. On the existence and computation of rank-revealing LU factorizations. Linear Algebra and its Applications, 316(1):199–222, 2000.
- [213] Y. Panagakis, J. Kossaifi, G. Chrysos, J. Oldfield, T. Patti, M. Nicolaou, A. Anandkumar, and S. Zafeiriou. Tensor methods in deep learning, pages 1009–1048. Academic Press, 01 2024.
- [214] C. H. Papadimitriou, P. Raghavan, H. Tamaki, and S. Vempala. Latent semantic indexing: A probabilistic analysis. Journal of Computer and System Sciences, 61(2):217–235, 2000.
- [215] E. E. Papalexakis, C. Faloutsos, and N. D. Sidiropoulos. Tensors for data mining and data fusion: Models, applications, and scalable algorithms. ACM Trans. Intell. Syst. Technol., 8(2), Oct. 2016.
- [216] K. J. Pearce, C. Chen, Y. Dong, and P.-G. Martinsson. Adaptive parallelizable algorithms for interpolative decompositions via partially pivoted lu. Numerical Linear Algebra with Applications, 32(1):e70002, 2025.
- [217] C. Peng, C. Lewis, X. Wang, M. Clement, K. Pierce, V. Rishi, F. Pavošević, S. Slattery, J. Zhang, N. Teke, A. Kumar, C. Masteran, A. Asadchev, J. Calvin, and E. Valeev. Massively parallel quantum chemistry: A high-performance research platform for electronic structure. The Journal of Chemical Physics, 153:044120, 07 2020.
- [218] I. Perros, R. Chen, R. Vuduc, and J. Sun. Sparse hierarchical tucker factorization and its application to healthcare. In Proceedings of the 2015 IEEE International Conference on Data Mining (ICDM), ICDM '15, page 943–948, USA, 2015. IEEE Computer Society.
- [219] D. Persson, A. Cortinovis, and D. Kressner. Improved variants of the Hutch++ algorithm for trace estimation. SIAM Journal on Matrix Analysis and Applications, 43(3):1162–1185, 2022.
- [220] D. Persson and D. Kressner. Randomized low-rank approximation of monotone matrix functions. SIAM Journal on Matrix Analysis and Applications, 44(2):894–918, 2023.
- [221] G. Peters and J. H. Wilkinson. On the Stability of Gauss-Jordan Elimination with Pivoting. Commun. ACM, 18(1):20–24, Jan. 1975.
- [222] N. Pham and R. Pagh. Fast and scalable polynomial kernels via explicit feature maps. In Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '13, page 239–247, New York, NY, USA, 2013. Association for Computing Machinery.
- [223] K. Pierce, V. Rishi, and E. Valeev. Robust approximation of tensor networks: Application to grid-free tensor factorization of the coulomb interaction. Journal of Chemical Theory and Computation, 17, 03 2021.
- [224] M. Pilanci and M. J. Wainwright. Iterative hessian sketch: Fast and accurate solution approximation for constrained least-squares. Journal of Machine Learning Research, 17(1):1842–1879, 2016.
- [225] N. Pritchard, T. Park, Y. Nakatsukasa, and P.-G. Martinsson. Fast rank adaptive cur via a recycled small sketch. arXiv preprint arXiv:2509.21963, 2025.
- [226] G. Quintana-Ortí, X. Sun, and C. H. Bischof. A BLAS-3 Version of the QR Factorization with Column Pivoting. SIAM Journal on Scientific Computing, 19(5):1486–1494, 1998.

- [227] M. J. Reynolds, A. Doostan, and G. Beylkin. Randomized Alternating Least Squares for Canonical Tensor Decompositions: Application to a PDE with Random Data. SIAM Journal on Scientific Computing, 38(5):A2634–A2664, 2016.
- [228] E. Robeva. Orthogonal decomposition of symmetric tensors. SIAM Journal on Matrix Analysis and Applications, 37(1):86–102, 2016.
- [229] V. Rokhlin, A. Sztam, and M. Tygert. A randomized algorithm for principal component analysis. SIAM Journal on Matrix Analysis and Applications, 31(3):1100–1124, Aug. 2009.
- [230] V. Rokhlin, A. Sztam, and M. Tygert. A randomized algorithm for principal component analysis. SIAM Journal on Matrix Analysis and Applications, 31(3):1100–1124, 2010.
- [231] V. Rokhlin and M. Tygert. A fast randomized algorithm for overdetermined linear least-squares regression. Proceedings of the National Academy of Sciences, 105(36):13212–13217, 2008.
- [232] F. Roosta-Khorasani and U. M. Ascher. Improved bounds on sample size for implicit matrix trace estimators. Foundations of Computational Mathematics, 15(5):1187–1212, 2015.
- [233] Y. Saad. The Origin and Development of Krylov Subspace Methods. Computing in Science & Engineering, 24:28–39, 07 2022.
- [234] Y. Saad and M. H. Schultz. GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems. SIAM Journal on Scientific and Statistical Computing, 7(3):856–869, 1986.
- [235] A. K. Saibaba. HOID: Higher Order Interpolatory Decomposition for Tensors Based on Tucker Representation. SIAM Journal on Matrix Analysis and Applications, 37(3):1223–1249, 2016.
- [236] T. Sarlós. Improved approximation algorithms for large matrices via random projections. In 2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06), pages 143–152. IEEE, 2006.
- [237] R. Schneider. Pseudospectral Divide-and-Conquer for the Generalized Eigenvalue Problem. PhD thesis, University of California San Diego, 2024.
- [238] N. D. Sidiropoulos and R. Bro. On the uniqueness of multilinear decomposition of n-way arrays. Journal of Chemometrics, 14(3):229–239, 2000.
- [239] N. D. Sidiropoulos, L. De Lathauwer, X. Fu, K. Huang, E. E. Papalexakis, and C. Faloutsos. Tensor decomposition for signal processing and machine learning. IEEE Transactions on Signal Processing, 65(13):3551–3582, 2017.
- [240] A. Sietsema, Z. Vural, J. Chapman, Y. Yaniv, and D. Needell. Stratified non-negative tensor factorization. arXiv preprint, 2024. Submitted 27 November 2024.
- [241] V. Simoncini and D. B. Szyld. Theory of inexact krylov subspace methods and applications to scientific computing. SIAM Journal on Scientific Computing, 25(2):454–477, 2003.
- [242] Z. Song, D. P. Woodruff, and P. Zhong. Relative error tensor low rank approximation. In Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 2772–2789. SIAM, 2019.
- [243] D. C. Sorensen and M. Embree. A DEIM Induced CUR Factorization. SIAM J. Sci. Comput., 38(3):A1454–A1482, 2016.

- [244] Y. Sun, Y. Guo, C. Luo, J. A. Tropp, and M. Udell. Low-rank Tucker approximation of a tensor from streaming data. SIAM Journal on Mathematics of Data Science, 2(4):1123–1150, 2020.
- [245] S. Szalay, M. Pfeffer, V. Murg, G. Barcza, F. Verstraete, R. Schneider, and O. Legeza. Tensor product methods and entanglement optimization for *ab initio* quantum chemistry. International Journal of Quantum Chemistry, 115(19):1342–1391, 2015.
- [246] D. Tao, X. Li, W. Hu, S. Maybank, and X. Wu. Supervised tensor learning. In Proceedings of the Fifth IEEE International Conference on Data Mining, ICDM '05, page 450–457, USA, 2005. IEEE Computer Society.
- [247] D. A. Tarzanagh and G. Michailidis. Fast randomized algorithms for t-product based tensor operations and decompositions with applications to imaging data. SIAM Journal on Imaging Sciences, 11(4):2629–2664, 2018.
- [248] P. Tichavský and Z. Koldovský. Algorithms for nonorthogonal approximate joint block diagonalization. In Proceedings of the 20th European Signal Processing Conference (EUSIPCO), pages 2094–2098, Bucharest, Romania, 2012. IEEE.
- [249] P. Tichavský, A. H. Phan, and A. Cichocki. Non-orthogonal tensor diagonalization. Signal Processing, 138:313–320, 2017.
- [250] Top500. Top #1 systems, 2025.
- [251] L. N. Trefethen and D. Bau. Numerical Linear Algebra, volume 181. SIAM, Philadelphia, PA, USA, 2022.
- [252] L. N. Trefethen and R. S. Schreiber. Average-Case Stability of Gaussian Elimination. SIAM Journal on Matrix Analysis and Applications, 11(3):335–360, 1990.
- [253] J. A. Tropp. Improved Analysis of the Subsampled Randomized Hadamard Transform. Advances in Adaptive Data Analysis, 03(01n02):115–126, 2011.
- [254] J. A. Tropp. Improved analysis of the subsampled randomized Hadamard transform. Advances in Adaptive Data Analysis, 3(1-2):115–126, 2011.
- [255] J. A. Tropp. An introduction to matrix concentration inequalities. Foundations and Trends in Machine Learning, 8(1–2):1–230, 2015.
- [256] J. A. Tropp. Randomized block Krylov methods for approximating extreme eigenvalues. Numerische Mathematik, 150(1):217–255, 2022.
- [257] J. A. Tropp. Comparison theorems for the minimum eigenvalue of a random positive-semidefinite matrix. arXiv preprint:arXiv 2501.16578, 2025.
- [258] J. A. Tropp. Analysis of Randomized Block Krylov Methods. Technical report, California Institute of Technology, June 2021.
- [259] J. A. Tropp, A. Yurtsever, M. Udell, and V. Cevher. Practical sketching algorithms for low-rank matrix approximation. SIAM Journal on Matrix Analysis and Applications, 38(4):1454–1485, 2017.
- [260] J. A. Tropp, A. Yurtsever, M. Udell, and V. Cevher. More practical sketching algorithms for low-rank matrix approximation. Technical Report Technical Report No. 2018-01, California Institute of Technology, October 2018. Applied & Computational Mathematics, Caltech.
- [261] J. A. Tropp, A. Yurtsever, M. Udell, and V. Cevher. Streaming low-rank matrix approximation with an application to scientific simulation. SIAM Journal on Scientific Computing, 41(4):A2430–A2463, 2019.

- [262] C. E. Tsourakakis. MACH: Fast randomized tensor decompositions. In Proceedings of the 2010 SIAM International Conference on Data Mining, pages 689–700. SIAM, 2010.
- [263] L. R. Tucker. Implications of factor analysis of three-way matrices for measurement of change. In C. W. Harris, editor, Problems in measuring change, pages 122–137. University of Wisconsin Press, Madison WI, 1963.
- [264] L. R. Tucker. The extension of factor analysis to three-dimensional matrices. In H. Gulliksen and N. Frederiksen, editors, Contributions to Mathematical Psychology, pages 110–127. Holt, Rinehart and Winston, New York, 1964.
- [265] L. R. Tucker. Some mathematical notes on three-mode factor analysis. Psychometrika, 31:279–311, 1966.
- [266] S. Ubaru, J. Chen, and Y. Saad. Fast estimation of $\text{tr}(f(a))$ via stochastic lanczos quadrature. SIAM Journal on Matrix Analysis and Applications, 38(4):1075–1099, 2017.
- [267] S. Ubaru and Y. Saad. Applications of trace estimation techniques. In T. Kozubek, M. Čermák, P. Tichý, R. Blaheta, J. Šístek, D. Lukáš, and J. Jaroš, editors, High Performance Computing in Science and Engineering, pages 19–33. Springer International Publishing, Cham, Switzerland, 2018.
- [268] N. Vannieuwenhoven, R. Vandebril, and K. Meerbergen. A New Truncation Strategy for the Higher-Order Singular Value Decomposition. SIAM Journal on Scientific Computing, 34(2):A1027–A1052, 2012.
- [269] M. A. O. Vasilescu and D. Terzopoulos. Multilinear analysis of image ensembles: Tensorfaces. In A. Heyden, G. Sparr, M. Nielsen, and P. Johansen, editors, Computer Vision — ECCV 2002, pages 447–460, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- [270] N. Vervliet and L. De Lathauwer. A randomized block sampling approach to canonical polyadic decomposition of large-scale tensors. IEEE Journal of Selected Topics in Signal Processing, 10(2):284–295, 2016.
- [271] N. Vervliet, O. Debals, L. Sorber, and L. D. Lathauwer. Breaking the curse of dimensionality using decompositions of incomplete tensors: Tensor-based scientific computing in big data analysis. IEEE Signal Processing Magazine, 31:71–79, 2014.
- [272] J. Von Neumann. Various techniques used in connection with random digits. In National Bureau of Standards symposium, NBS, Applied Mathematics Series 12, pages 36–38. National Bureau of Standards, Washington, D.C., 1951.
- [273] M. Wang, H. Cui, and H. Li. Svd-based algorithms for tensor wheel decomposition. Advances in Computational Mathematics, 50(5), 2024.
- [274] Q. Wang, C. Cui, and D. Han. Accelerated doubly stochastic gradient descent for tensor cp decomposition. Journal of Optimization Theory and Applications, 197(2):665–704, 2023.
- [275] S. Wang, Z. Zhang, and T. Zhang. Improved analyses of the randomized power method and block lanczos method. arXiv.1508.06429, 08 2015.
- [276] Y. Wang, H.-F. Tung, A. J. Smola, and A. Anandkumar. Fast and guaranteed tensor decomposition via sketching. In Advances in Neural Information Processing Systems, pages 991–999, 2015.

- [277] R. Whitman. It Took Half a Ton of Hard Drives to Store the Black Hole Image Data, April 11, 2019.
- [278] D. P. Woodruff. Sketching as a tool for numerical linear algebra. Found. Trends Theor. Comput. Sci., 10(1–2):1–157, Oct. 2014.
- [279] F. Woolfe, E. Liberty, V. Rokhlin, and M. Tygert. A fast randomized algorithm for the approximation of matrices. Applied and Computational Harmonic Analysis, 25(3):335–366, 2008.
- [280] B. Yang, A. Zamzam, and N. D. Sidiropoulos. Parasketch: Parallel tensor factorization via sketching. In Proceedings of the 2018 SIAM International Conference on Data Mining, pages 396–404. SIAM, 2018.
- [281] W. Yu, Y. Gu, J. Li, S. Liu, and Y. Li. Single-pass pca of large high-dimensional data. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17, pages 3350–3356, 2017.
- [282] Y. Yu and H. Li. A block-randomized stochastic method with importance sampling for cp tensor decomposition. Advances in Computational Mathematics, 50(2):1–27, 2024.
- [283] L. Yuan, C. Li, J. Cao, and Q. Zhao. Randomized tensor ring decomposition and its application to large-scale data reconstruction. In ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 2127–2131, 2019.
- [284] Q. Yuan, M. Gu, and B. Li. Superlinear convergence of randomized block lanczos algorithm. In Proceedings of the IEEE International Conference on Data Mining (ICDM 2018), pages 1404–1409, Singapore, 2018. IEEE.
- [285] A. Zhang, O. Mickelin, J. Kileel, E. J. Verbeke, N. F. Marshall, M. A. Gilles, and A. Singer. Moment-based metrics for molecules computable from cryogenic electron microscopy images. Biological Imaging, 4:e3, Feb 2024.
- [286] J. Zhang, A. K. Saibaba, M. E. Kilmer, and S. Aeron. A randomized tensor singular value decomposition based on the t-product. Numerical Linear Algebra with Applications, 25(5):e2179, 2018.
- [287] Y. Zhang, M. Fornace, and M. Lindsey. Fast and accurate interpolative decompositions for general, sparse, and structured tensors. arXiv preprint, arXiv.2503.18921, pages 1–53, 03 2025.
- [288] K. Zhao, M. Vouvakis, and J.-F. Lee. The adaptive cross approximation algorithm for accelerated method of moments computations of emc problems. IEEE Transactions on Electromagnetic Compatibility, 47(4):763–773, 2005.
- [289] S. Zhao, D. P. Woodruff, and P. Zhong. Relative error tensor low rank approximation. arXiv preprint, arXiv 1704.08246, 2018.
- [290] G. Zhou, A. Cichocki, and S. Xie. Decomposition of big tensors with low multilinear rank. arXiv preprint arXiv:1412.1885, 2014.
- [291] G. Zhou, A. Cichocki, Y. Zhang, and D. P. Mandic. Group component analysis for multiblock data: Common and individual feature extraction. IEEE Transactions on Neural Networks and Learning Systems, 27(11):2426–2439, 2016.
- [292] G. Zhou, A. Cichocki, Q. Zhao, and S. Xie. Nonnegative matrix and tensor factorizations: An algorithmic perspective. IEEE Signal Processing Magazine, 31(3):54–65, 2014.

- [293] G. Zhou, A. Cichocki, Q. Zhao, and S. Xie. Efficient nonnegative tucker decompositions: Algorithms and uniqueness. IEEE Transactions on Image Processing, 24(12):4990–5003, dec 2015.